Getting Started

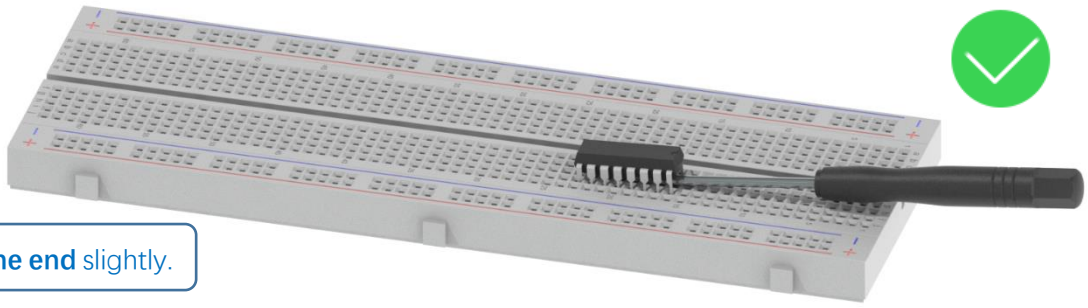
Thank you for choosing Freenove products!
After you download the ZIP file we provide. Unzip it and you will get a folder contains several files and folders. There are three PDF files:

- **Tutorial.pdf**
  It contains basic operations such as installing system for Raspberry Pi.
  The code in this PDF is in C.
- **Tutorial_GPIOZero.pdf**
  It contains basic operations such as installing system for Raspberry Pi.
  The code in this PDF is in Python.
- **Processing.pdf** in Freenove_LCD1602_Starter_Kit_for_Raspberry_Pi\Processing
  The code in this PDF is in Java.

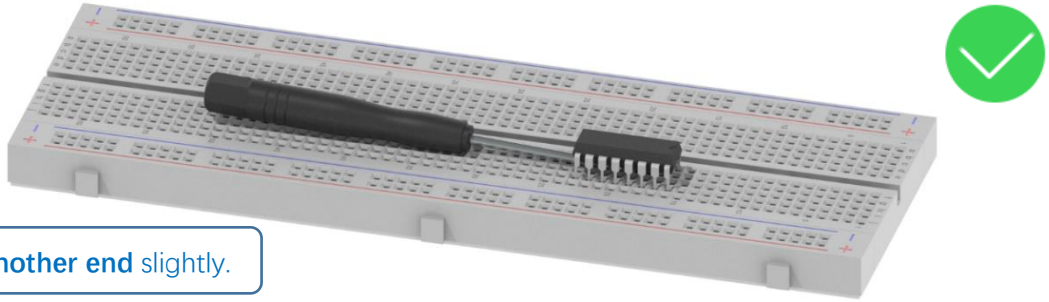
We recommend you to start with **Tutorial.pdf** or **Tutorial_GPIOZero.pdf** first.
If you want to start with Processing.pdf or skip some chapters of Tutorial.pdf, you need to finish necessary steps in **Chapter 7 AD/DA** of **Tutorial.pdf** first.

Remove the Chips

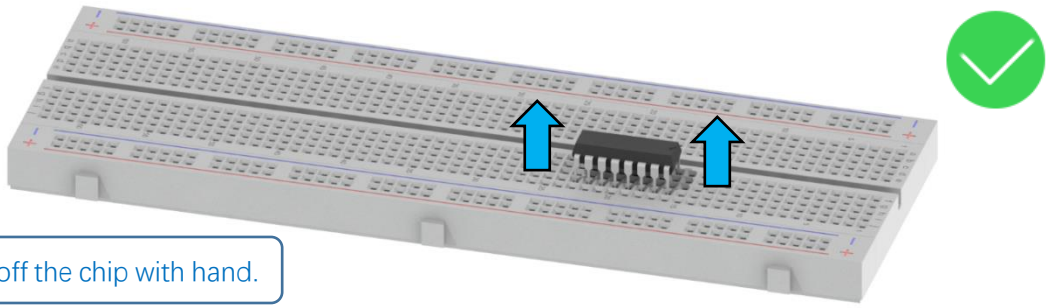
Some chips and modules are inserted into the breadboard to protect their pins.
You need to remove them from breadboard before use. (There is no need to remove GPIO Extension Board.)
Please find a tool (like a little screw driver) to handle them like below:



Step 1, lift **one end** slightly.

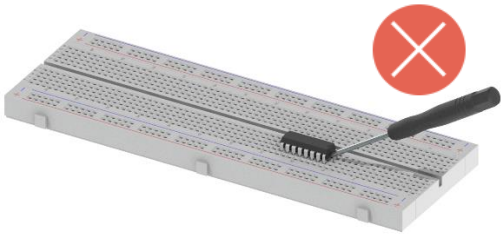


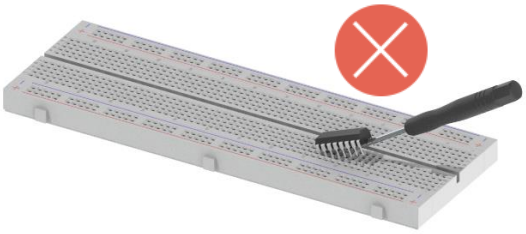
Step 2, lift **another end** slightly.



Step 3, take off the chip with hand.

Avoid lifting one end with big angle directly.





Get Support and Offer Input

Freenove provides free and responsive product and technical support, including but not limited to:

- Product quality issues
- Product use and build issues
- Questions regarding the technology employed in our products for learning and education
- Your input and opinions are always welcome
- We also encourage your ideas and suggestions for new products and product improvements

For any of the above, you may send us an email to:

support@freenove.com

Safety and Precautions

Please follow the following safety precautions when using or storing this product:

- Keep this product out of the reach of children under 6 years old.
- This product should be used only when there is adult supervision present as young children lack necessary judgment regarding safety and the consequences of product misuse.
- This product contains small parts and parts, which are sharp. This product contains electrically conductive parts. Use caution with electrically conductive parts near or around power supplies, batteries and powered (live) circuits.
- When the product is turned ON, activated or tested, some parts will move or rotate. To avoid injuries to hands and fingers, keep them away from any moving parts!
- It is possible that an improperly connected or shorted circuit may cause overheating. Should this happen, immediately disconnect the power supply or remove the batteries and do not touch anything until it

cools down! When everything is safe and cool, review the product tutorial to identify the cause.
- Only operate the product in accordance with the instructions and guidelines of this tutorial, otherwise parts may be damaged or you could be injured.
- Store the product in a cool dry place and avoid exposing the product to direct sunlight.
- After use, always turn the power OFF and remove or unplug the batteries before storing.

About Freenove

Freenove provides open source electronic products and services worldwide.

Freenove is committed to assist customers in their education of robotics, programming and electronic circuits so that they may transform their creative ideas into prototypes and new and innovative products. To this end, our services include but are not limited to:

- Educational and Entertaining Project Kits for Robots, Smart Cars and Drones
- Educational Kits to Learn Robotic Software Systems for Arduino, Raspberry Pi and micro: bit
- Electronic Component Assortments, Electronic Modules and Specialized Tools
- **Product Development and Customization Services**

You can find more about Freenove and get our latest news and updates through our website:

http://www.freenove.com

Copyright

Contents

Preface

Raspberry Pi is a low cost, **credit card sized computer** that plugs into a computer monitor or TV, and uses a standard keyboard and mouse. It is an incredibly capable little device that enables people of all ages to explore computing, and to learn how to program in a variety of computer languages like Scratch and Python. It is capable of doing everything you would expect from a desktop computer, such as browsing the internet, playing high-definition video content, creating spreadsheets, performing word-processing, and playing video games. For more information, you can refer to Raspberry Pi official website. For clarification, this tutorial will also reference Raspberry Pi as RPi, RPI and RasPi.

In this tutorial, most chapters consist of **Components List**, **Component Knowledge**, **Circuit**, and **Code** (**Python** code). We provide Python code for each project in this tutorial. After completing this tutorial, you can learn Java by reading Processing.pdf.

This kit does not contain **Raspberry and its accessories**. You can also use the components and modules in this kit to create projects of your own design.

Additionally, if you encounter any issues or have questions about this tutorial or the contents of kit, you can always contact us for free technical support at:

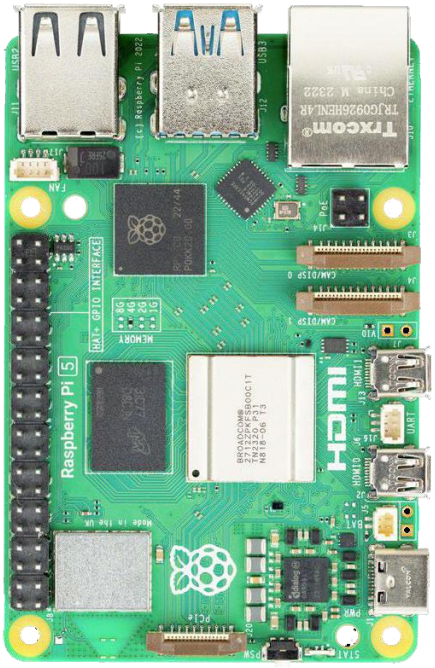
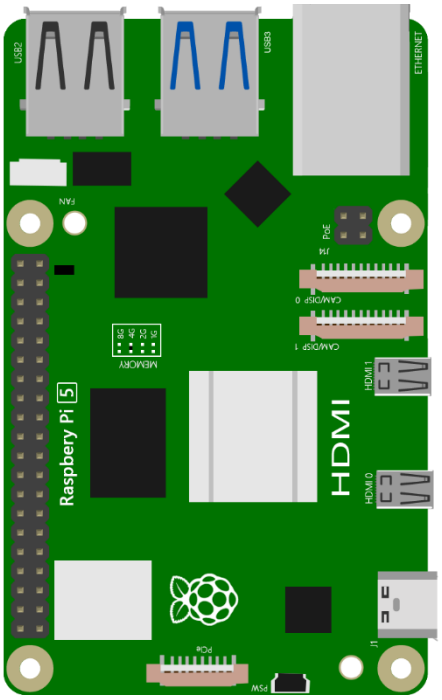
support@freenove.com

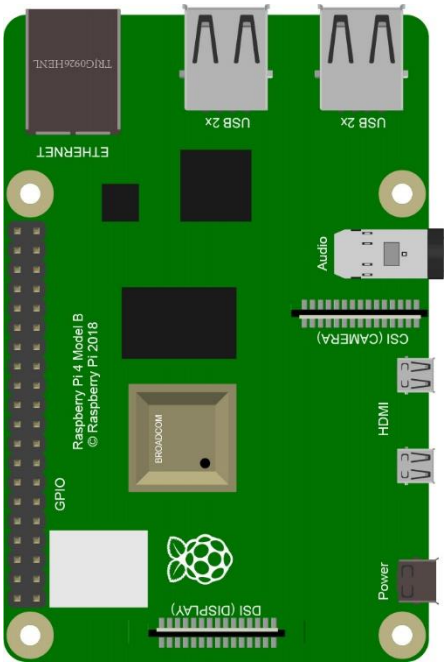
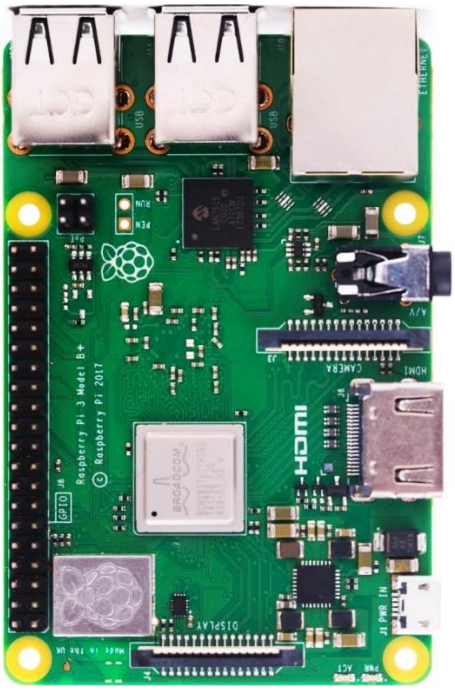
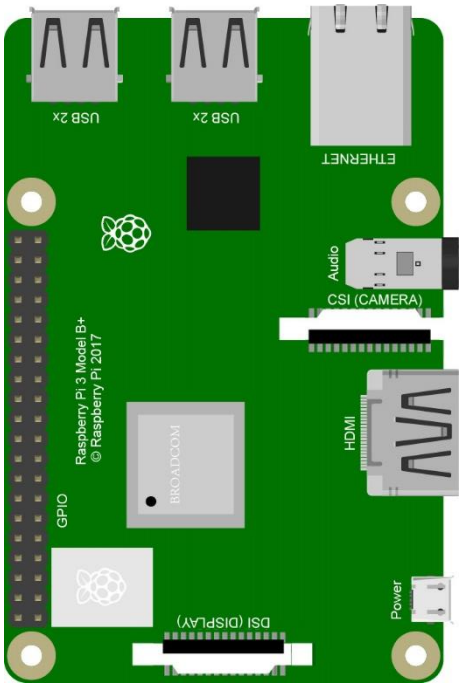
Raspberry Pi

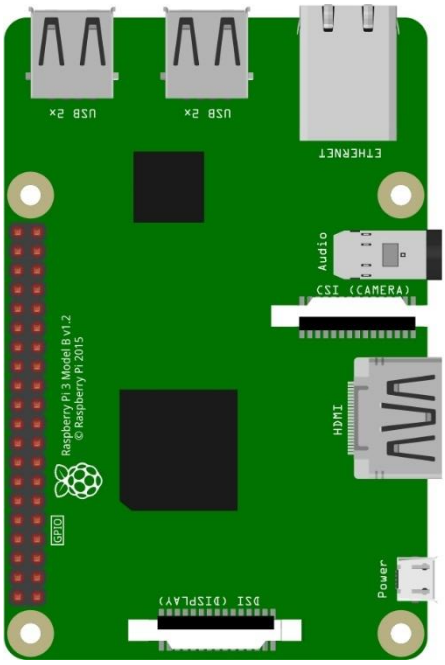
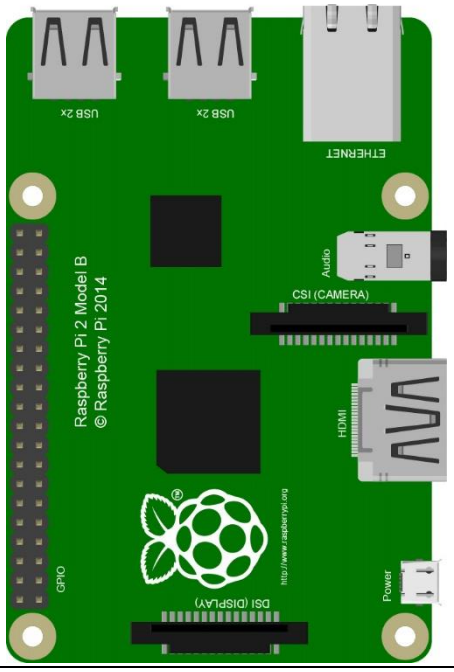
So far, at this writing, Raspberry Pi has advanced to its fifth generation product offering. Version changes are accompanied by increases in upgrades in hardware and capabilities.

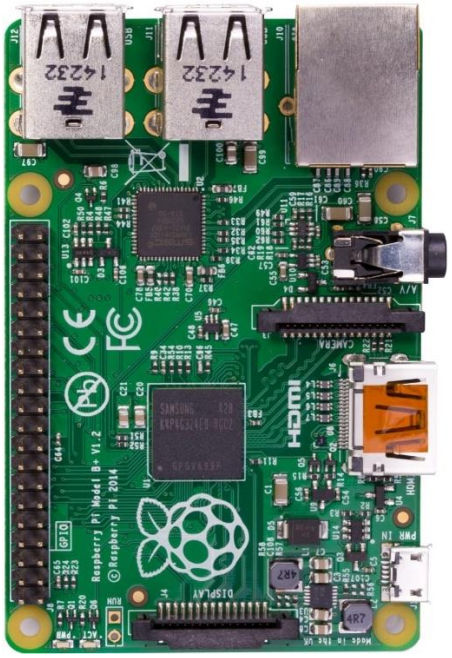
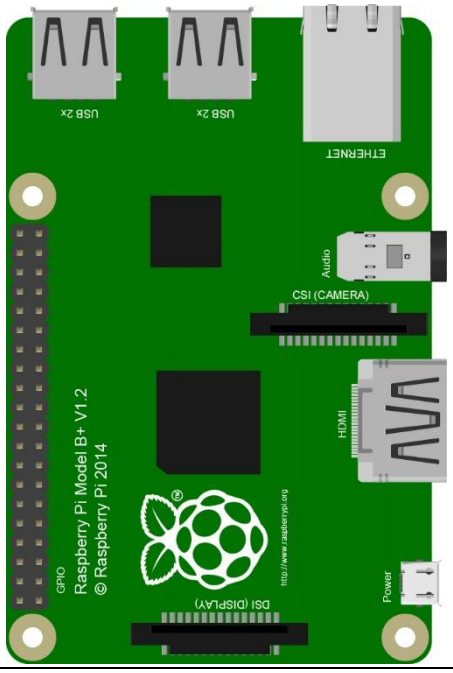
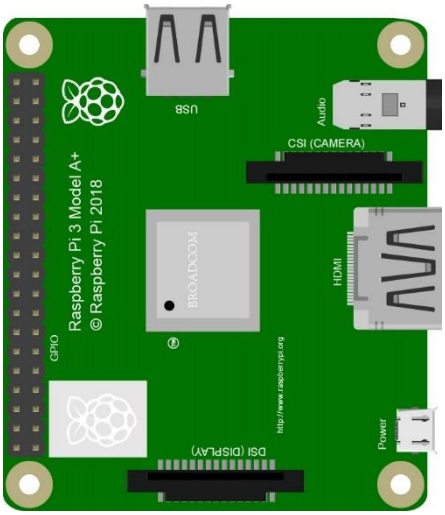
The A type and B type versions of the first generation products have been discontinued due to various reasons. What is most important is that other popular and currently available versions are consistent in the order and number of pins and their assigned designation of function, making compatibility of peripheral devices greatly enhanced between versions.

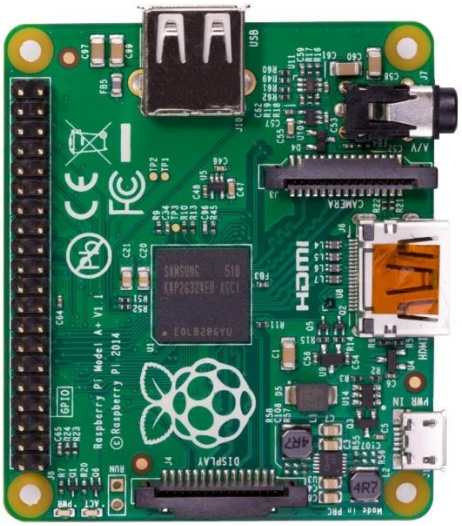
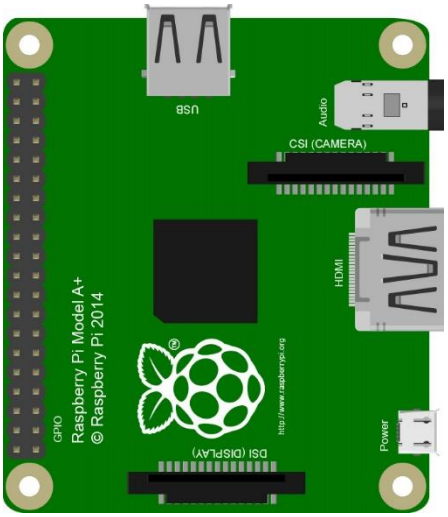
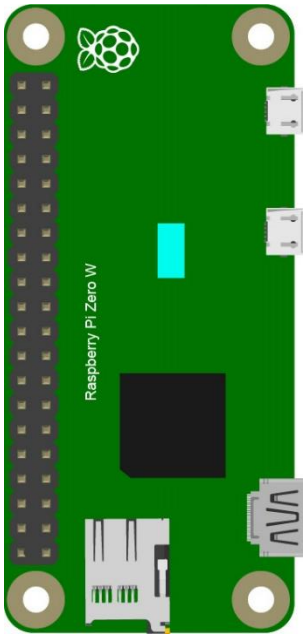
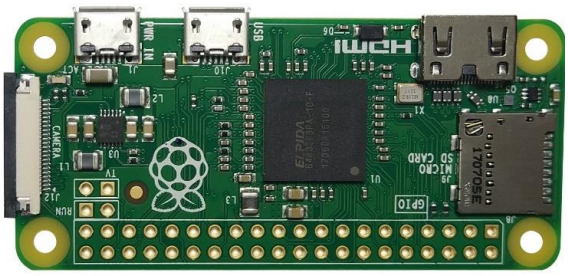
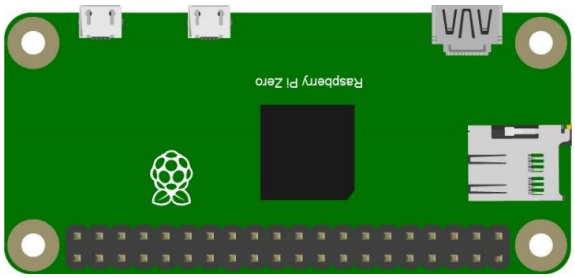
Below are the raspberry pi pictures and model pictures supported by this product. They have 40 pins.

Practicality picture of Raspberry Pi 5:	Model diagram of Raspberry Pi 5:
	

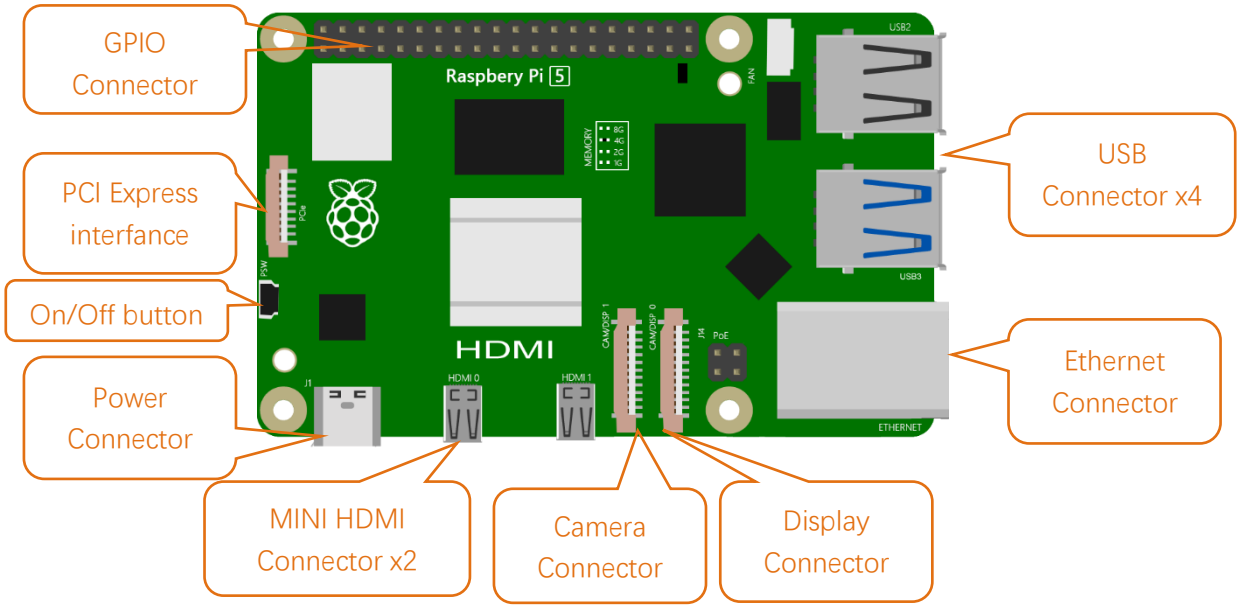
Actual image of Raspberry Pi 4 Model B: 	CAD image of Raspberry Pi 4 Model B: 
Actual image of Raspberry Pi 3 Model B+: 	CAD image of Raspberry Pi 3 Model B+: 

Actual image of Raspberry Pi 3 Model B: 	CAD image of Raspberry Pi 3 Model B: 
Actual image of Raspberry Pi 2 Model B: 	CAD image of Raspberry Pi 2 Model B: 

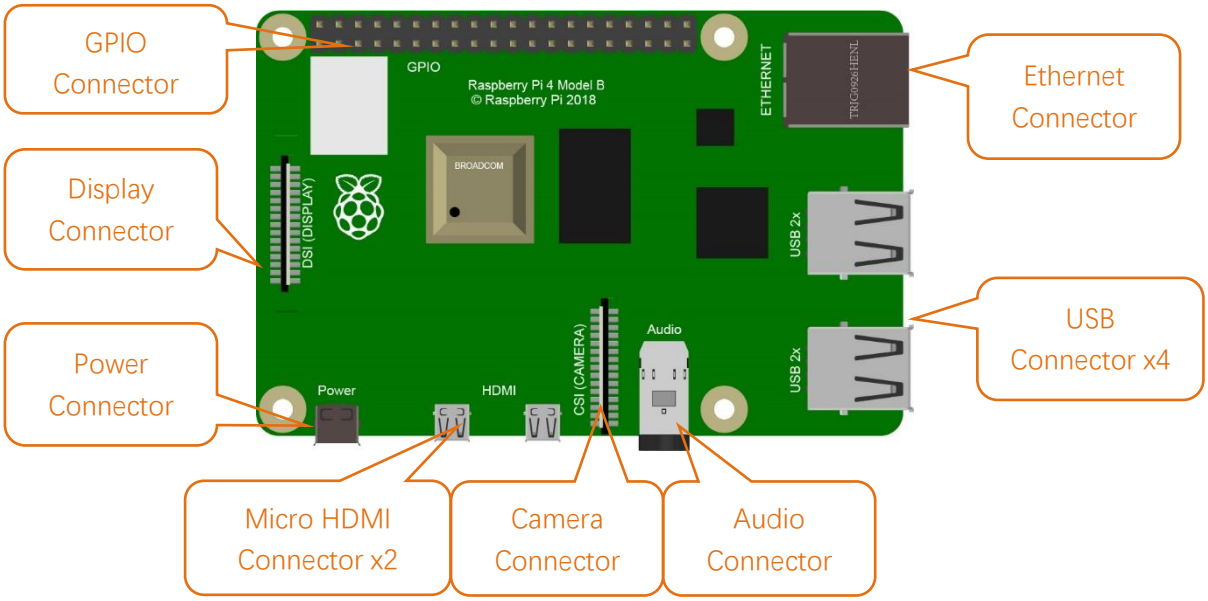
Actual image of Raspberry Pi 1 Model B+: 	CAD image of Raspberry Pi 1 Model B+: 
Actual image of Raspberry Pi 3 Model A+: 	CAD image of Raspberry Pi 3 Model A+: 

Actual image of Raspberry Pi 1 Model A+:	CAD image of Raspberry Pi 1 Model A+:
	
Actual image of Raspberry Pi Zero W:	CAD image of Raspberry Pi Zero W:
	
Actual image of Raspberry Pi Zero:	CAD image of Raspberry Pi Zero:
	

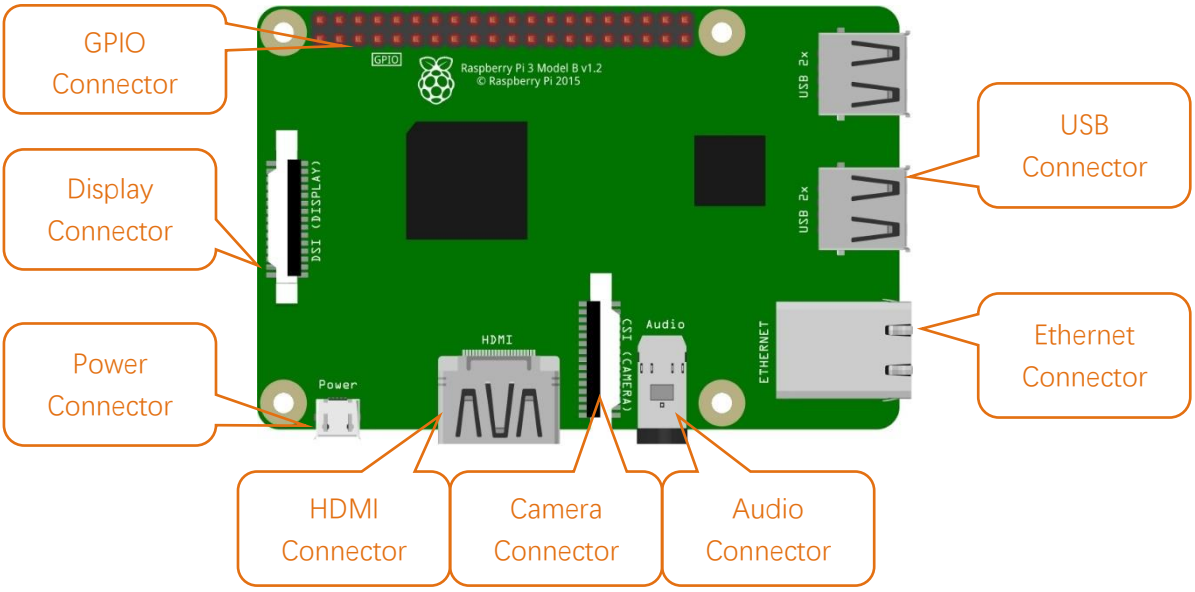
Below are the raspberry pi pictures and model pictures supported by this product. They have 40 pins.
Hardware interface diagram of RPi 5 is shown below:



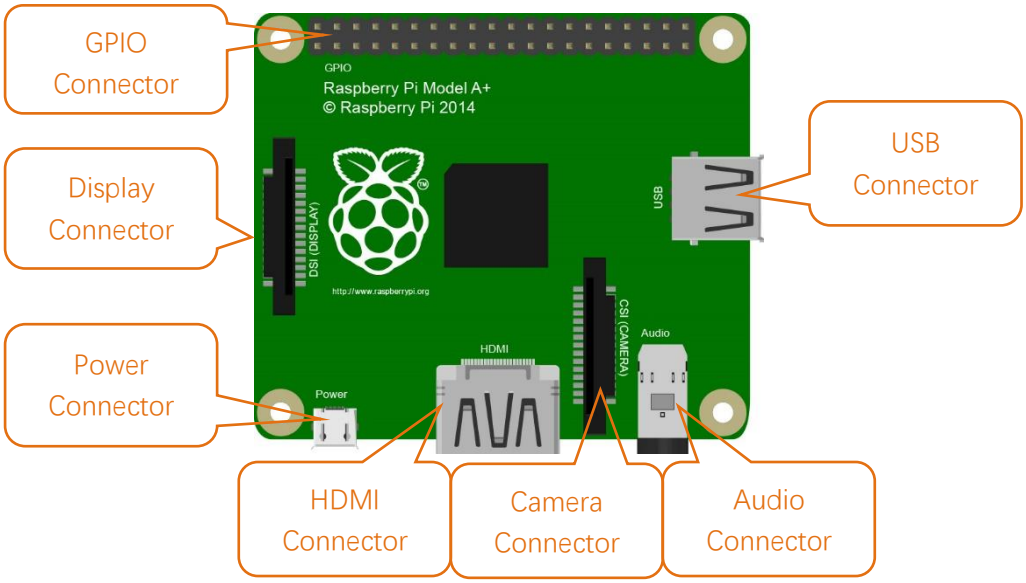
Hardware interface diagram of RPi 4B is shown below:



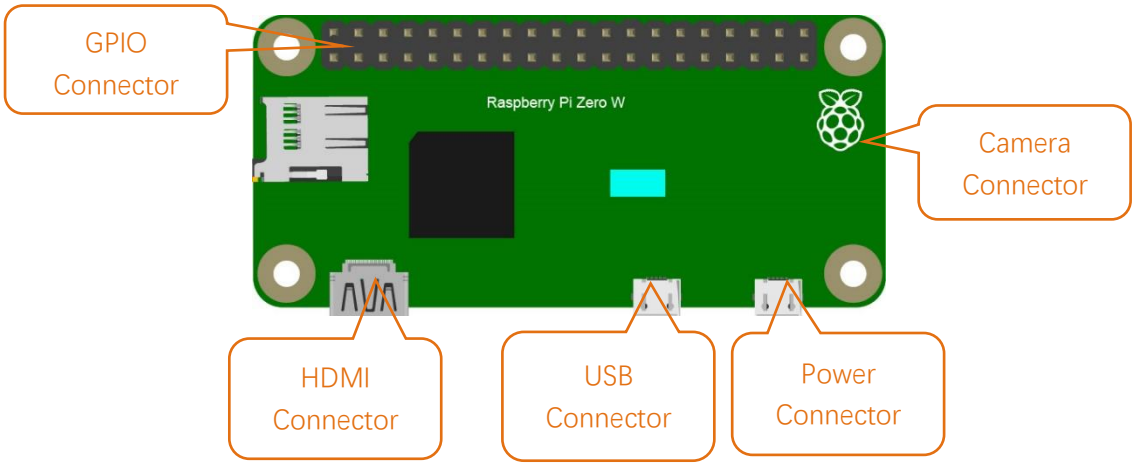
Hardware interface diagram of RPi 3B+/3B/2B/1B+:



Hardware interface diagram of RPi 3A+/A+:



Hardware interface diagram of RPi Zero/Zero W:



Chapter 0 Preparation

Why "Chapter 0"? Because in program code the first number is 0. We choose to follow this rule. In this chapter, we will do some necessary foundational preparation work: Start your Raspberry Pi and install some necessary libraries.

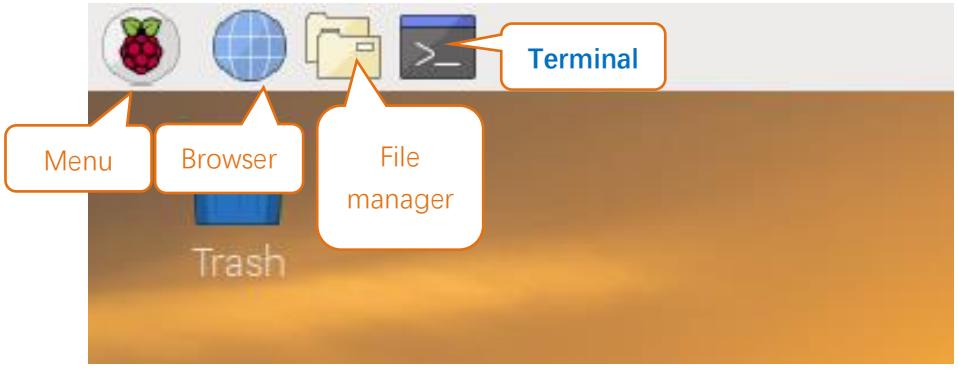
If you haven't installed the system for Raspberry Pi yet, please refer to:

Freenove_LCD1602_Starter_Kit_for_Raspberry_Pi/Installing Raspberry Pi OS.pdf

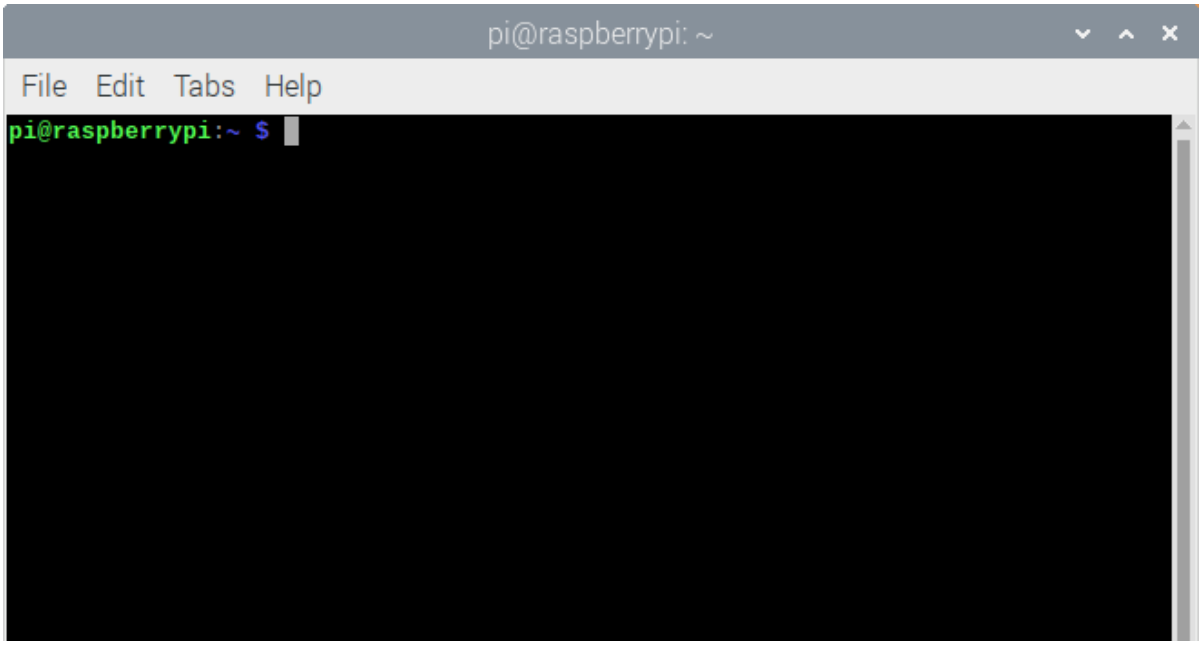
Linux Command

Raspberry Pi OS is based on the Linux Operation System. Now we will introduce you to some frequently used Linux commands and rules.

First, open the Terminal. All commands are executed in Terminal.

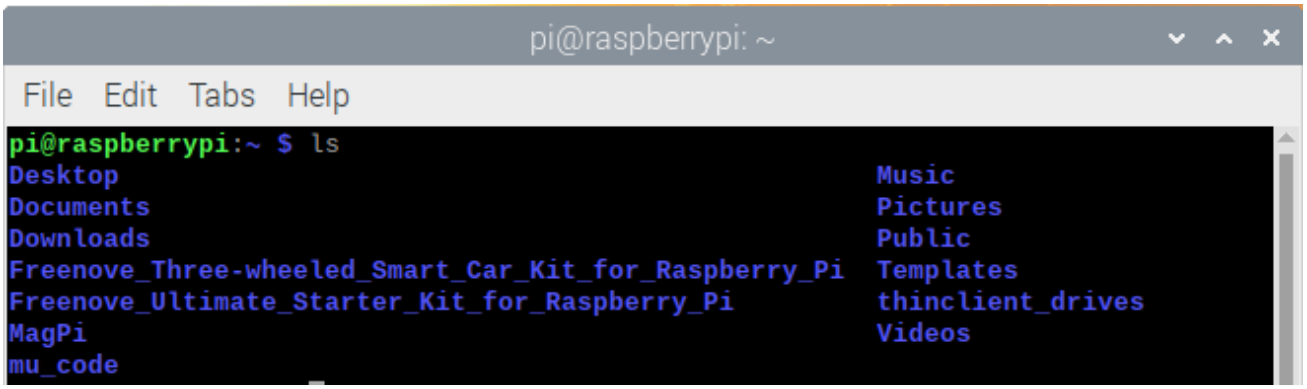


When you click the Terminal icon, following interface appears.



Note: The Linux is case sensitive.

First, type "ls" into the Terminal and press the "Enter" key. The result is shown below:



The "ls" command lists information about the files (the current directory by default).

Content between "$" and "pi@raspberrypi:" is the current working path. "~" represents the user directory, which refers to "/home/pi" here.

```
pi@raspberrypi:~ $ pwd
/home/pi
```

"cd" is used to change directory. "/" represents the root directory.

```
pi@raspberrypi:~ $ cd /usr
pi@raspberrypi:/usr $ ls
bin  games  include  lib  local  man  sbin  share  src
pi@raspberrypi:/usr $ cd ~
pi@raspberrypi:~ $
```

Later in this Tutorial, we will often change the working path. Typing commands under the wrong directory may cause errors and break the execution of further commands.

Many frequently used commands and instructions can be found in the following reference table.

Command	instruction
ls	Lists information about the FILEs (the current directory by default) and entries alphabetically.
cd	Changes directory
sudo + cmd	Executes cmd under root authority
./	Under current directory
gcc	GNU Compiler Collection
git clone URL	Use git tool to clone the contents of specified repository, and URL in the repository address.

There are many commands, which will come later. For more details about commands. You can refer to: http://www.linux-commands-examples.com

Shortcut Key

Now, we will introduce several commonly used shortcuts that are very useful in Terminal.

1. **Up and Down Arrow Keys**: Pressing "↑" (the Up key) will go backwards through the command history and pressing "↓" (the Down Key) will go forwards through the command history.

2. **Tab Key**: The Tab key can automatically complete the command/path you want to type. When there is only one eligible option, the command/path will be completely typed as soon as you press the Tab key even you only type one character of the command/path.

As shown below, under the '~' directory, you enter the Documents directory with the "cd" command. After typing "cd D", pressing the Tab key (there is no response), pressing the Tab key again then all the files/folders that begin with "D" will be listed. Continue to type the letters "oc" and then pressing the Tab key, the "Documents" is typed automatically.

```
pi@raspberrypi:~ $ cd D
Desktop/   Documents/ Downloads/
pi@raspberrypi:~ $ cd Doc
```

```
pi@raspberrypi:~ $ cd D
Desktop/   Documents/ Downloads/
pi@raspberrypi:~ $ cd Documents/
```

Install GPIO Zero Python library

GPIO Zero is a simple interface to GPIO devices with Raspberry Pi. GPIO Zero is installed by default in the Raspberry Pi OS desktop image, and the Raspberry Pi Desktop image for PC/Mac, both available from raspberrypi.org. Follow these guides to installing on Raspberry Pi OS Lite and other operating systems.

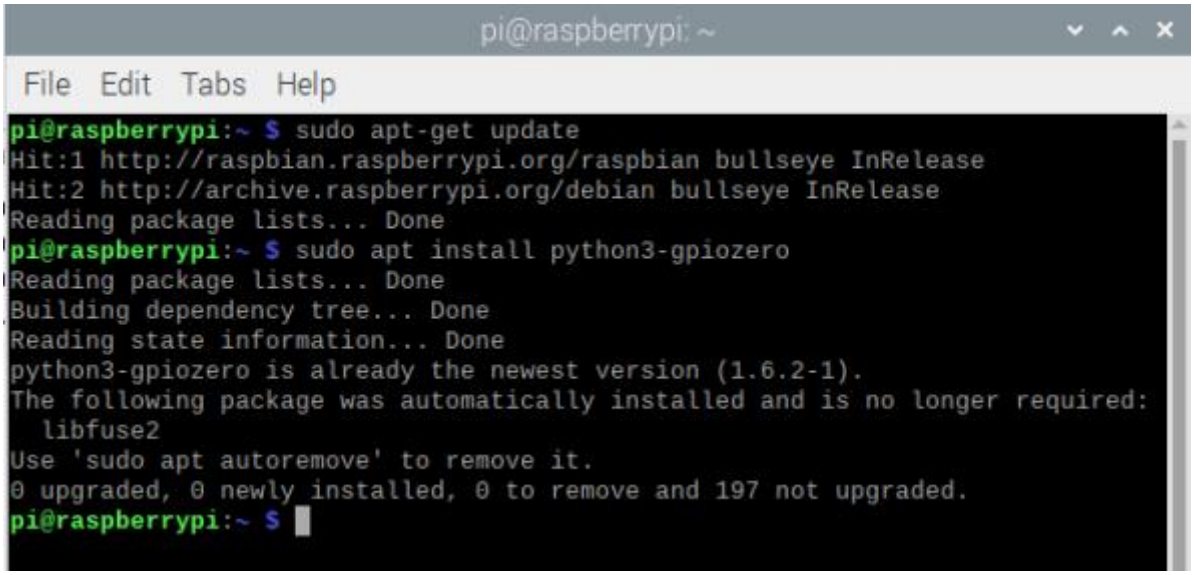
GPIO Zero Python library Installation Steps

To install the GPIO Zero Python library, please open the Terminal and then follow the steps and commands below.

Note: For a command containing many lines, execute them one line at a time.

Enter the following commands **one by one** in the **terminal** to install GPIO Zero:

```
sudo apt-get update
sudo apt install python3-gpiozero
```



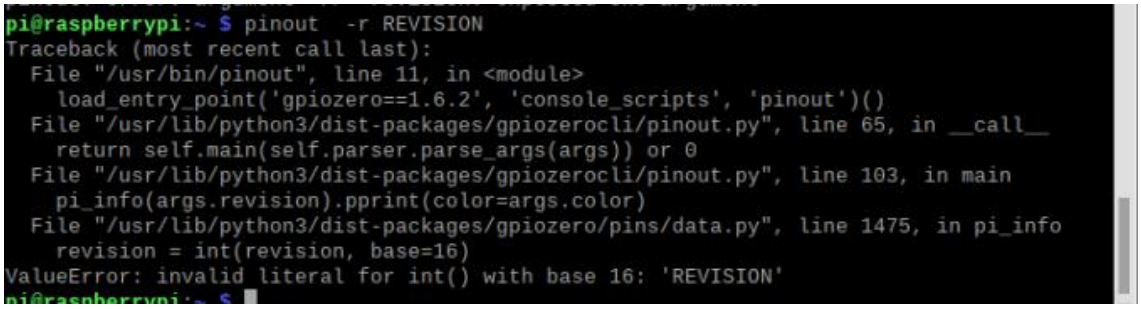
If you're using another operating system on your Raspberry Pi, you may need to use pip to install GPIO Zero instead. Install pip using get-pip and then type:

```
sudo pip3 install gpiozero
```

Run the gpiozero command to check the installation:

```
pinout  -r REVISION
```

That should give you some confidence that the installation was a success.



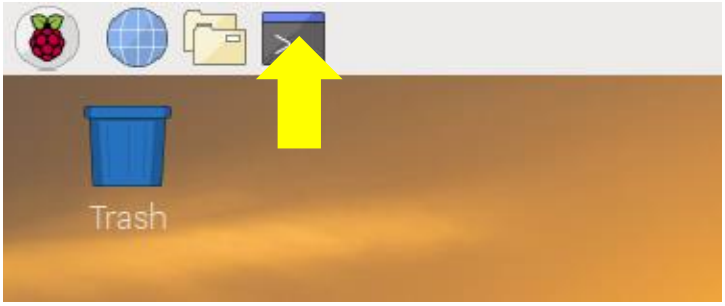
Obtain the Project Code

After the above installation is completed, you can visit our official website (http://www.freenove.com) or our GitHub resources at (https://github.com/freenove) to download the latest available project code.
In this tutorial, we provide Python language code for each project.

This is the method for obtaining the code:
In the pi directory of the RPi terminal, enter the following command.

```
cd
git clone --depth 1 https://github.com/freenove/Freenove_LCD1602_Starter_Kit_for_Raspberry_Pi
```

(There is no need for a password. If you get some errors, please check your commands.)

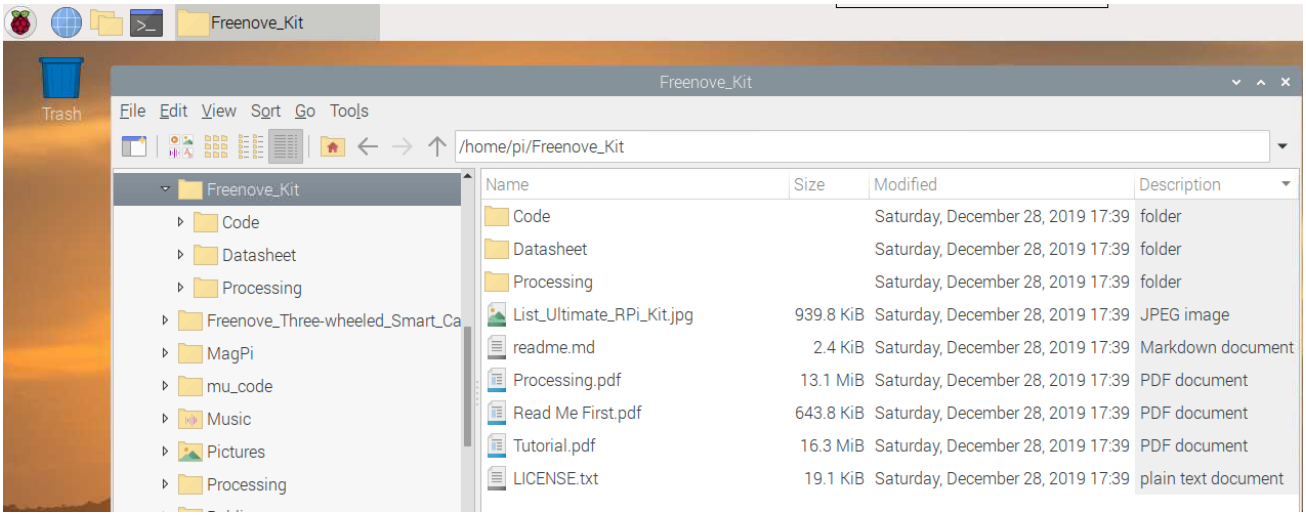


After the download is completed, a new folder "Freenove_LCD1602_Starter_Kit_for_Raspberry_Pi" is generated, which contains all of the tutorials and required code.

This folder name seems a little too long. We can simply rename it by using the following command.

```
mv Freenove_LCD1602_Starter_Kit_for_Raspberry_Pi Freenove_Kit
```

"Freenove_Kit" is now the new and much shorter folder name.



If you have no experience with Python, we suggest that you refer to this website for basic information and knowledge.
https://python.swaroopch.com/basics.html

Python2 & Python3

Python code, used in our kits, can now run on Python2 and Python3. **Python3 is recommend**. If you want to use Python2, please make sure your Python version is 2.7 or above. Python2 and Python3 are not fully compatible. However, Python2.6 and Python2.7 are transitional versions to python3, therefore you can also use Python2.6 and 2.7 to execute some Python3 code.

You can type "python2" or "python3" respectively into Terminal to check if python has been installed. Press Ctrl-Z to exit.

```
pi@raspberrypi:~ $ python2
Python 2.7.18 (default, Jul 14 2021, 08:11:37)
[GCC 10.2.1 20210110] on linux2
Type "help", "copyright", "credits" or "license" for more information.
>>>
[1]+  Stopped                 python2
pi@raspberrypi:~ $ python3
Python 3.9.2 (default, Mar 12 2021, 04:06:34)
[GCC 10.2.1 20210110] on linux
Type "help", "copyright", "credits" or "license" for more information.
>>>
[2]+  Stopped                 python3
pi@raspberrypi:~ $
```

Type "python", and Terminal shows that it links to python3.

```
pi@raspberrypi:~ $ python
Python 3.9.2 (default, Mar 12 2021, 04:06:34)
[GCC 10.2.1 20210110] on linux
Type "help", "copyright", "credits" or "license" for more information.
>>>
[3]+  Stopped                 python
^Xpi@raspberrypi:~ $
```

Set Python3 as default python

First, execute python to check the default python on your raspberry Pi. Press Ctrl-Z to exit.

```
pi@raspberrypi:~ $ python
Python 3.9.2 (default, Mar 12 2021, 04:06:34)
[GCC 10.2.1 20210110] on linux
Type "help", "copyright", "credits" or "license" for more information.
>>>
[3]+  Stopped                 python
^Xpi@raspberrypi:~ $
```

If it is python3, you can skip this section.

If it is python2, you need execute the following commands to set default python to python3.

1. Enter directory /usr/bin

```
cd /usr/bin
```

2. Delete the originalpython link.

```
sudo rm python
```

3. Create new python links to python.

```
sudo ln -s python3 python
```

4. Check python. Press Ctrl-Z to exit.

```
python
```

```
pi@raspberrypi:/usr/bin $ sudo rm python
pi@raspberrypi:/usr/bin $ sudo ln -s python3 python
pi@raspberrypi:/usr/bin $ python
Python 3.5.3 (default, Jan 19 2017, 14:11:04)
[GCC 6.3.0 20170124] on linux
Type "help", "copyright", "credits" or "license" for more information.
>>>
```

If you want to set python2 as default python in **other projects**, just repeat the commands above and change python3 to python2.

Shortcut Key

Now, we will introduce several shortcuts that are very **useful** and **commonly used** in terminal.

1. **up and down arrow keys**. History commands can be quickly brought back by using up and down arrow keys, which are very useful when you need to reuse certain commands.

When you need to type commands, pressing "↑" will go backwards through the history of typed commands, and pressing "↓" will go forwards through the history of typed command.

2. **Tab key**. The Tab key can automatically complete the command/path you want to type. When there are multiple commands/paths conforming to the already typed letter, pressing Tab key once won't have any result. And pressing Tab key again will list all the eligible options. This command/path will be completely typed as soon as you press the Tab key when there is only one eligible option.

As shown below, under the '~'directory, enter the Documents directory with the "cd" command. After typing "cd D", press Tab key, then there is no response. Press Tab key again, then all the files/folders that begin with "D" is listed. Continue to type the character "oc", then press the Tab key, and then "Documents" is completely typed automatically.

```
pi@raspberrypi:~ $ cd D
Desktop/   Documents/ Downloads/
pi@raspberrypi:~ $ cd Doc
```

```
pi@raspberrypi:~ $ cd D
Desktop/   Documents/ Downloads/
pi@raspberrypi:~ $ cd Documents/
```

Chapter 1 LED

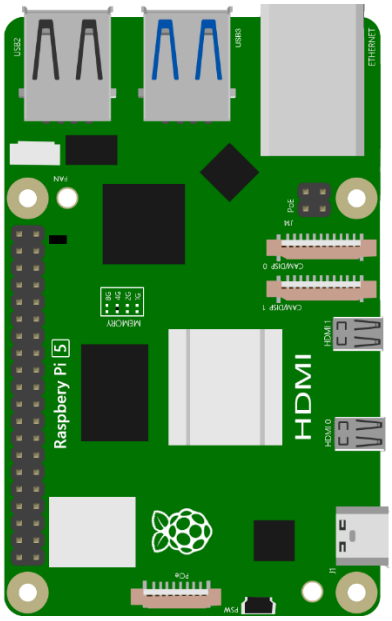
This chapter is the Start Point in the journey to build and explore RPi electronic projects. We will start with simple “Blink” project.

Project 1.1 Blink

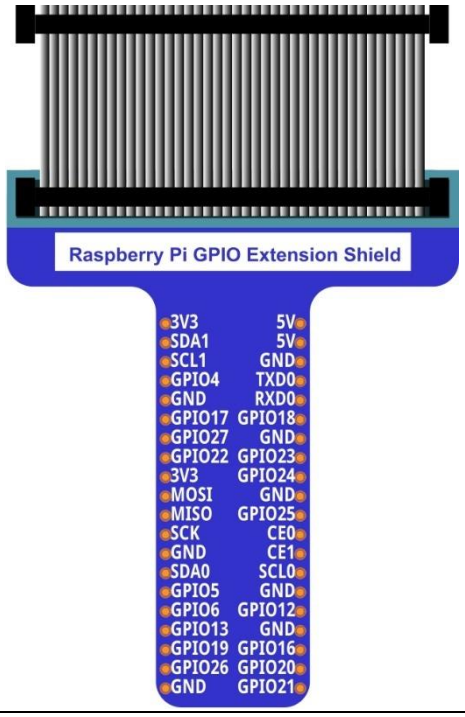
In this project, we will use RPi to control blinking a common LED.

Component List

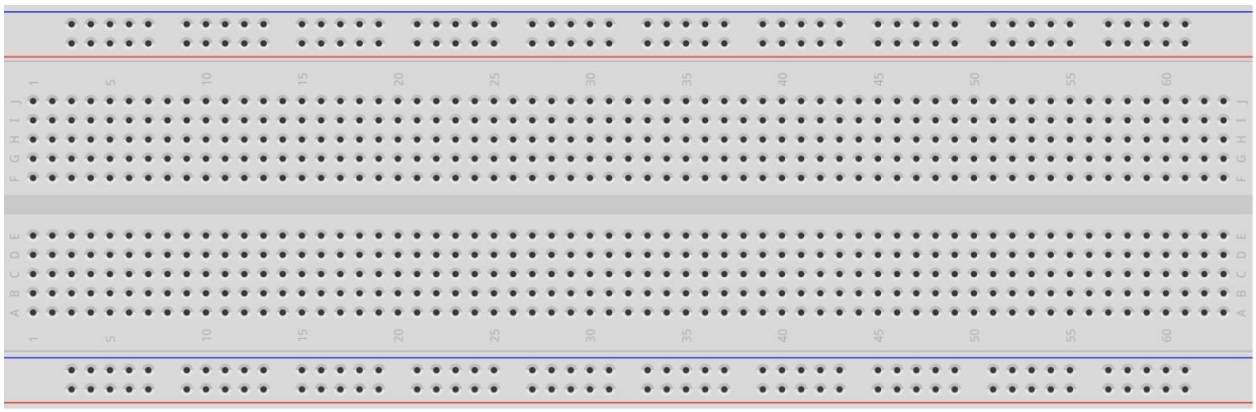
Raspberry Pi
(Recommended: Raspberry Pi 4B / 3B+ / 3B
Compatible: 3A+ / 2B / 1B+ / 1A+ / Zero W / Zero)



GPIO Extension Board & Ribbon Cable



Breadboard x1



LED x1	Resistor 220Ω x1	Jumper **Specific quantity depends on the circuit.**
		

In the components list, 3B GPIO, Extension Shield Raspberry and Breadboard are necessary for each project. Later, they will be reference by text only (no images as in above).

GPIO

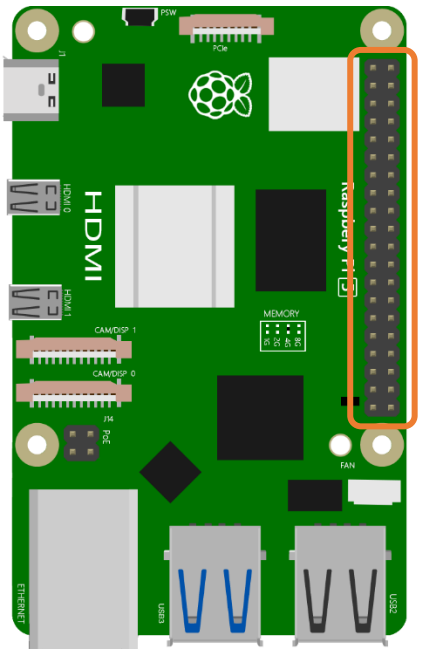
GPIO: General Purpose Input/Output. Here we will introduce the specific function of the pins on the Raspberry Pi and how you can utilize them in all sorts of ways in your projects. Most RPi Module pins can be used as either an input or output, depending on your program and its functions.

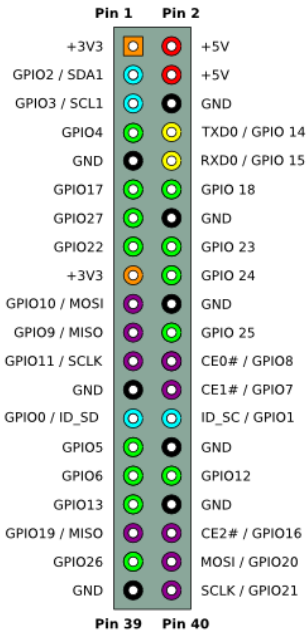
When programming GPIO pins there are 3 different ways to reference them: GPIO Numbering, Physical Numbering and WiringPi GPIO Numbering.

BCM GPIO Numbering

The Raspberry Pi CPU uses Broadcom (BCM) processing chips BCM2835, BCM2836 or BCM2837. GPIO pin numbers are assigned by the processing chip manufacturer and are how the computer recognizes each pin. The pin numbers themselves do not make sense or have meaning as they are only a form of identification. Since their numeric values and physical locations have no specific order, there is no way to remember them so you will need to have a printed reference or a reference board that fits over the pins.

Each pin's functional assignment is defined in the image below:

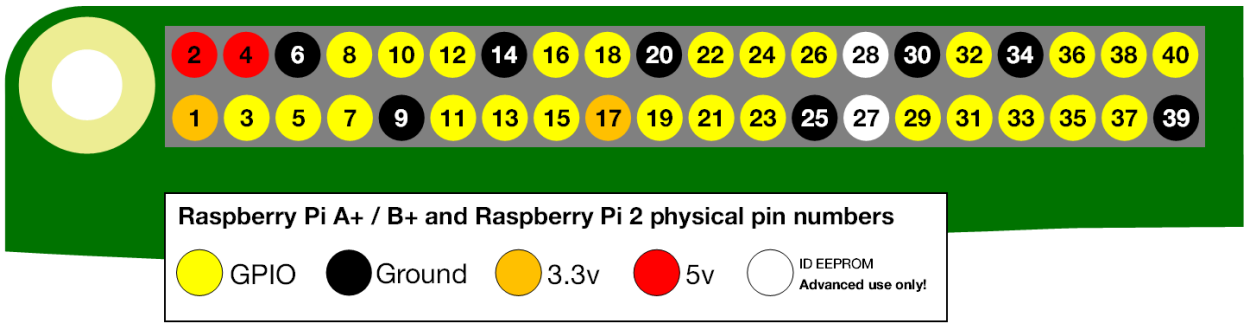




For more details about pin definition of GPIO, please refer to http://pinout.xyz/

PHYSICAL Numbering

Another way to refer to the pins is by simply counting across and down from pin 1 at the top left (nearest to the SD card). This is 'Physical Numbering', as shown below:



GPIO Numbering

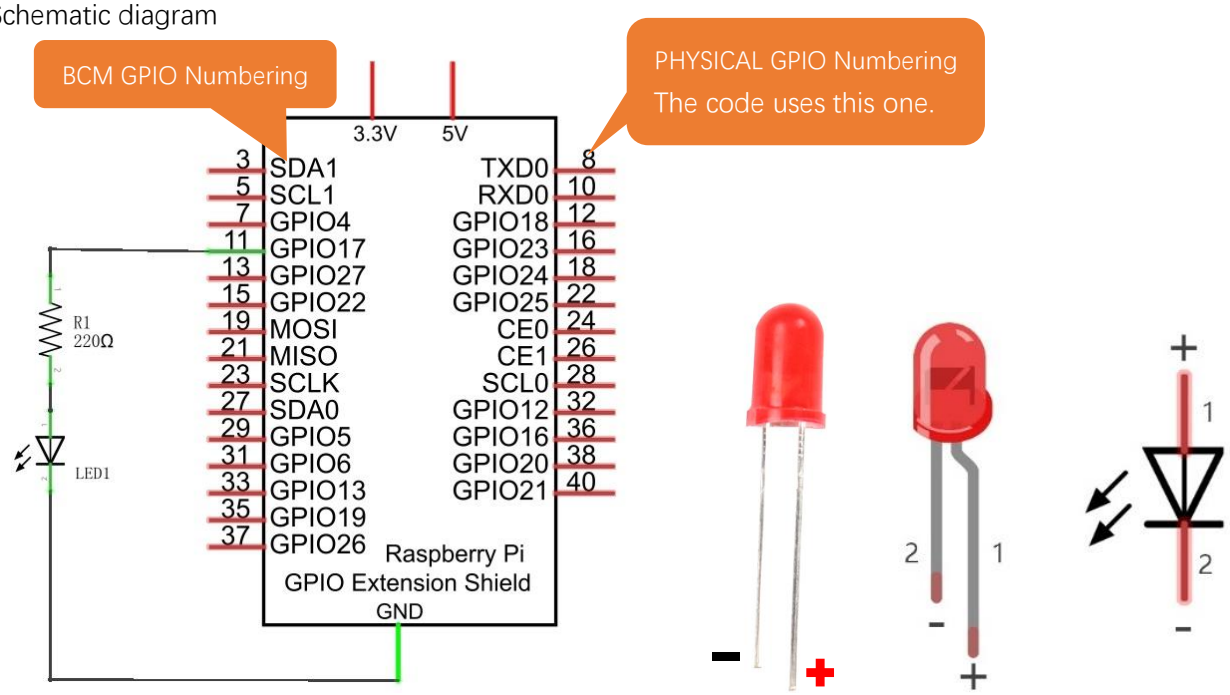
You can use the following command to view their correlation.

```
Pinout
```

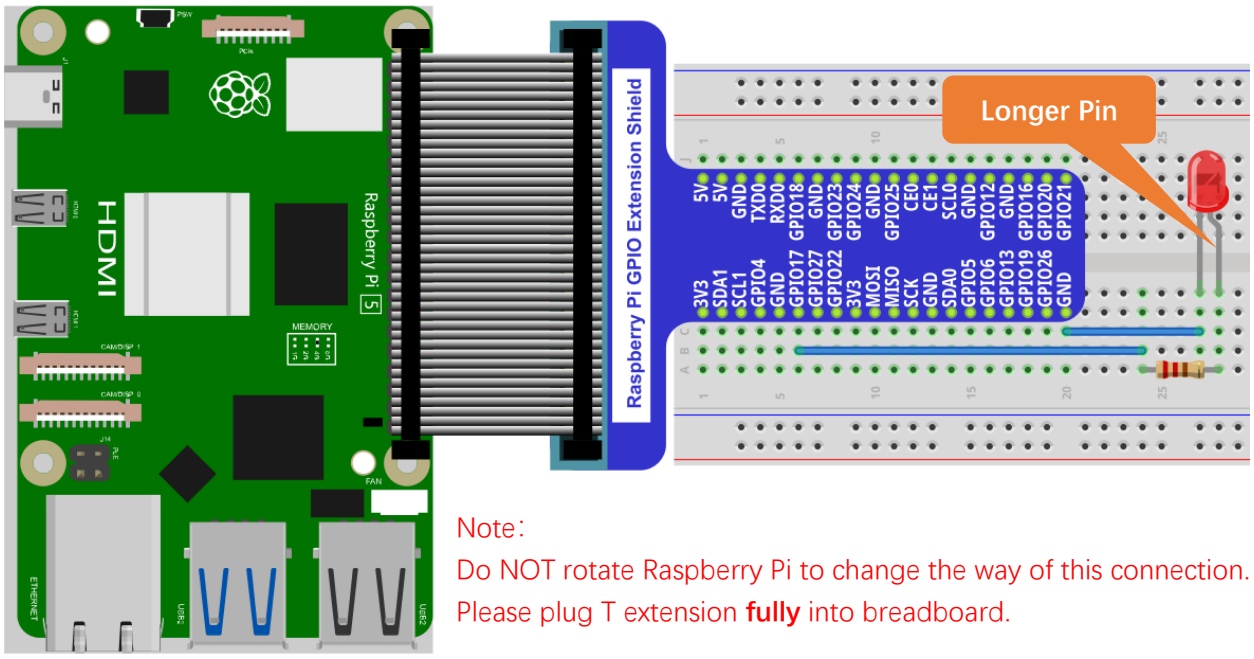
Circuit

First, disconnect your RPi from the GPIO Extension Shield. Then build the circuit according to the circuit and hardware diagrams. After the circuit is built and verified correct, connect the RPi to GPIO Extension Shield.
CAUTION: Avoid any possible short circuits (especially connecting 5V or GND, 3.3V and GND)!
WARNING: A short circuit can cause high current in your circuit, create excessive component heat and cause permanent damage to your RPi!

Schematic diagram

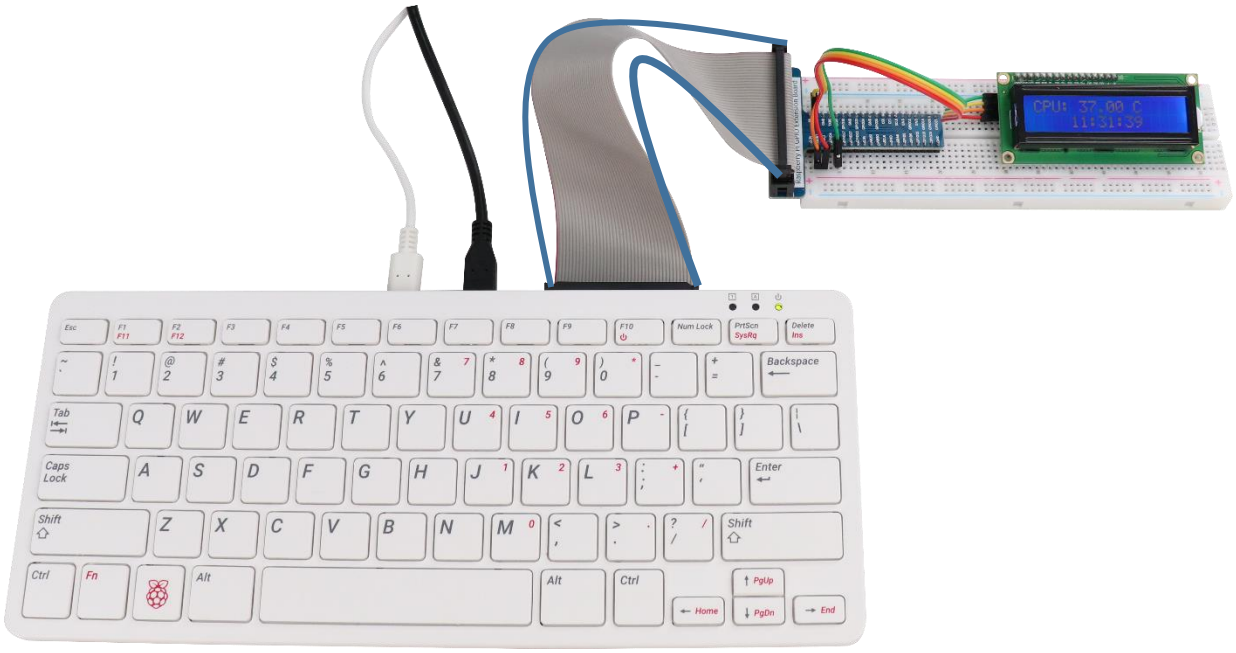


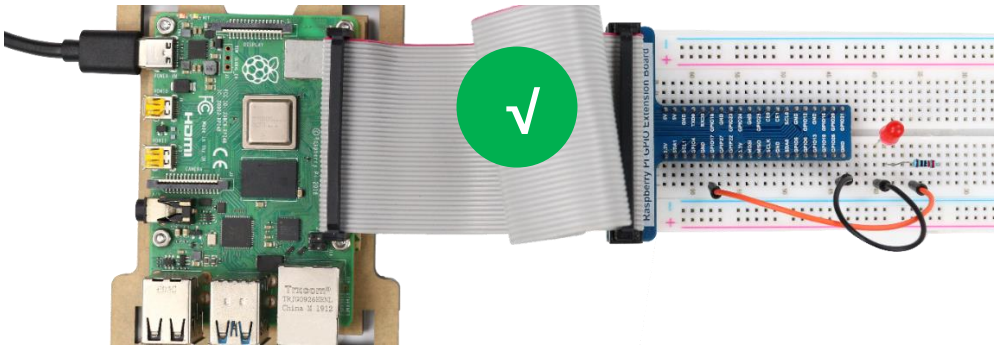
Hardware connection. **If you need any support, please contact us via: support@freenove.com**

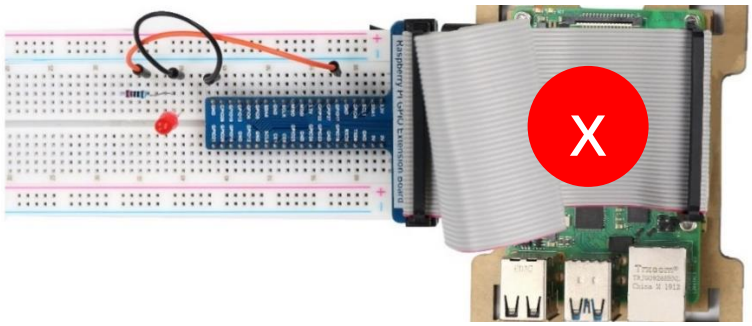


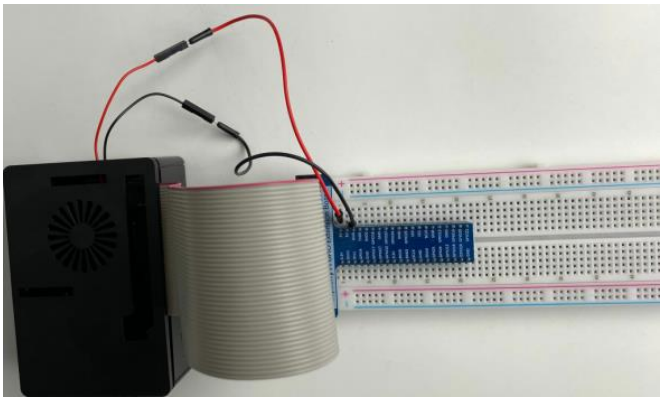
Note:
Do NOT rotate Raspberry Pi to change the way of this connection.
Please plug T extension **fully** into breadboard.

The connection of Raspberry Pi T extension board is as below. Don't reverse the ribbon.









If you have a fan, you can connect it to 5V GND of breadboard via jumper wires.

How to distinguish resistors?

There are only three kind of resistors in this kit.

The one with 1 red ring is 10KΩ 

The one with 2 red rings is 220Ω 

The one with 0 red ring is 1KΩ 

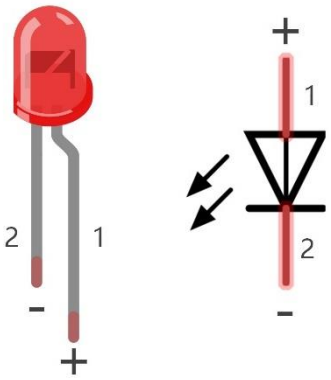
Future hardware connection diagrams will only show that part of breadboard and GPIO Extension Shield.

Component knowledge

LED

An LED is a type of diode. All diodes only work if current is flowing in the correct direction and have two Poles. An LED will only work (light up) if the longer pin (+) of LED is connected to the positive output from a power source and the shorter pin is connected to the negative (-) output, which is also referred to as Ground (GND). This type of component is known as “Polar” (think One-Way Street).

All common 2 lead diodes are the same in this respect. Diodes work only if the voltage of its positive electrode is higher than its negative electrode and there is a narrow range of operating voltage for most all common diodes of 1.9 and 3.4V. If you use much more than 3.3V the LED will be damaged and burnt out.



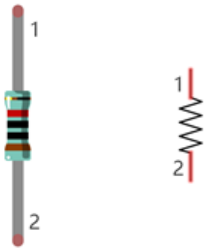
LED	Voltage	Maximum current	Recommended current
Red	1.9-2.2V	20mA	10mA
Green	2.9-3.4V	10mA	5mA
Blue	2.9-3.4V	10mA	5mA
Volt ampere characteristics conform to diode			

Note: LEDs cannot be directly connected to a power supply, which usually ends in a damaged component. A resistor with a specified resistance value must be connected in series to the LED you plan to use.

Resistor

Resistors use Ohms (Ω) as the unit of measurement of their resistance (R). 1MΩ=1000kΩ, 1kΩ=1000Ω.

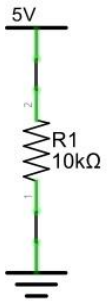
A resistor is a passive electrical component that limits or regulates the flow of current in an electronic circuit. On the left, we see a physical representation of a resistor, and the right is the symbol used to represent the presence of a resistor in a circuit diagram or schematic.



The bands of color on a resistor is a shorthand code used to identify its resistance value. For more details of resistor color codes, please refer to the card in the kit package.

With a fixed voltage, there will be less current output with greater resistance added to the circuit. The relationship between Current, Voltage and Resistance can be expressed by this formula: I=V/R known as Ohm’s Law where I = Current, V = Voltage and R = Resistance. Knowing the values of any two of these allows you to solve the value of the third.

In the following diagram, the current through R1 is: I=U/R=5V/10kΩ=0.0005A=0.5mA.

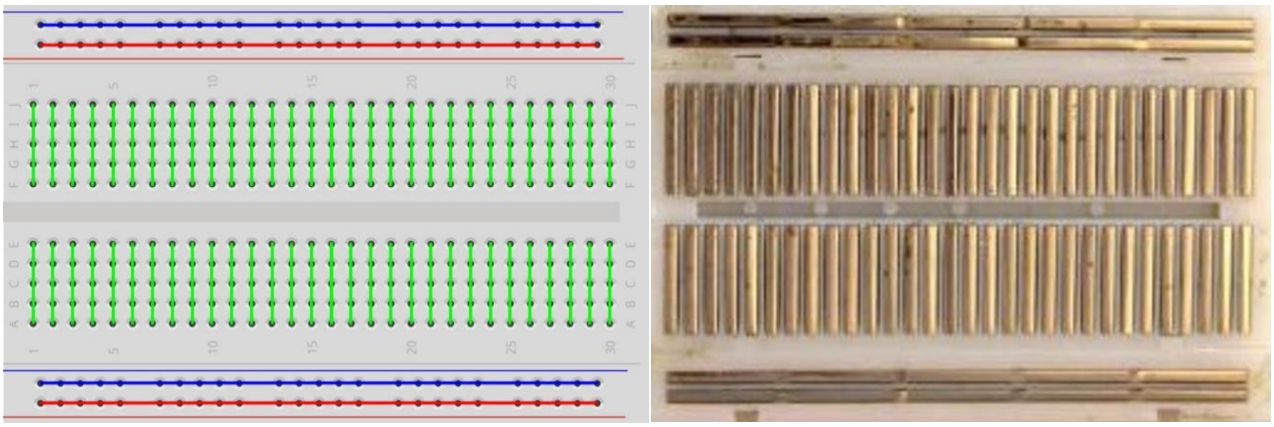


WARNING: Never connect the two poles of a power supply with anything of low resistance value (i.e. a metal object or bare wire) this is a Short and results in high current that may damage the power supply and electronic components.

Note: Unlike LEDs and Diodes, Resistors have no poles and re non-polar (it does not matter which direction you insert them into a circuit, it will work the same)

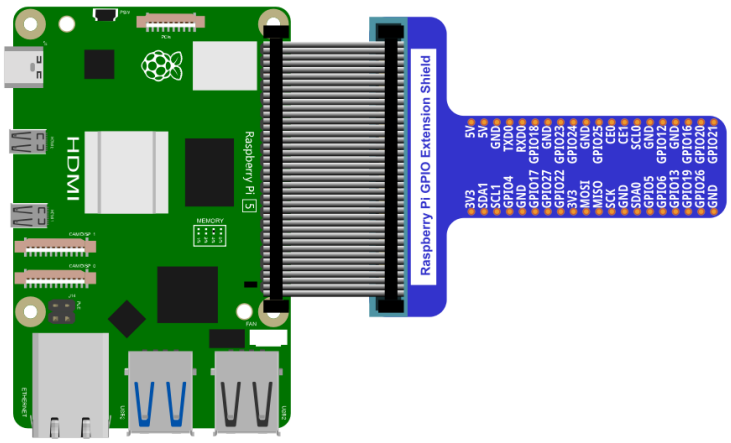
Breadboard

Here we have a small breadboard as an example of how the rows of holes (sockets) are electrically attached. The left picture shows the ways the pins have shared electrical connection and the right picture shows the actual internal metal, which connect these rows electrically.



GPIO Extension Board

GPIO board is a convenient way to connect the RPi I/O ports to the breadboard directly. The GPIO pin sequence on Extension Board is identical to the GPIO pin sequence of RPi.



Code

According to the circuit, when the GPIO17 of RPi output level is high, the LED turns ON. Conversely, when the GPIO17 RPi output level is low, the LED turns OFF. Therefore, we can let GPIO17 cycle output high and output low level to make the LED blink. We will use Python code to achieve the target.

Python Code 1.1.1 Blink

Now, we will use Python language to make a LED blink.

First, observe the project result, and then learn about the code in detail.

If you have any concerns, please contact us via: support@freenove.com

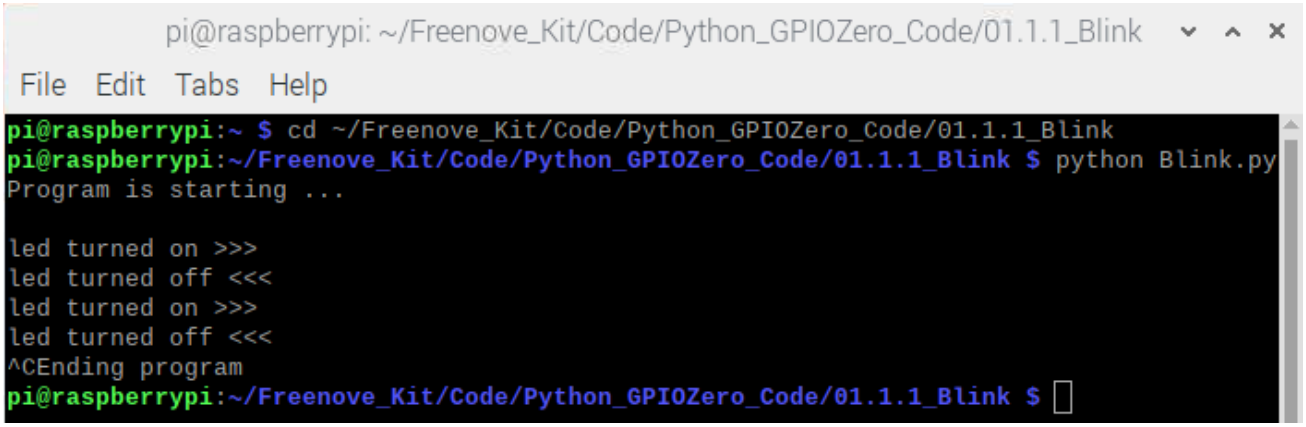
1. Use cd command to enter 01.1.1_Blink directory of Python code.

```
cd ~/Freenove_Kit/Code/Python_GPIOZero_Code/01.1.1_Blink
```

2. Use python command to execute python code blink.py.

```
python Blink.py
```

The LED starts blinking.



You can press "Ctrl+C" to end the program. The following is the program code:

```
1  from gpiozero import LED
2  from time import sleep
3
4  led = LED(17)           # define LED pin according to BCM Numbering
5  #led = LED("J8:11")     # BOARD Numbering
6  '''
7  # pins numbering, the following lines are all equivalent
8  led = LED(17)           # BCM
9  led = LED("GPIO17")     # BCM
10 led = LED("BCM17")      # BCM
11 led = LED("BOARD11")    # BOARD
12 led = LED("WPI0")       # WiringPi
13 led = LED("J8:11")      # BOARD
14 '''
15 def loop():
16     while True:
```

```
17          led.on()     # turn on LED
18          print ('led turned on >>>')  # print message on terminal
19          sleep(1)     # wait 1 second
20          led.off()    # turn off LED
21          print ('led turned off <<<') # print message on terminal
22          sleep(1)     # wait 1 second
23
24  if __name__ == '__main__':     # Program entrance
25      print ('Program is starting ... \n')
26      try:
27          loop()
28      except KeyboardInterrupt:  # Press ctrl-c to end the program.
29          print("Ending program")
```

Import the LED class from the gpiozero library.

```
from gpiozero import LED
```

Create an LED assembly for controlling the LED.

```
led = LED(17)           # define LED pin according to BCM Numbering
```

Turn on LED device.

```
        led.on()     # turn on LED
```

Turn off LED devices.

```
        led.off()    # turn off LED
```

The main function turns on the LED for one second and then turns it off for one second, which repeats endless.

```
def loop():
    while True:
        led.on()     # turn on LED
        print ('led turned on >>>')  # print message on terminal
        sleep(1)     # wait 1 second
        led.off()    # turn off LED
        print ('led turned off <<<') # print message on terminal
        sleep(1)     # wait 1 second
```

Reference

About GPIO Zero:

GPIO Zero

A simple interface to GPIO devices with Raspberry Pi, Using the GPIO Zero library makes it easy to get started with controlling GPIO devices with Python. The library is comprehensively documented at https://gpiozero.readthedocs.io/en/stable/
https://github.com/gpiozero/gpiozero

For more information about the methods used by the LED class in the GPIO Zero library,please refer to:
https://gpiozero.readthedocs.io/en/stable/api_output.html#led

For more information about the methods used by the DigitalOutputDevice class in the GPIO Zero library,please refer to:
https://gpiozero.readthedocs.io/en/stable/api_output.html#digitaloutputdevice

“import time” time is a module of python.

https://docs.python.org/2/library/time.html?highlight=time%20time#module-time

In Python, libraries and functions used in a script must be imported by name at the top of the file, with the exception of the functions built into Python by default.

For example, to use the LED interface from GPIO Zero, it should be explicitly imported:

```
from gpiozero import LED
```

Now LEDis available directly in your script:

```
led = LED(17)            # define LED pin according to BCM Numbering
#led = LED("J8:11")      # BOARD Numbering
```

Alternatively, the whole GPIO Zero library can be imported:

```
import gpiozero
```

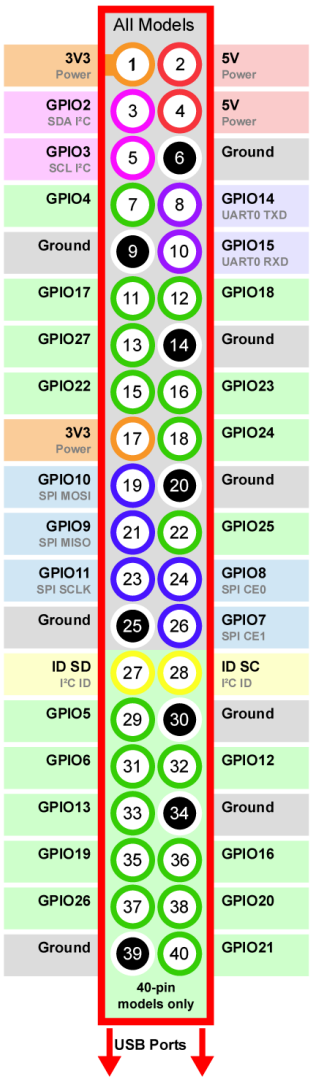
In this case, all references to items within GPIO Zero must be prefixed:

```
led = gpiozero.LED(17)            # define LED pin according to BCM Numbering
#led = gpiozero.LED("J8:11")      # BOARD Numbering
```

Pin Numbering

This library uses Broadcom (BCM) pin numbering for the GPIO pins, as opposed to physical (BOARD) numbering. Unlike in the RPi.GPIO library, this is not configurable. However, translation from other schemes can be used by providing prefixes to pin numbers (see below).

Any pin marked “GPIO” in the diagram below can be used as a pin number. For example, if an LED was attached to “GPIO17” you would specify the pin number as 17 rather than 11:



If you wish to use physical (BOARD) numbering you can specify the pin number as “BOARD11”. If you are familiar with the wiringPi pin numbers (another physical layout) you could use “WPI0” instead.

Finally, you can specify pins as “header:number”, e.g. “J8:11” meaning physical pin 11 on header J8 (the GPIO header on modern Pis). Hence, the following lines are all equivalent:

```
led = LED(17)
led = LED("GPIO17")
led = LED("BCM17")
led = LED("BOARD11")
led = LED("WPI0")
led = LED("J8:11")
```

Note that these alternate schemes are merely translations. If you request the state of a device on the command line, the associated pin number will always be reported in the Broadcom (BCM) scheme:

```
    led = LED("BOARD11")
    led
<gpiozero.LED object on pin GPIO17, active_high=True, is_active=False>
```

In this tutorial, we will use the default integer pin number in the Broadcom (BCM) layout.

GPIO Numbering Relationship

WingPi	BCM(Extension)	Physical		BCM(Extension)	WingPi
3.3V	3.3V	1	2	5V	5V
8	GPIO2/SDA1	3	4	5V	5V
9	GPIO3/SCL1	5	6	GND	GND
7	GPIO4	7	8	GPIO14/TXD0	15
GND	GND	9	10	GPIO15/RXD0	16
0	GPIO17	11	12	GPIO18	1
2	GPIO27	13	14	GND	GND
3	GPIO22	15	16	GPIO23	4
3.3V	3.3V	17	18	GPIO24	5
12	GPIO10/MOSI	19	20	GND	GND
13	GPIO9/MOIS	21	22	GPIO25	6
14	GPIO11/SCLK	23	24	GPIO8/CE0	10
GND	GND	25	26	GPIO7/CE1	11
30	GPIO0/SDA0	27	28	GPIO1/SCL0	31
21	GPIO5	29	30	GND	GND
22	GPIO6	31	32	GPIO12	26
23	GPIO13	33	34	GND	GND
24	GPIO19	35	36	GPIO16	27
25	GPIO26	37	38	GPIO20	28
GND	GND	39	40	GPIO21	29

In loop(), there is a while loop, which is an endless loop (a while loop). That is, the program will always be executed in this loop, unless it is ended because of external factors. In this loop, set LED output high level, then the LED turns ON. After a period of time delay, set LED output low level, then the LED turns OFF, which is followed by a delay. Repeat the loop, then LED will start blinking.

```
def loop():
    while True:
        led.on()     # turn on LED
        print ('led turned on >>>')  # print message on terminal
        sleep(1)     # wait 1 second
        led.off()    # turn off LED
        print ('led turned off <<<') # print message on terminal
        sleep(1)     # wait 1 second
```

In gpiozero, at the end of your script, cleanup is run automatically, restoring your GPIO pins to the state they were found.To explicitly close a connection to a pin, you can manually call the close() method on a device object:

```
    led = LED(17)
```

```
    led.on()
    led
<gpiozero.LED object on pin GPIO17, active_high=True, is_active=True>
    led.close()
    led
<gpiozero.LED object closed>
```

This means that you can reuse the pin for another device, and that despite turning the LED on (and hence, the pin high), after calling close() it is restored to its previous state (LED off, pin low).

In this tutorial, most projects have added an active run cleanup program to restore the GPIO pin to the found default state.

Freenove Car, Robot and other products for Raspberry Pi

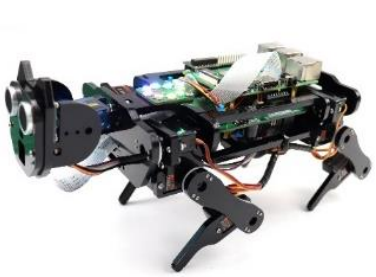
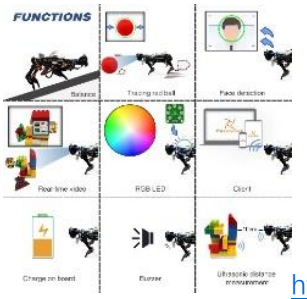
We also have car and robot kits for Raspberry Pi. You can visit our website for details.

https://www.amazon.com/freenove

FNK0043 **Freenove 4WD Smart Car Kit for Raspberry Pi**


https://www.youtube.com/watch?v=4Zv0GZUQjZc

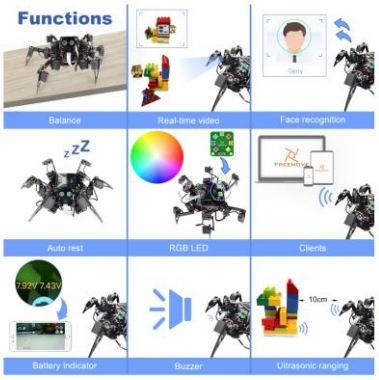
FNK0050 **Freenove Robot Dog Kit for Raspberry Pi**

https://www.youtube.com/watch?v=7BmlZ8_R9d4

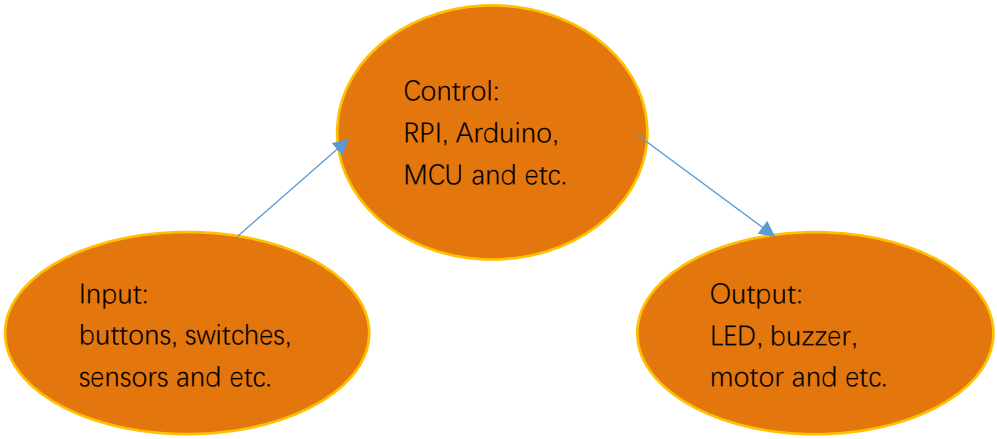
FNK0052 **Freenove_Big_Hexapod_Robot_Kit_for_Raspberry_Pi**

https://youtu.be/LvghnJ2DNZ0

Chapter 2 Buttons & LEDs

Usually, there are three essential parts in a complete automatic control device: INPUT, OUTPUT, and CONTROL. In last section, the LED module was the output part and RPI was the control part. In practical applications, we not only make LEDs flash, but also make a device sense the surrounding environment, receive instructions and then take the appropriate action such as turn on LEDs, make a buzzer beep and so on.



Next, we will build a simple control system to control an LED through a push button switch.

Project 2.1 Push Button Switch & LED

In the project, we will control the LED state through a Push Button Switch. When the button is pressed, our LED will turn ON, and when it is released, the LED will turn OFF. This describes a Momentary Switch.

Component List

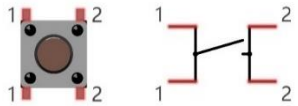
Raspberry Pi (with 40 GPIO) x1 GPIO Extension Board & Wire x1 Breadboard x1	LED x1 	Resistor 220Ω x1 	Resistor 10kΩ x2 	Push Button Switch x1 
Jumper Wire 				

Please Note: In the code “button” represents switch action.

Component knowledge

Push Button Switch

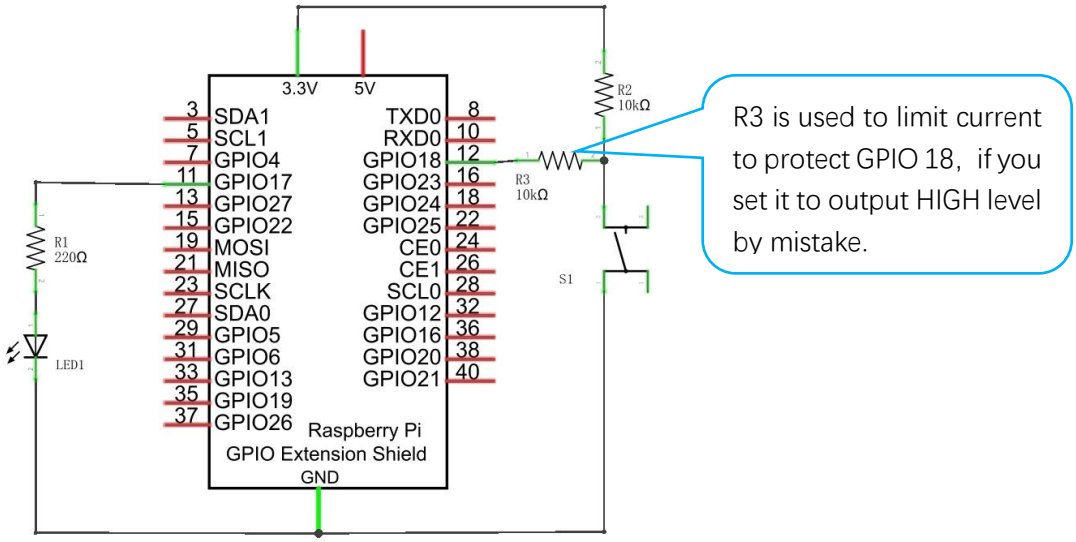
This type of Push Button Switch has 4 pins (2 Pole Switch). Two pins on the left are connected, and both left and right sides are the same per the illustration:



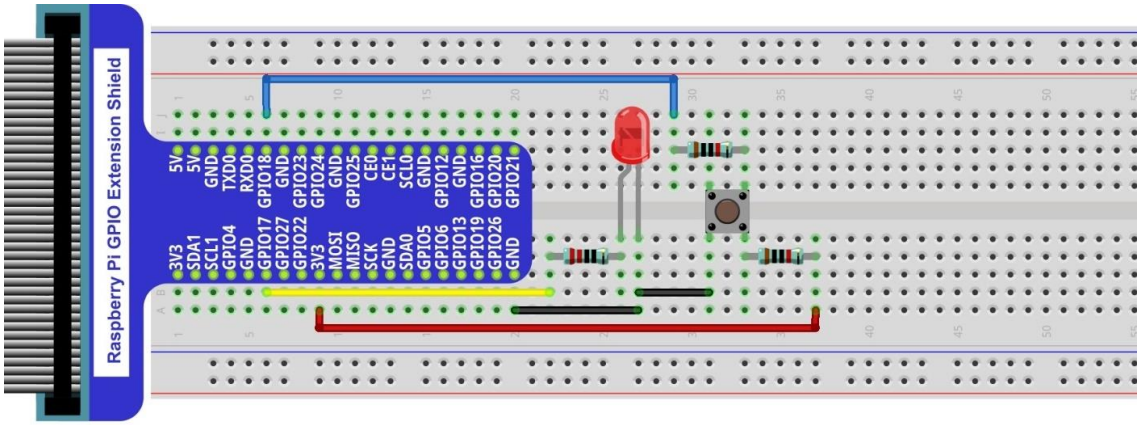
When the button on the switch is pressed, the circuit is completed (your project is Powered ON).

Circuit

Schematic diagram



Hardware connection. If you need any support, please feel free to contact us via:support@freenove.com



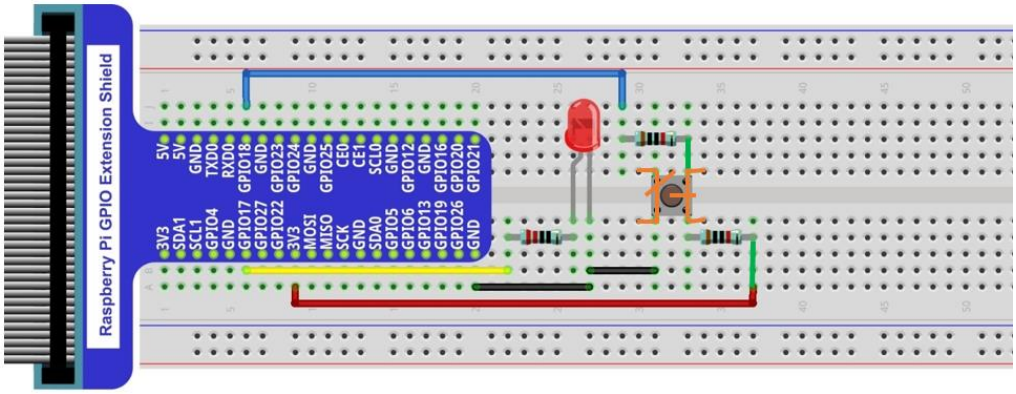
There are two kinds of push button switch in this kit.

The smaller push button switches are contained in a plastic bag.

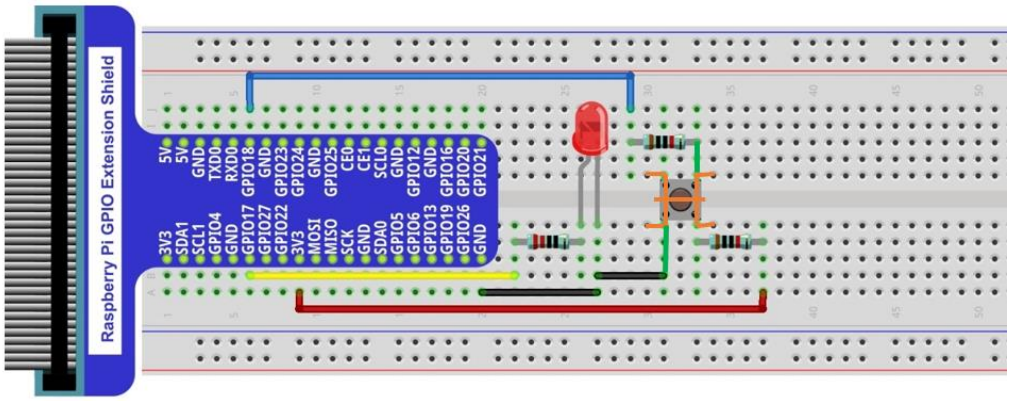
Youtube video: https://youtu.be/_5ge1d6f1nM

This is how it works.

When button switch is released:



When button switch is pressed:



Code

This project is designed for learning how to use Push Button Switch to control an LED. We first need to read the state of switch, and then determine whether to turn the LED ON in accordance to the state of the switch.

Python Code 2.1.1 ButtonLED

First, observe the project result, then learn about the code in detail. Remember in code “button” = switch function

If you have any concerns, please contact us via: support@freenove.com

1. Use cd command to enter 02.1.1_ButtonLED directory of Python code.

```
cd ~/Freenove_Kit/Code/Python_GPIOZero_Code/02.1.1_ButtonLED
```

2. Use Python command to execute btnLED.py.

```
python ButtonLED.py
```

Then the Terminal window continues to show the characters “led off…”, press the switch button and the LED turns ON and then Terminal window shows "led on…". Release the button, then LED turns OFF and then the terminal window text "led off…" appears. You can press "Ctrl+C" at any time to terminate the program.

The following is the program code:

```
1  from gpiozero import LED, Button
2
```

```
led = LED(17)        # define LED pin according to BCM Numbering
button = Button(18) # define Button pin according to BCM Numbering

def loop():
    while True:
        if button.is_pressed:  # if button is pressed
            led.on()         # turn on led
            print("Button is pressed, led turned on >>>") # print information on terminal
        else : # if button is relessed
            led.off() # turn off led
            print("Button is released, led turned off <<<")

if __name__ == '__main__':     # Program entrance
    print ('Program is starting...')
    try:
        loop()
    except KeyboardInterrupt:  # Press ctrl-c to end the program.
        print("Ending program")
```

Import the Button class that controls Button from the gpiozero library.

```
from gpiozero import LED, Button
```

Define GPIO17 as the LED control pin and GPIO18 as the button control pin. The button is set to the input mode with a pull-up resistor by default.

```
led = LED(17)        # define LED pin according to BCM Numbering
button = Button(18) # define Button pin according to BCM Numbering
```

The loop continuously determines whether the key is pressed. When the button is pressed, the variable button.is_pressed has a value of 1 and the LED lights up. Otherwise, the LED will be off.

```
def loop():
    while True:
        if button.is_pressed:  # if button is pressed
            led.on()         # turn on led
            print("Button is pressed, led turned on >>>") # print information on terminal
        else : # if button is relessed
            led.off() # turn off led
            print("Button is released, led turned on <<<")
```

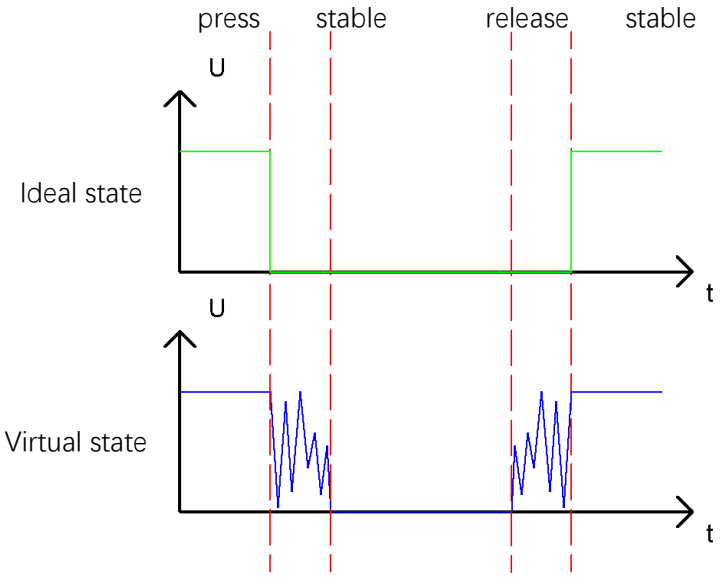
For more information about GPIOZero, please refer to the link below:
https://gpiozero.readthedocs.io/en/stable/
For more information about the methods used by the Button class in the GPIO Zero library,please refer to:
https://gpiozero.readthedocs.io/en/stable/api_input.html#button

Project 2.2 MINI Table Lamp

We will also use a Push Button Switch, LED and RPi to make a MINI Table Lamp but this will function differently: Press the button, the LED will turn ON, and pressing the button again, the LED turns OFF. The ON switch action is no longer momentary (like a door bell) but remains ON without needing to continually press on the Button Switch.
First, let us learn something about the push button switch.

Debounce a Push Button Switch

When a Momentary Push Button Switch is pressed, it will not change from one state to another state immediately. Due to tiny mechanical vibrations, there will be a short period of continuous buffeting before it stabilizes in a new state too fast for Humans to detect but not for computer microcontrollers. The same is true when the push button switch is released. This unwanted phenomenon is known as “bounce”.



Therefore, if we can directly detect the state of the Push Button Switch, there are multiple pressing and releasing actions in one pressing cycle. This buffeting will mislead the high-speed operation of the microcontroller to cause many false decisions. Therefore, we need to eliminate the impact of buffeting. Our solution: to judge the state of the button multiple times. Only when the button state is stable (consistent) over a period of time, can it indicate that the button is actually in the ON state (being pressed).
This project needs the same components and circuits as we used in the previous section.

Code

In this project, we still detect the state of Push Button Switch to control an LED. Here we need to define a variable to define the state of LED. When the button switch is pressed once, the state of LED will be changed once. This will allow the circuit to act as a virtual table lamp.

Python Code 2.2.1 Tablelamp

First, observe the project result, and then learn about the code in detail.

If you have any concerns, please contact us via: support@freenove.com

1. Use cd command to enter 02.2.1_Tablelamp directory of Python code

```
cd ~/Freenove_Kit/Code/Python_GPIOZero_Code/02.2.1_Tablelamp
```

2. Use python command to execute python code "Tablelamp.py".

```
python Tablelamp.py
```

When the program is executed, pressing the Button Switch once turns the LED ON. Pressing the Button Switch again turns the LED OFF.

```
1  from gpiozero import LED, Button
2  import time
3
4  led = LED(17) # define LED pin according to BCM Numbering
5  button = Button(18) # define Button pin according to BCM Numbering
6
7  def onButtonPressed():  # When button is pressed, this function will be executed
8      led.toggle()
9      if led.is_lit :
10         print("Led turned on >>>")
       else :
11         print("Led turned off <<<")
12 def loop():
13     #Button detect
14     button.when_pressed = onButtonPressed
15     while True:
16         time.sleep(1)
17 def destroy():
18     led.close()
19     button.close()
20 if __name__ == '__main__':     # Program entrance
21     print ('Program is starting...')
22     try:
23         loop()
24     except KeyboardInterrupt:  # Press ctrl-c to end the program.
25         destroy()
26         print("Ending program")
```

In GPIO Zero, you assign the when_pressed and when_released properties to set up callbacks on those actions.

Once it detects that the button is pressed, it executes the specified function onButtonPressed().In the onButtonPressed function, the led. toggle () function reverses the state of the LED device.If it's on, turn it off; If it's off, turn it on.Each time the key is pressed, the state of the LED will change once.

```
def onButtonPressed():  # When button is pressed, this function will be executed
    led.toggle()
    if led.is_lit :
        print("Led turned on >>>")
    else :
        print("Led turned off <<<")

def loop():
    #Button detect
    button.when_pressed = onButtonPressed
    while True:
        time.sleep(1)
```

To explicitly close a connection to a pin, you can manually call the close() method on a device object:

```
def destroy():
    led.close()
    button.close()

    except KeyboardInterrupt:  # Press ctrl-c to end the program.
        destroy()
        print("Ending program")
```

For more information about the methods used by the Button class in the GPIO Zero library,please refer to: https://gpiozero.readthedocs.io/en/stable/api_input.html#button

Chapter 3 LED Bar Graph

We have learned how to control one LED to blink. Next, we will learn how to control a number of LEDs.

Project 3.1 Flowing Water Light

In this project, we use a number of LEDs to make a flowing water light.

Component List

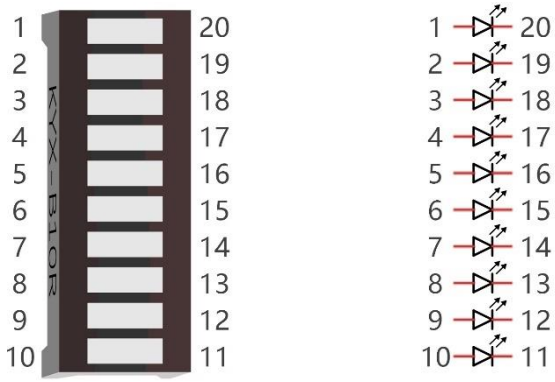
Raspberry Pi (with 40 GPIO) x1 GPIO Extension Board & Ribbon Cable x1 Breadboard x1	Bar Graph LED x1 	Resistor 220Ω x10 
Jumper Wire x 1 		

Component knowledge

Let us learn about the basic features of these components to use and understand them better.

Bar Graph LED

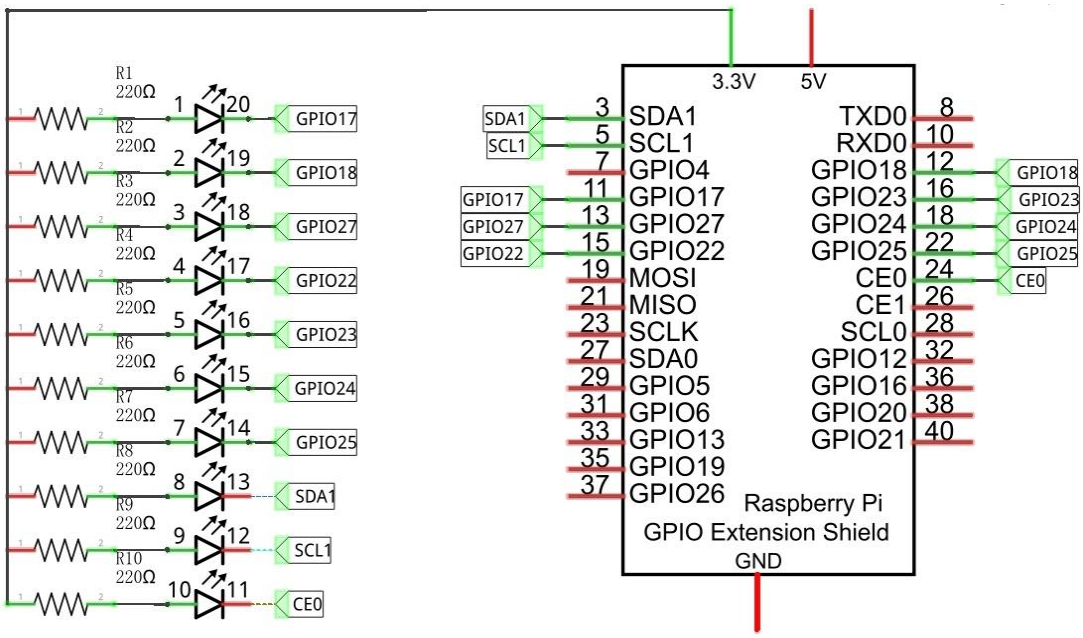
A Bar Graph LED has 10 LEDs integrated into one compact component. The two rows of pins at its bottom are paired to identify each LED like the single LED used earlier.



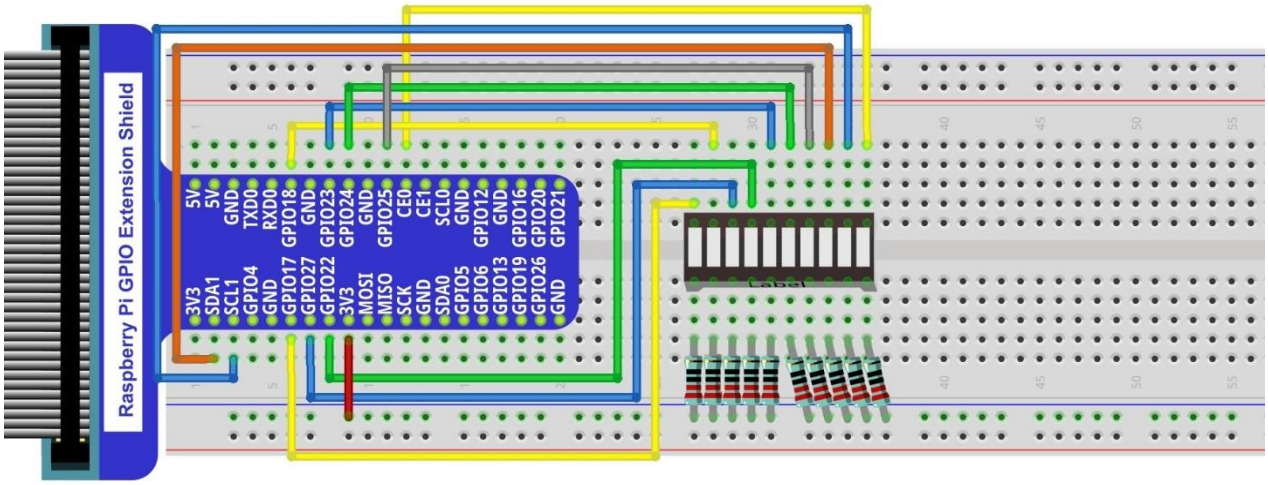
Circuit

A reference system of labels is used in the circuit diagram below. Pins with the same network label are connected together.

Schematic diagram



Hardware connection. If you need any support, please feel free to contact us via: support@freenove.com



If LEDbar doesn't work, rotate LEDbar 180° to try. The label is random.

Youtube video: https://youtu.be/3rh-b05VoiU

In this circuit, the cathodes of the LEDs are connected to the GPIO, which is different from the previous circuit. The LEDs turn ON when the GPIO output is low level in the program.

Code

This project is designed to make a flowing water lamp, which are these actions: First turn LED #1 ON, then

turn it OFF. Then turn LED #2 ON, and then turn it OFF... and repeat the same to all 10 LEDs until the last LED is turns OFF. This process is repeated to achieve the "movements" of flowing water.

Python Code 3.1.1 LightWater

First observe the project result, and then view the code.

If you have any concerns, please contact us via: support@freenove.com

1. Use cd command to enter 03.1.1_LightWater directory of Python code.

```
cd ~/Freenove_Kit/Code/Python_GPIOZero_Code/03.1.1_LightWater
```

2. Use Python command to execute Python code "LightWater.py".

```
python LightWater.py
```

After the program is executed, you will see that LED Bar Graph starts with the flowing water way to be turned on from left to right, and then from right to left.

The following is the program code:

```
1  from gpiozero import LED
2  from time import sleep
3  ledPins = [17, 18, 27, 22, 23, 24, 25, 2, 3, 8]
4  leds = [LED(pin=pin) for pin in ledPins]
5
6  def loop():
7      while True:
8          for index in range(0,len(ledPins),1):      # make led(on) move from left to right
9              leds[index].off()
10             sleep(0.1)
11             leds[index].on()
12         for index in range(len(ledPins)-1,-1,-1):   #move led(on) from right to left
13             leds[index].off()
14             sleep(0.1)
15             leds[index].on()
16
17 if __name__ == '__main__':     # Program entrance
18     print ('Program is starting...')
19     try:
20         loop()
21     except KeyboardInterrupt:  # Press ctrl-c to end the program.
22         print("Ending program")
23     finally:
24         for index in range(0,len(ledPins),1):
25             leds[index].close()
```

Import the LED class that controls LED Bar Graph from the gpiozero library.

```
from gpiozero import LED
```

Create the LED class for controlling the LEDBarGraph.

```
ledPins = [17, 18, 27, 22, 23, 24, 25, 2, 3, 8]
leds = [LED(pin=pin) for pin in ledPins]
```

The LED is turned on or off by specifying the index of the LED, if no parameter is specified, the same Settings are applied to all leds. leds.off() means that all leds are turned off.

```
        for index in range(0,len(ledPins),1):       # make led(on) move from left to right
            leds[index].off()
            sleep(0.1)
            leds[index].on()
        for index in range(len(ledPins)-1,-1,-1):   #move led(on) from right to left
            leds[index].off()
            sleep(0.1)
            leds[index].on()
```

In the program, first define 10 pins connected to the LED and set them to output mode. In the loop() function, two for loops are used to make the lights flow from right to left and from left to right.

```
def loop():
    while True:
        for index in range(0,len(ledPins),1):       # make led(on) move from left to right
            leds.off(index)
            sleep(0.1)
            leds.onindex)
        for index in range(len(ledPins)-1,-1,-1):   #move led(on) from right to left
            leds.offndex)
            sleep(0.1)
            leds.onindex)
```

For more information about the methods used by the LEDBoard class in the GPIO Zero library,please refer to: https://gpiozero.readthedocs.io/en/stable/api_boards.html#ledboard
For more information about the methods used by the LEDBarGraph class in the GPIO Zero library,please refer to: https://gpiozero.readthedocs.io/en/stable/api_boards.html#ledbargraph

In this experiment you can use the LEDBoard and LEDBarGraph classes to control the LEDBarGraph

Chapter 4 Analog & PWM

In previous chapters, we learned that a Push Button Switch has two states: Pressed (ON) and Released (OFF), and an LED has a Light ON and OFF state. Is there a middle or intermediated state? We will next learn how to create an intermediate output state to achieve a partially bright (dim) LED.
First, let us learn how to control the brightness of an LED.

Project 4.1 Breathing LED

We describe this project as a Breathing Light. This means that an LED that is OFF will then turn ON gradually and then gradually turn OFF like "breathing". Okay, so how do we control the brightness of an LED to create a Breathing Light? We will use PWM to achieve this goal.

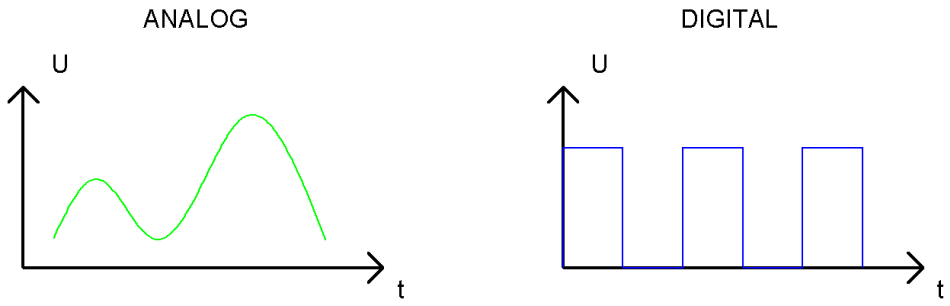
Component List

<table>
<tr><td>Raspberry Pi (with 40 GPIO) x1
GPIO Extension Board & Ribbon Cable x1
Breadboard x1</td><td rowspan="2">LED x1
  </td><td rowspan="2">Resistor 220Ω x1
  </td></tr>
<tr><td>Jumper Wire
  </td></tr>
</table>

Component Knowledge

Analog & Digital

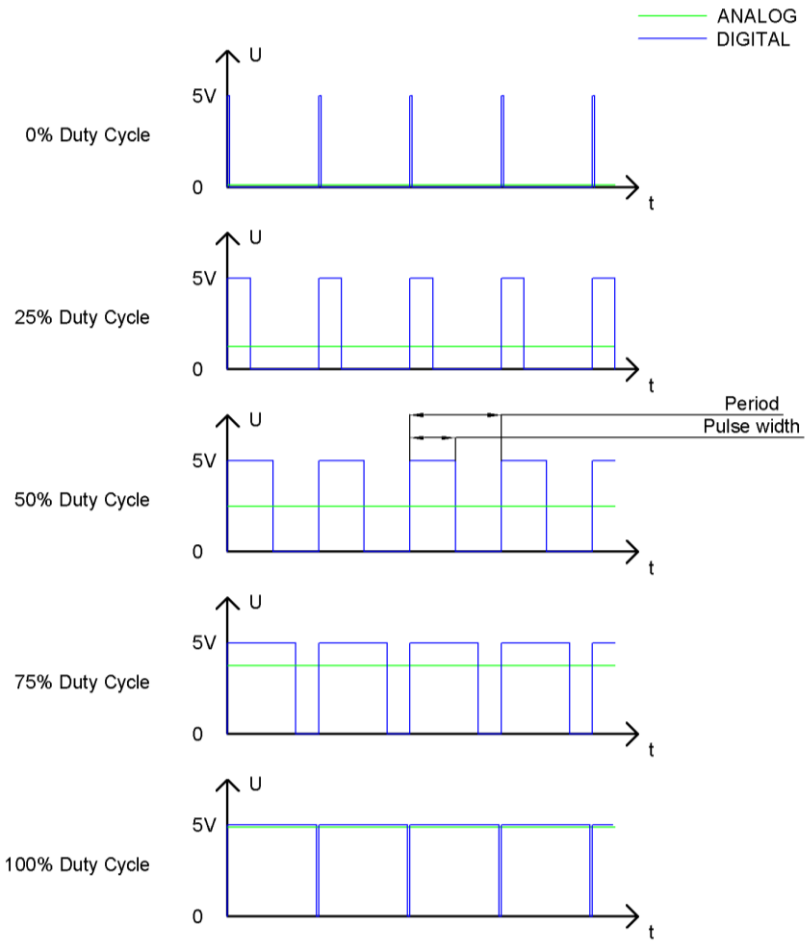
An Analog Signal is a continuous signal in both time and value. On the contrary, a Digital Signal or discrete-time signal is a time series consisting of a sequence of quantities. Most signals in life are analog signals. A familiar example of an Analog Signal would be how the temperature throughout the day is continuously changing and could not suddenly change instantaneously from 0℃ to 10℃. However, Digital Signals can instantaneously change in value. This change is expressed in numbers as 1 and 0 (the basis of binary code). Their differences can more easily be seen when compared when graphed as below.



Note that the Analog signals are curved waves and the Digital signals are “Square Waves”.
In practical applications, we often use binary as the digital signal, that is a series of 0’s and 1’s. Since a binary signal only has two values (0 or 1) it has great stability and reliability. Lastly, both analog and digital signals can be converted into the other.

PWM

PWM, Pulse-Width Modulation, is a very effective method for using digital signals to control analog circuits. Digital processors cannot directly output analog signals. PWM technology makes it very convenient to achieve this conversion (translation of digital to analog signals).
PWM technology uses digital pins to send certain frequencies of square waves, that is, the output of high levels and low levels, which alternately last for a while. The total time for each set of high levels and low levels is generally fixed, which is called the period (Note: the reciprocal of the period is frequency). The time of high level outputs are generally called “pulse width”, and the duty cycle is the percentage of the ratio of pulse duration, or pulse width (PW) to the total period (T) of the waveform. The longer the output of high levels last, the longer the duty cycle and the higher the corresponding voltage in the analog signal will be. The following figures show how the analog signal voltages vary between 0V-5V (high level is 5V) corresponding to the pulse width 0%-100%:



The longer the PWM duty cycle is, the higher the output power will be. Now that we understand this relationship, we can use PWM to control the brightness of an LED or the speed of DC motor and so on.

It is evident, from the above, that PWM is not actually analog but the effective value of voltage is equivalent to the corresponding analog value. Therefore, by using PWM, we can control the output power of to an LED and control other devices and modules to achieve multiple effects and actions.

In RPi, GPIO18 pin has the ability to output to hardware via PWM with a 10-bit accuracy. This means that 100% of the pulse width can be divided into 2^{10}=1024 equal parts.

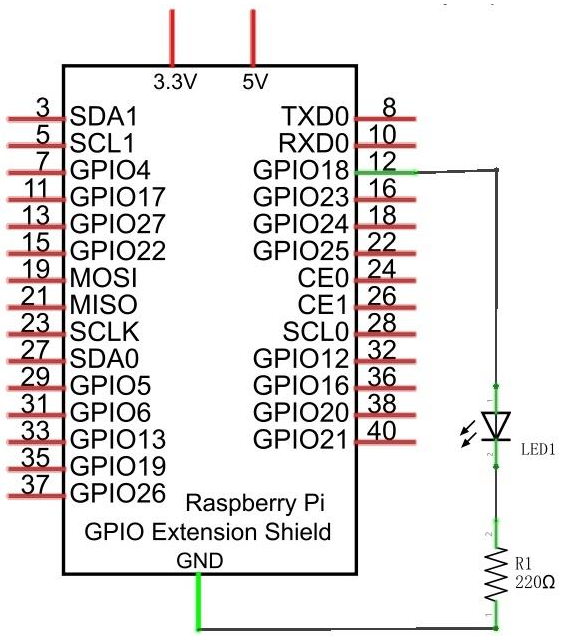
The wiringPi library of C provides both a hardware PWM and a software PWM method, while the wiringPi library of Python does not provide a hardware PWM method. There is only a software PWM option for Python.

The hardware PWM only needs to be configured, does not require CPU resources and is more precise in time control. The software PWM requires the CPU to work continuously by using code to output high level and low level. This part of the code is carried out by multi-threading, and the accuracy is relatively not high enough.

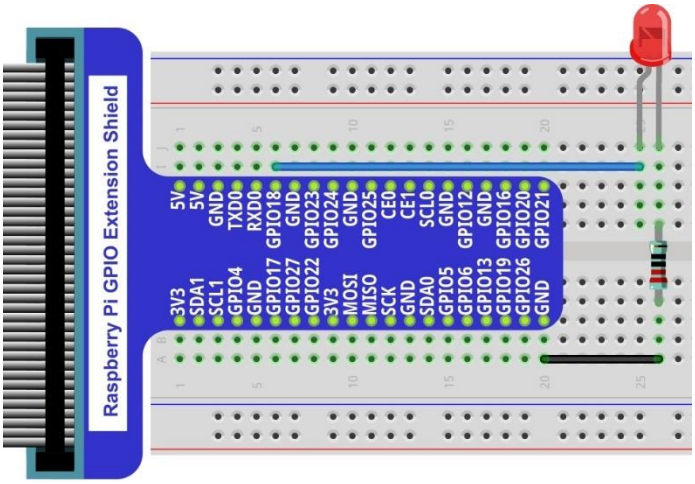
In order to keep the results running consistently, we will use software PWM.

Circuit

Schematic diagram



Hardware connection. If you need any support, please feel free to contact us via: support@freenove.com



Youtube video: https://youtu.be/rYxykuVgYtA

Code

This project uses the PWM output from the GPIO18 pin to make the pulse width gradually increase from 0% to 100% and then gradually decrease from 100% to 0% to make the LED glow brighter then dimmer.

Python Code 4.1.1 BreathingLED

First, observe the project result, and then learn about the code in detail.

If you have any concerns, please contact us via: support@freenove.com

1. Use cd command to enter 04.1.1_BreathingLED directory of Python code.

```
cd ~/Freenove_Kit/Code/Python_GPIOZero_Code/04.1.1_BreathingLED
```

2. Use the Python command to execute Python code "BreathingLED.py".

```
python BreathingLED.py
```

After the program is executed, you will see that the LED gradually turns ON and then gradually turns OFF similar to "breathing".

The following is the program code:

```
1  from gpiozero import PWMLED
2  import time
3
4  led = PWMLED(18 ,initial_value=0 ,frequency=1000)
5  def loop():
6      while True:
7          for b in range(0, 101, 1):     # make the led brighter
8              led.value = b / 100.0      # set dc value as the duty cycle
```

```
            time.sleep(0.01)
        time.sleep(1)
        for b in range(100, -1, -1):  # make the led darker
            led.value = b / 100.0     # set dc value as the duty cycle
            time.sleep(0.01)
        time.sleep(1)
def destroy():
    led.close()
if __name__ == '__main__':     # Program entrance
    print ('Program is starting ... ')
    try:
        loop()
    except KeyboardInterrupt:  # Press ctrl-c to end the program.
        destroy()
        print("Ending program")
```

Import the PWMLED class that controls leds from the gpiozero library.

```
from gpiozero import PWMLED
```

Create the PWMLED class for controlling the LED.

```
led = PWMLED(18 ,initial_value=0 ,frequency=1000)
```

PWMLED is connected to GPIO18, and its PWM frequency is set to 1000HZ, and the initial duty cycle to 0%.

```
led = PWMLED(18 ,initial_value=0 ,frequency=1000) # Set the PWM frequency to 1000Hz and the initial duty cycle to 0
```

There are two “for” loops used to control the breathing LED in the next endless “while” loop. The first loop outputs a power signal to the led PWM from 0% to 100% and the second loop outputs a power signal to the led PWM from 100% to 0%.

led.value represents:The duty cycle of the PWM device. 0.0 is off, 1.0 is fully on. led.value in between may be specified for varying levels of power in the device.

```
def loop():
    while True:
        for b in range(0, 101, 1):    # make the led brighter
            led.value = b / 100.0     # set dc value as the duty cycle
            time.sleep(0.01)
        time.sleep(1)
        for b in range(100, -1, -1):  # make the led darker
            led.value = b / 100.0     # set dc value as the duty cycle
            time.sleep(0.01)
        time.sleep(1)
```

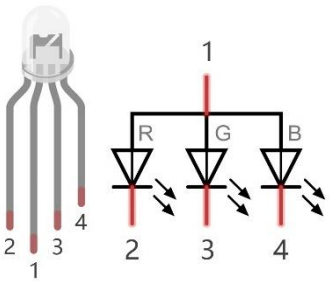
For more information about the methods used by the PWMLED class in the GPIO Zero library,please refer to: https://gpiozero.readthedocs.io/en/stable/api_output.html#pwmled

For more information about the methods used by the PWMOutputDevice class in the GPIO Zero library,please refer to: https://gpiozero.readthedocs.io/en/stable/api_output.html#pwmoutputdevice

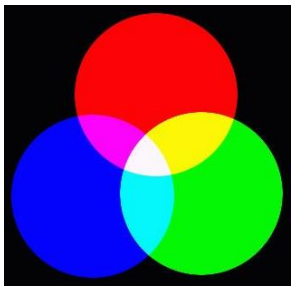
Chapter 5 RGB LED

In this chapter, we will learn how to control a RGB LED.

An RGB LED has 3 LEDs integrated into one LED component. It can respectively emit Red, Green and Blue light. In order to do this, it requires 4 pins (this is also how you identify it). The long pin (1) is the common which is the Anode (+) or positive lead, the other 3 are the Cathodes (-) or negative leads. A rendering of a RGB LED and its electronic symbol are shown below. We can make RGB LED emit various colors of light and brightness by controlling the 3 Cathodes (2, 3 & 4) of the RGB LED



Red, Green, and Blue light are called 3 Primary Colors when discussing light (Note: for pigments such as paints, the 3 Primary Colors are Red, Blue and Yellow). When you combine these three Primary Colors of light with varied brightness, they can produce almost any color of visible light. Computer screens, single pixels of cell phone screens, neon lamps, etc. can all produce millions of colors due to phenomenon.



RGB

If we use a three 8 bit PWM to control the RGB LED, in theory, we can create $2^8*2^8*2^8$=16777216 (16 million) colors through different combinations of RGB light brightness.

Next, we will use RGB LED to make a multicolored LED.

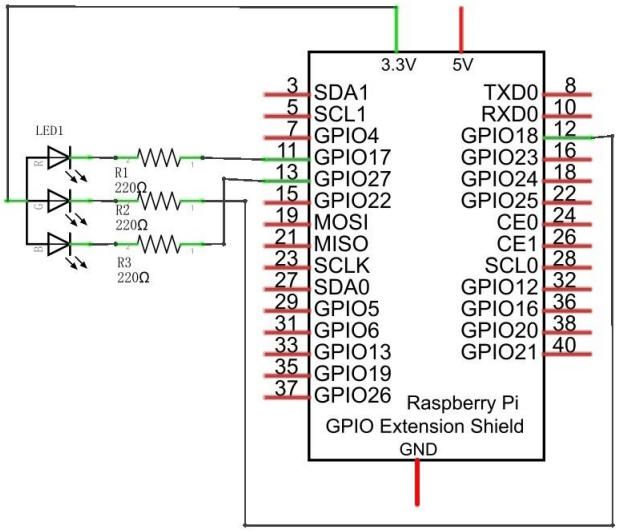
Project 5.1 Multicolored LED

Component List

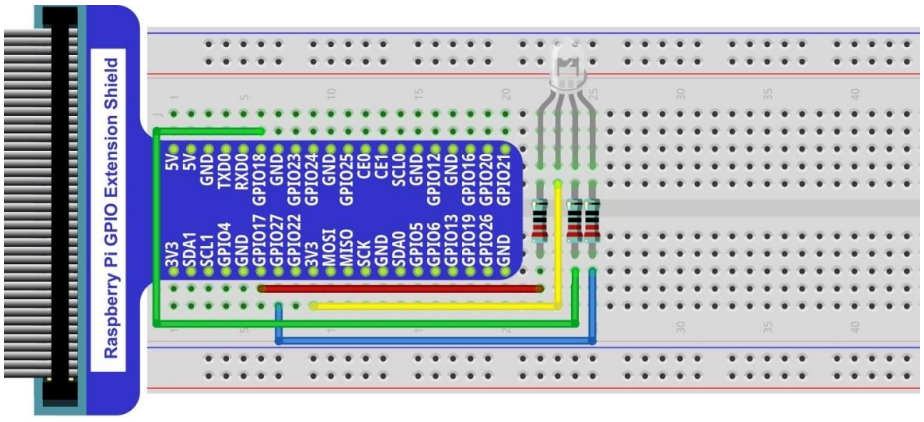
Raspberry Pi (with 40 GPIO) x1 GPIO Extension Board & Wire x1 Breadboard x1	RGB LED x1 	Resistor 220Ω x3 
Jumper Wire 		

Circuit

Schematic diagram



Hardware connection. If you need any support, please feel free to contact us via: support@freenove.com

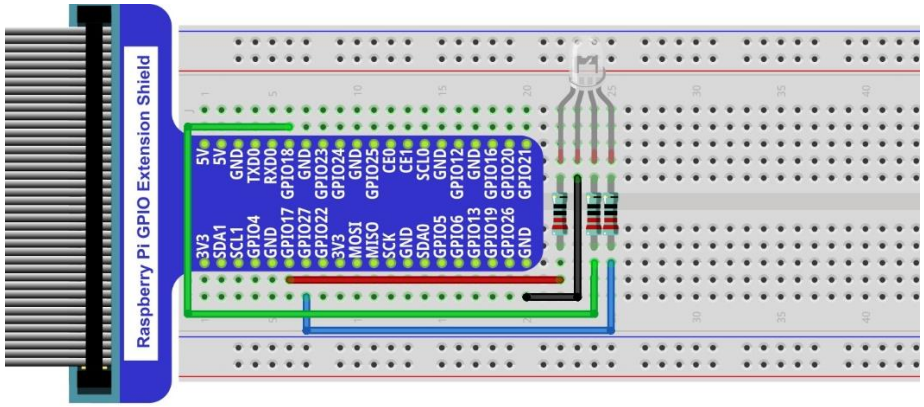


Video: https://youtu.be/tbnX2AsX2y4

In this kit, the RGB led is **Common anode**. The **voltage difference** between LED will make it work. There is no visible GND. The GPIO ports can also receive current while in output mode.

If circuit above doesn't work, the RGB LED may be common cathode. Please try following wiring.

There is no need to modify code for random color.



Code

We need to use RGBLED class to control RGBLED. The parameters for setting the RGBLED as common cathode or common anode are provided in the RGBLED class. You can set it according to the type of your RGB LED, and the default setting in our example code is based on common anode.

Python Code 5.1.1 ColorfulLED

First, observe the project result, and then learn about the code in detail.

If you have any concerns, please contact us via: support@freenove.com

1. Use cd command to enter 05.1.1_ColorfulLED directory of Python code.

```
cd ~/Freenove_Kit/Code/Python_GPIOZero_Code/05.1.1_ColorfulLED
```

2. Use python command to execute python code "ColorfulLED.py".

```
python ColorfulLED.py
```

After the program is executed, you will see that the RGB LED randomly lights up different colors.

The following is the program code:

```
1  from gpiozero import RGBLED
2  import time
3  import random
4
5  #led = RGBLED(red="J8:11", green="J8:12", blue="J8:13", active_high=False) # define the pins
   for R:11,G:12,B:13
6  led = RGBLED(red=17, green=18, blue=27, active_high=False) # define the pins for
   R:GPIO17,G:GPIO18,B:GPIO27
7  # If your RGBLED is a common cathode LED, set active_high to True
8
9  def setColor(r_val,g_val,b_val):      # change duty cycle for three pins to r_val,g_val,b_val
10     led.red=r_val/100                 # change pwmRed duty cycle to r_val
11     led.green = g_val/100             # change pwmRed duty cycle to r_val
12     led.blue = b_val/100              # change pwmRed duty cycle to r_val
13
14 def loop():
```

```
    while True :
        r=random.randint(0,100)   #get a random in (0,100)
        g=random.randint(0,100)
        b=random.randint(0,100)
        setColor(r,g,b)           #set random as a duty cycle value
        print ('r=%d, g=%d, b=%d ' %(r ,g, b))
        time.sleep(1)

def destroy():
    led.close()

if __name__ == '__main__':     # Program entrance
    print ('Program is starting ... ')
    try:
        loop()
    except KeyboardInterrupt:  # Press ctrl-c to end the program.
        destroy()
        print("Ending program")
```

Import the RGBLED class that controls RGBLED from the gpiozero library.

```
from gpiozero import RGBLED
```

Create the RGBLED class for controlling the RGBLED.

```
led = RGBLED(red=17, green=18, blue=27, active_high=False) # define the pins for
R:GPIO17,G:GPIO18,B:GPIO27
```

In the previous chapter, we learned how to make a pin output PWM using the Python language. In this project, we output to three pins via PWM. In the "while" loop of the "loop" function, we first generate three random numbers and then assign these three random numbers to the PWM values of the three pins, which will make the RGB LED randomly produce multiple colors.

```
def loop():
    while True :
        r=random.randint(0,100)   #get a random in (0,100)
        g=random.randint(0,100)
        b=random.randint(0,100)
        setColor(r,g,b)           #set random as a duty cycle value
        print ('r=%d, g=%d, b=%d ' %(r ,g, b))
        time.sleep(1)
```

For more information about the methods used by the RGBLED class in the GPIO Zero library,please refer to: https://gpiozero.readthedocs.io/en/stable/api_output.html#rgbled

Chapter 6 Buzzer

In this chapter, we will learn about buzzers and the sounds they make. And in our next project, we will use an active buzzer to make a doorbell and a passive buzzer to make an alarm.

Project 6.1 Doorbell

We will make a doorbell with this functionality: when the Push Button Switch is pressed the buzzer sounds and when the button is released, the buzzer stops. This is a momentary switch function.

Component List

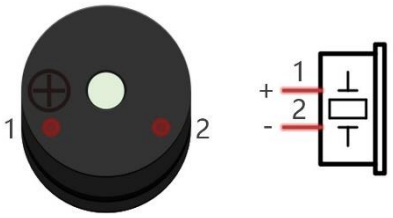
<table>
<tr><td colspan="2">Raspberry Pi (with 40 GPIO) x1
GPIO Extension Board & Ribbon Cable x1
Breadboard x1</td><td colspan="3">Jumper Wire
  </td></tr>
<tr><td>NPN transistorx1
(S8050)
  </td><td>Active buzzer x1
  </td><td>Push Button Switch x1
  </td><td>Resistor 1kΩ x1
  </td><td>Resistor 10kΩ x2
  </td></tr>
</table>

Component knowledge

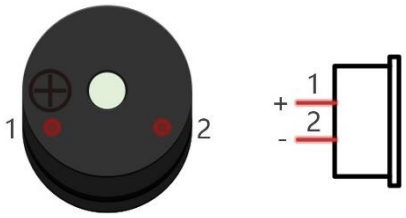
Buzzer

A buzzer is an audio component. They are widely used in electronic devices such as calculators, electronic alarm clocks, automobile fault indicators, etc. There are both active and passive types of buzzers. Active buzzers have oscillator inside, these will sound as long as power is supplied. Passive buzzers require an external oscillator signal (generally using PWM with different frequencies) to make a sound.

Active buzzer



Passive buzzer



Active buzzers are easier to use. Generally, they only make a specific sound frequency. Passive buzzers require an external circuit to make sounds, but passive buzzers can be controlled to make sounds of various frequencies. The resonant frequency of the passive buzzer in this Kit is 2kHz, which means the passive buzzer is the loudest when its resonant frequency is 2kHz.

How to identify active and passive buzzer?

1. As a rule, there is a label on an active buzzer covering the hole where sound is emitted, but there are exceptions to this rule.
2. Active buzzers are more complex than passive buzzers in their manufacture. There are many circuits and crystal oscillator elements inside active buzzers; all of this is usually protected with a waterproof coating (and a housing) exposing only its pins from the underside. On the other hand, passive buzzers do not have protective coatings on their underside. From the pin holes, view of a passive buzzer, you can see the circuit board, coils, and a permanent magnet (all or any combination of these components depending on the model.

Active buzzer bottom

Passive buzzer bottom

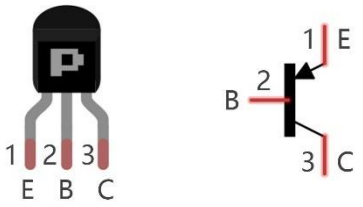
Transistors

A transistor is required in this project due to the buzzer's current being so great that GPIO of RPi's output capability cannot meet the power requirement necessary for operation. A NPN transistor is needed here to

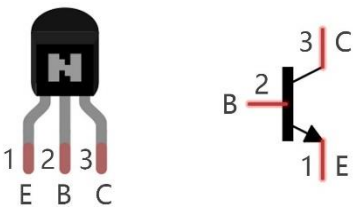
amplify the current.

Transistors, full name: semiconductor transistor, is a semiconductor device that controls current (think of a transistor as an electronic “amplifying or switching device”. Transistors can be used to amplify weak signals, or to work as a switch. Transistors have three electrodes (PINs): base (b), collector (c) and emitter (e). When there is current passing between "be" then "ce" will have a several-fold current increase (transistor magnification), in this configuration the transistor acts as an amplifier. When current produced by "be" exceeds a certain value, "ce" will limit the current output. at this point the transistor is working in its saturation region and acts like a switch. Transistors are available as two types as shown below: PNP and NPN,

PNP transistor



NPN transistor



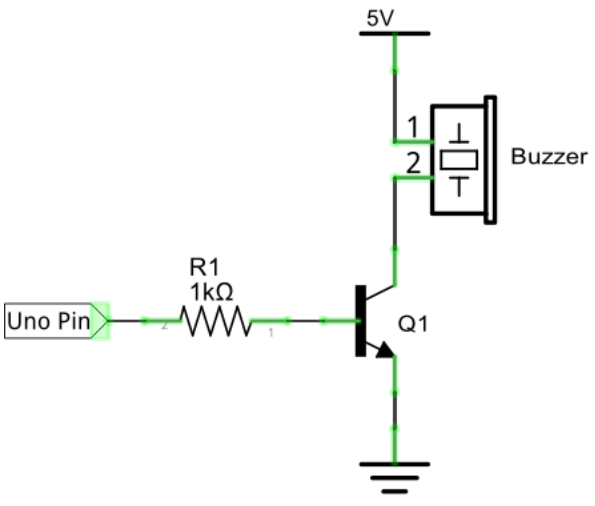
In our kit, the PNP transistor is marked with 8550, and the NPN transistor is marked with 8050.

Thanks to the transistor's characteristics, they are often used as switches in digital circuits. As micro-controllers output current capacity is very weak, we will use a transistor to amplify its current in order to drive components requiring higher current.

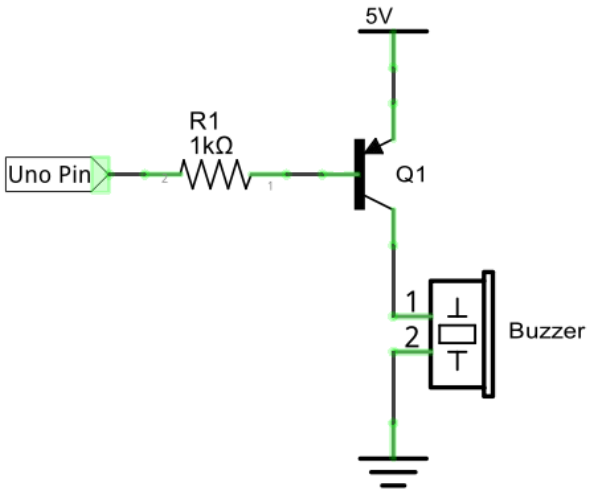
When we use a NPN transistor to drive a buzzer, we often use the following method. If GPIO outputs high level, current will flow through R1 (Resistor 1), the transistor conducts current and the buzzer will make sounds. If GPIO outputs low level, no current will flow through R1, the transistor will not conduct currentand buzzer will remain silent (no sounds).

When we use a PNP transistor to drive a buzzer, we often use the following method. If GPIO outputs low level, current will flow through R1. The transistor conducts current and the buzzer will make sounds. If GPIO outputs high level, no current flows through R1, the transistor will not conduct current and buzzer will remain silent (no sounds). Below are the circuit schematics for both a NPN and PNP transistor to power a buzzer.

NPN transistor to drive buzzer

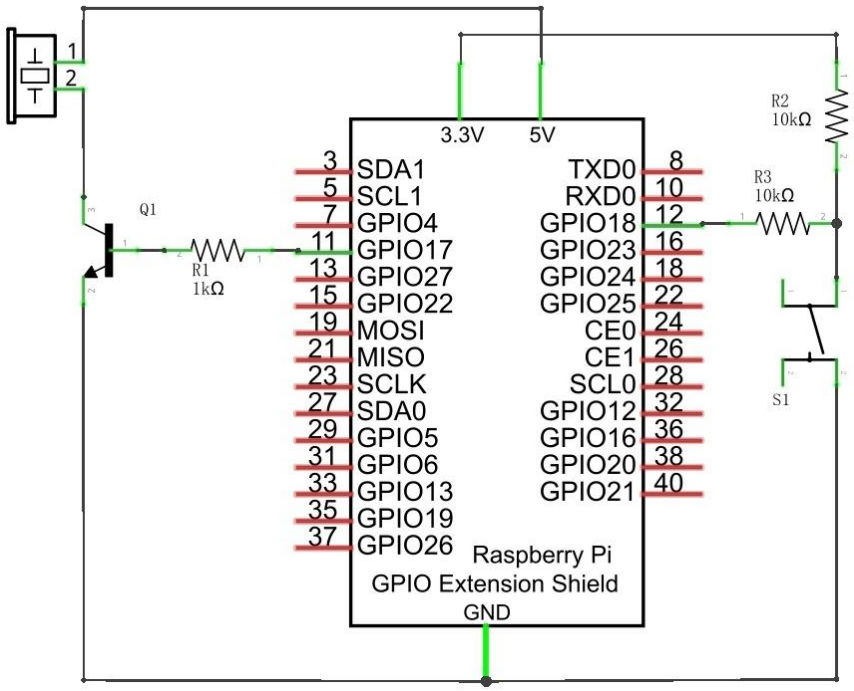


PNP transistor to drive buzzer

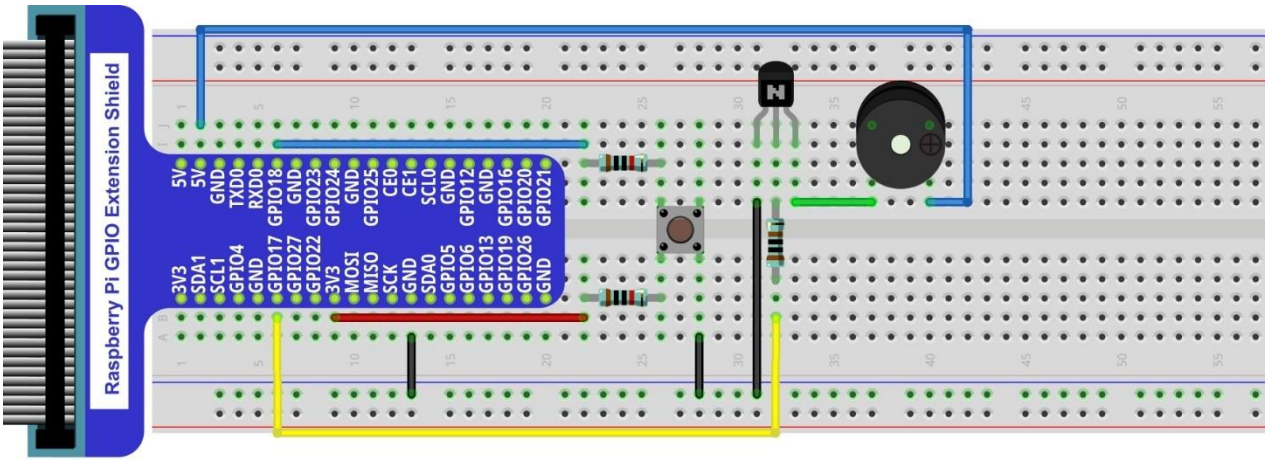


Circuit

Schematic diagram with RPi GPIO Extension Shield.



Hardware connection. If you need any support, please feel free to contact us via: support@freenove.com



Video: https://youtu.be/R_dmi3YwY-U

Note: in this circuit, the power supply for the buzzer is 5V, and pull-up resistor of the push button switch is connected to the 3.3V power feed. Actually, the buzzer can work when connected to the 3.3V power feed but this will produce a weak sound from the buzzer (not very loud).

Code

In this project, a buzzer will be controlled by a push button switch. When the button switch is pressed, the buzzer sounds and when the button is released, the buzzer stops. It is analogous to our earlier project that controlled an LED ON and OFF.

Python Code 6.1.1 Doorbell

First, observe the project result, then learn about the code in detail.

If you have any concerns, please contact us via: support@freenove.com

1. Use cd command to enter 06.1.1_Doorbell directory of Python code.

```
cd ~/Freenove_Kit/Code/Python_GPIOZero_Code/06.1.1_Doorbell
```

2. Use python command to execute python code "Doorbell.py".

```
python Doorbell.py
```

After the program is executed, press the push button switch and the buzzer will sound. Release the push button switch and the buzzer will stop.

The following is the program code:

```
1  from gpiozero import Buzzer, Button
2  import time
3
4  buzzer = Buzzer(17)
5  button = Button(18)
6
7  def onButtonPressed():
8      buzzer.on()
9      print("Button is pressed, buzzer turned on >>>")
10
11 def onButtonReleased():
12     buzzer.off()
13     print("Button is released, buzzer turned on <<<")
14
15 def loop():
16     button.when_pressed = onButtonPressed
17     button.when_released = onButtonReleased
18     while True :
19         time.sleep(1)
20
21 def destroy():
22     buzzer.close()
23     button.close()
24
25 if __name__ == '__main__':     # Program entrance
26     print ('Program is starting ... ')
27     try:
28         loop()
29     except KeyboardInterrupt:  # Press ctrl-c to end the program.
30         destroy()
31         print("Ending program")
```

The code is exactly the same as when we used a push button switch to control an LED. You can also try using

the PNP transistor to achieve the same results.

Import the Buzzer class that controls Buzzer from the gpiozero library.

```
from gpiozero import Buzzer, Button
```

Create the Buzzer class for controlling the Buzzer.

```
buzzer = Buzzer(17)
```

In GPIO Zero, you assign the when_pressed and when_released properties to set up callbacks on those actions.

Once it detects that the button is pressed, it executes the specified function onButtonPressed(). Once it detects that the button is released, it executes the specified function onButtonReleased()

```
def loop():
    button.when_pressed = onButtonPressed
    button.when_released = onButtonReleased
```

For more information about the methods used by the Buzzer class in the GPIO Zero library,please refer to: https://gpiozero.readthedocs.io/en/stable/api_output.html#buzzer

Project 6.2 Alertor

Next, we will use a passive buzzer to make an alarm.
The list of components and the circuit is similar to the doorbell project. We only need to take the Doorbell circuit and replace the active buzzer with a passive buzzer.

Code

In this project, our buzzer alarm is controlled by the push button switch. Press the push button switch and the buzzer will sound. Release the push button switch and the buzzer will stop.
As stated before, it is analogous to our earlier project that controlled an LED ON and OFF.
To control a passive buzzer requires PWM of certain sound frequency.

Python Code 6.2.1 Alertor

First observe the project result, and then learn about the code in detail.
If you have any concerns, please contact us via: support@freenove.com

1. Use cd command to enter 06.2.1_Alertor directory of Python code.

```
cd ~/Freenove_Kit/Code/Python_GPIOZero_Code/06.2.1_Alertor
```

2. Use the python command to execute the Python code "Alertor.py".

```
python Alertor.py
```

After the program is executed, press the push button switch and the buzzer will sound. Release the push button switch and the buzzer will stop.
The following is the program code:

```
1  from gpiozero import TonalBuzzer,Button
2  from gpiozero.tones import Tone
3  import time
4  import math
5
6  buzzer = TonalBuzzer(17)
7  button = Button(18) # define Button pin according to BCM Numbering
8
9  def loop():
10     while True:
11         if button.is_pressed:  # if button is pressed
12             alertor()
13             print ('alertor turned on >>> ')
14         else :
15             stopAlertor()
16             print ('alertor turned off <<<')
17 def alertor():
18     for x in range(0,361):      # Make frequency of the alertor consistent with the sine wave
19         sinVal = math.sin(x * (math.pi / 180.0))        # calculate the sine value
```

```
20          toneVal = 2000 + sinVal * 500   # Add to the resonant frequency with a Weighted
21          buzzer.play(Tone(toneVal))  # Change Frequency of PWM to toneVal
22          time.sleep(0.001)
23
24  def stopAlertor():
25      buzzer.stop()
26
27  def destroy():
28      buzzer.close()
29
30  if __name__ == '__main__':     # Program entrance
31      print ('Program is starting...')
32      try:
33          loop()
34      except KeyboardInterrupt:  # Press ctrl-c to end the program.
35          destroy()
36          print("Ending program")
```

Define GPIO17 as the buzzer control pin, and GPIO18 as the button control pin to control the passive buzzer. It requires a certain frequency of PWM to control a passive buzzer, so the TonalBuzzer class is needed.

```
buzzer = TonalBuzzer(17)
button = Button(18) # define Button pin according to BCM Numbering
```

In the while loop loop of the main function, when the push button switch is pressed the subfunction alertor() will be called and the alarm will issue a warning sound. The frequency curve of the alarm is based on a sine curve. We need to calculate the sine value from 0 to 360 degrees and multiplied by a certain value (here this value is 500) plus the resonant frequency of buzzer. We can set the PWM frequency through Tone(toneVal).

```
def alertor():
    buzzer.play(Tone(220.0))
    time.sleep(0.1)
```

When the push button switch is released, the buzzer (in this case our Alarm) will stop.

```
def stopAlertor():
    buzzer.stop()
```

For more information about the methods used by the TonalBuzzer class in the GPIO Zero library,please refer to: https://gpiozero.readthedocs.io/en/stable/api_output.html#tonalbuzzer

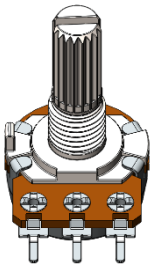
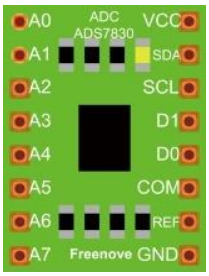
(Important) Chapter 7 ADC

We have learned how to control the brightness of an LED through PWM and that PWM is not a real analog signal. In this chapter, we will learn how to read analog values via an ADC Module and convert these analog values into digital.

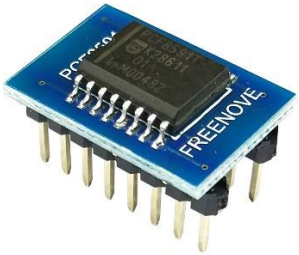
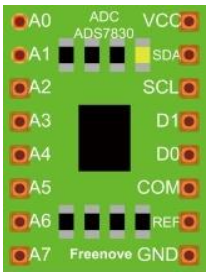
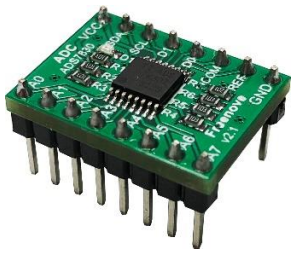
Project 7.1 Read the Voltage of Potentiometer

In this project, we will use the ADC function of an ADC Module to read the voltage value of a potentiometer.

Component List

<table>
<tr><td colspan="2">Raspberry Pi x1
GPIO Extension Board & Ribbon Cable x1
Breadboard x1</td><td>Jumper Wire M/M x1
  </td></tr>
<tr><td>Rotary potentiometer x1
  </td><td>ADC module x1
 
or
  </td><td>Resistor 10kΩ x2
  </td></tr>
</table>

This product contains **only one ADC module**, there are two types, PCF8591 and ADS7830. For the projects described in this tutorial, they function the same. Please build corresponding circuits according to the ADC module found in your Kit.

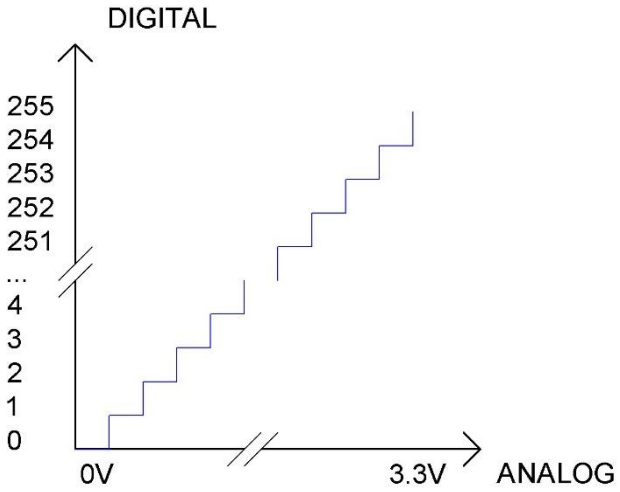
ADC module: PCF8591		**ADC module: ADS7830**	
Model diagram	Actual Picture	Model diagram	Actual Picture
			

Circuit knowledge

ADC

An ADC is an electronic integrated circuit used to convert analog signals such as voltages to digital or binary form consisting of 1s and 0s. The range of our ADC module is 8 bits, that means the resolution is 2^8=256, so that its range (at 3.3V) will be divided equally to 256 parts.

Any analog value can be mapped to one digital value using the resolution of the converter. So the more bits the ADC has, the denser the partition of analog will be and the greater the precision of the resulting conversion.



Subsection 1: the analog in range of 0V-3.3/256 V corresponds to digital 0;

Subsection 2: the analog in range of 3.3 /256 V-2*3.3 /256V corresponds to digital 1;

...

The resultant analog signal will be divided accordingly.

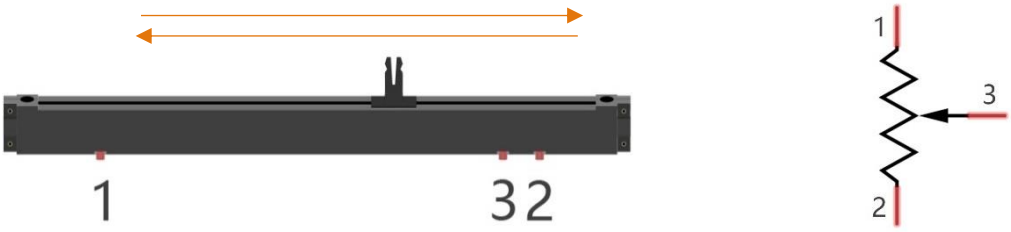
DAC

The reversing this process requires a DAC, Digital-to-Analog Converter. The digital I/O port can output high level and low level (0 or 1), but cannot output an intermediate voltage value. This is where a DAC is useful. The DAC module PCF8591 has a DAC output pin with 8-bit accuracy, which can divide VDD (here is 3.3V) into 2^8=256 parts. For example, when the digital quantity is 1, the output voltage value is 3.3/256 *1 V, and when the digital quantity is 128, the output voltage value is 3.3/256 *128=1.65V, the higher the accuracy of DAC, the higher the accuracy of output voltage value will be.

Component knowledge

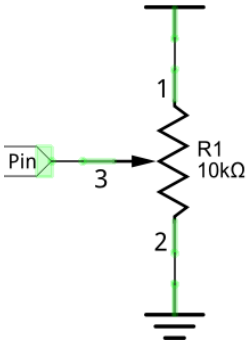
Potentiometer

Potentiometer is a resistive element with three Terminal parts. Unlike the resistors that we have used thus far in our project which have a fixed resistance value, the resistance value of a potentiometer can be adjusted. A potentiometer is often made up by a resistive substance (a wire or carbon element) and movable contact brush. When the brush moves along the resistor element, there will be a change in the resistance of the potentiometer's output side (3) (or change in the voltage of the circuit that is a part). The illustration below represents a linear sliding potentiometer and its electronic symbol on the right.



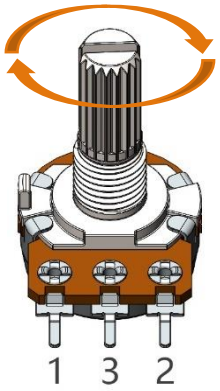
Between potentiometer pin 1 and pin 2 is the resistive element (a resistance wire or carbon) and pin 3 is connected to the brush that makes contact with the resistive element. In our illustration, when the brush moves from pin 1 to pin 2, the resistance value between pin 1 and pin 3 will increase linearly (until it reaches the highest value of the resistive element) and at the same time the resistance between pin 2 and pin 3 will decrease linearly and conversely down to zero. At the midpoint of the slider the measured resistance values between pin 1 and 3 and between pin 2 and 3 will be the same.

In a circuit, both sides of resistive element are often connected to the positive and negative electrodes of power. When you slide the brush "pin 3", you can get variable voltage within the range of the power supply.



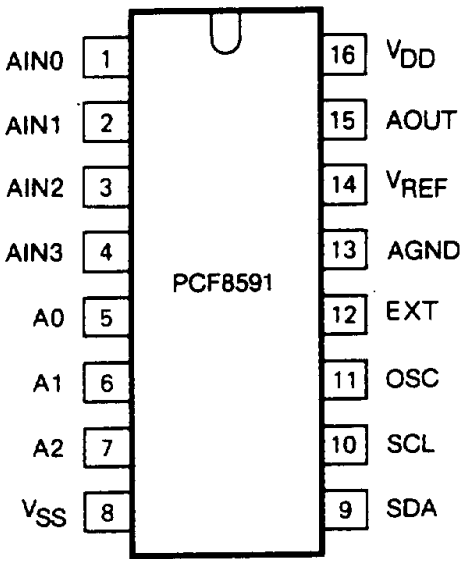
Rotary potentiometer

Rotary potentiometers and linear potentiometers have the same function; the only difference being the physical action being a rotational rather than a sliding movement.



PCF8591

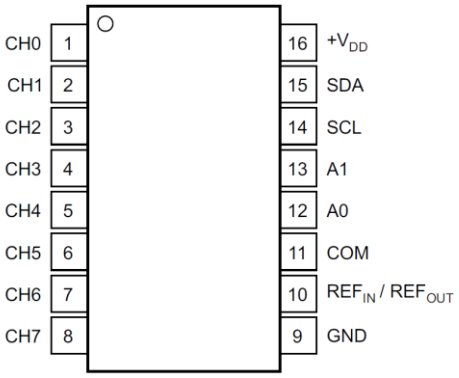
The PCF8591 is a single-chip, single-supply low power 8-bit CMOS data acquisition device with four analog inputs, one analog output and a serial I2C-bus interface. The following table is the pin definition diagram of PCF8591.

SYMBOL	PIN	DESCRIPTION	TOP VIEW
AIN0	1	Analog inputs (A/D converter)	
AIN1	2		
AIN2	3		
AIN3	4		
A0	5	Hardware address	
A1	6		
A2	7		
Vss	8	Negative supply voltage	
SDA	9	I2C-bus data input/output	
SCL	10	I2C-bus clock input	
OSC	11	Oscillator input/output	
EXT	12	external/internal switch for oscillator input	
AGND	13	Analog ground	
Vref	14	Voltage reference input	
AOUT	15	Analog output(D/A converter)	
Vdd	16	Positive supply voltage	

For more details about PCF8591, please refer to the datasheet which can be found on the Internet.

ADS7830

The ADS7830 is a single-supply, low-power, 8-bit data acquisition device that features a serial I2C interface and an 8-channel multiplexer. The following table is the pin definition diagram of ADS7830.

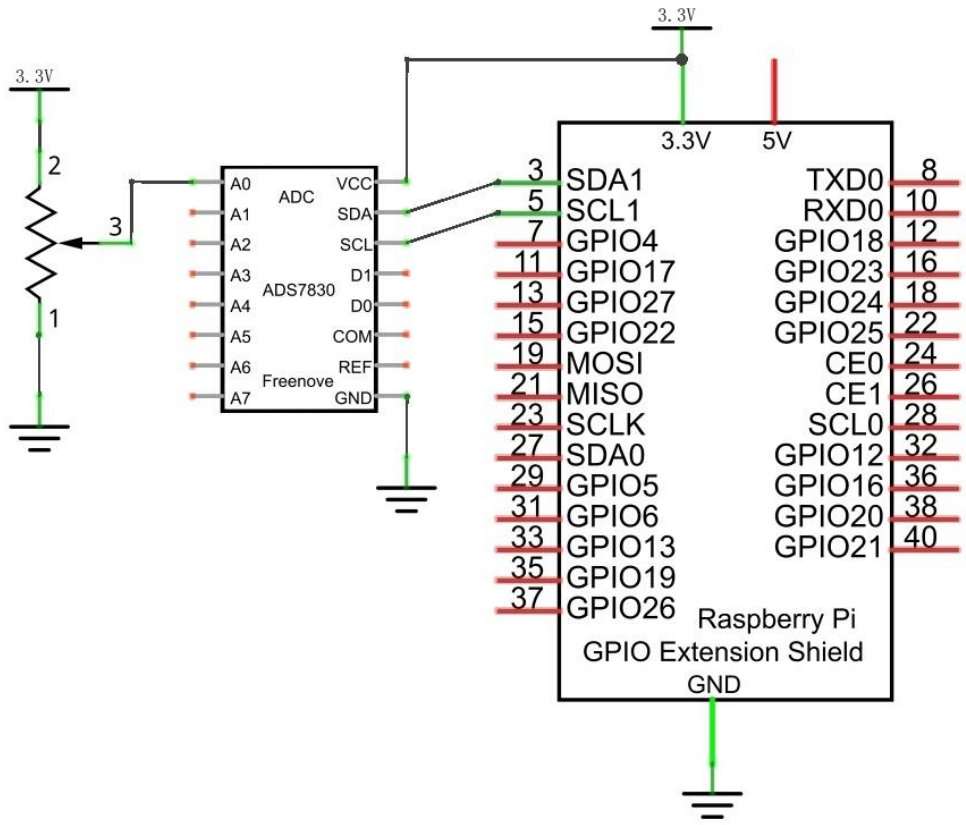
SYMBOL	PIN	DESCRIPTION	TOP VIEW
CH0	1	Analog input channels (A/D converter)	
CH1	2		
CH2	3		
CH3	4		
CH4	5		
CH5	6		
CH6	7		
CH7	8		
GND	9	Ground	
REF in/out	10	Internal +2.5V Reference, External Reference Input	
COM	11	Common to Analog Input Channel	
A0	12	Hardware address	
A1	13		
SCL	14	Serial Clock	
SDA	15	Serial Sata	
+VDD	16	Power Supply, 3.3V Nominal	

I2C communication

I2C (Inter-Integrated Circuit) has a two-wire serial communication mode, which can be used to connect a micro-controller and its peripheral equipment. Devices using I2C communications must be connected to the serial data line (SDA), and serial clock line (SCL) (called I2C bus). Each device has a unique address which can be used as a transmitter or receiver to communicate with devices connected via the bus.

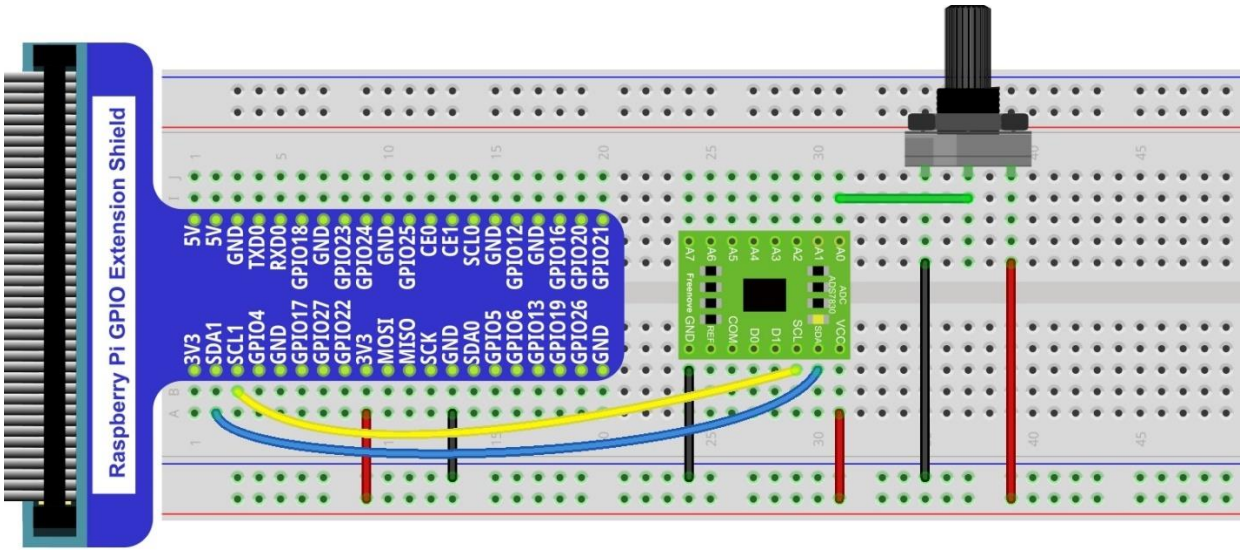
Circuit with ADS7830

Schematic diagram



Hardware connection. If you need any support, please feel free to contact us via: support@freenove.com

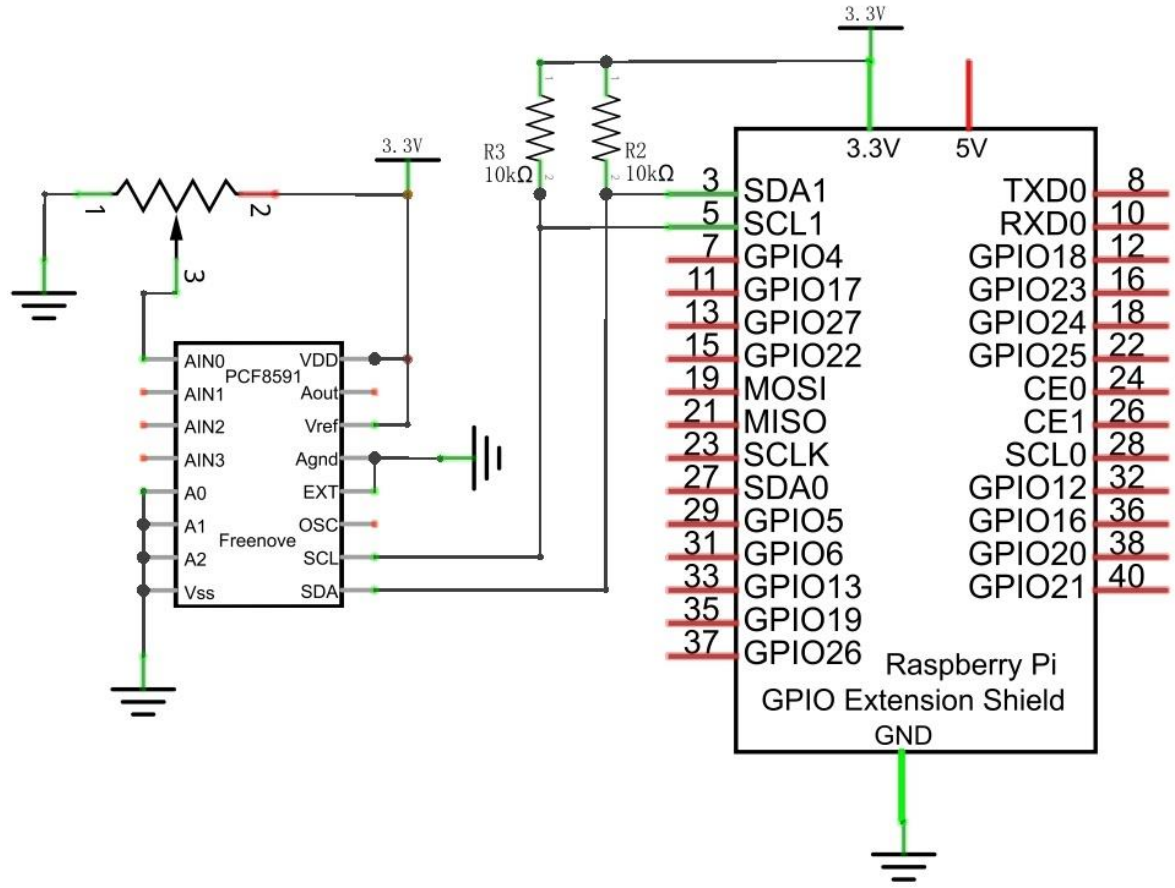
This product contains **only one ADC module.**



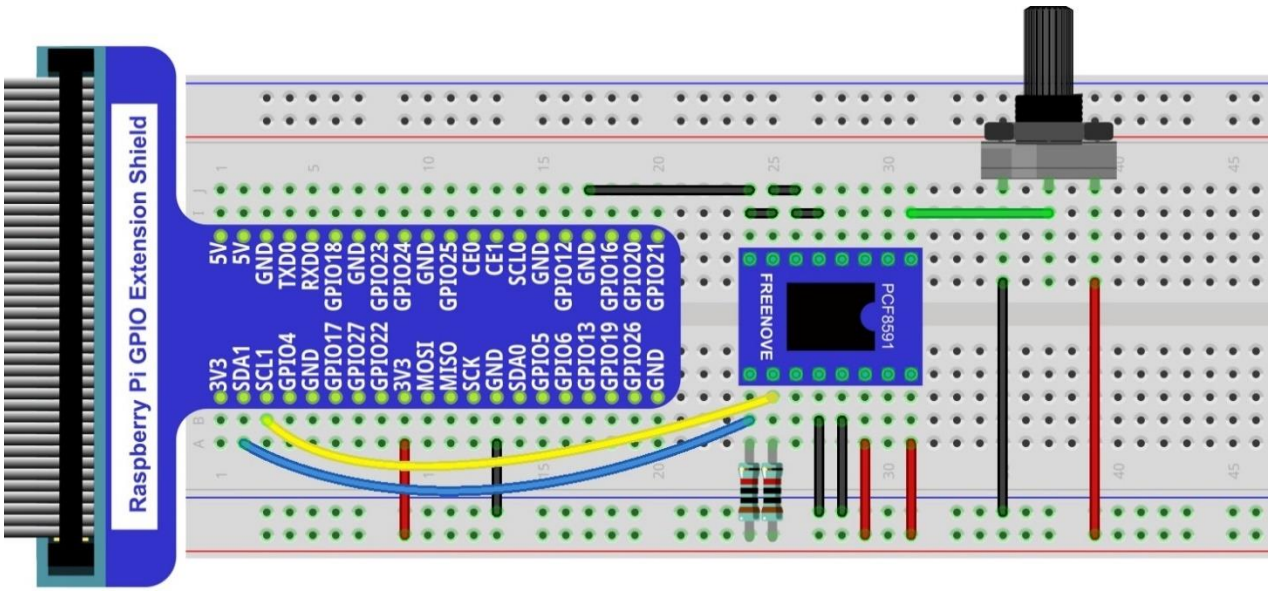
Video: https://youtu.be/PSUCctu_DqA

Circuit with PCF8591

Schematic diagram



Hardware connection



Please keep the **chip mark** consistent to make the chips under right direction and position.

Configure I2C and Install Smbus

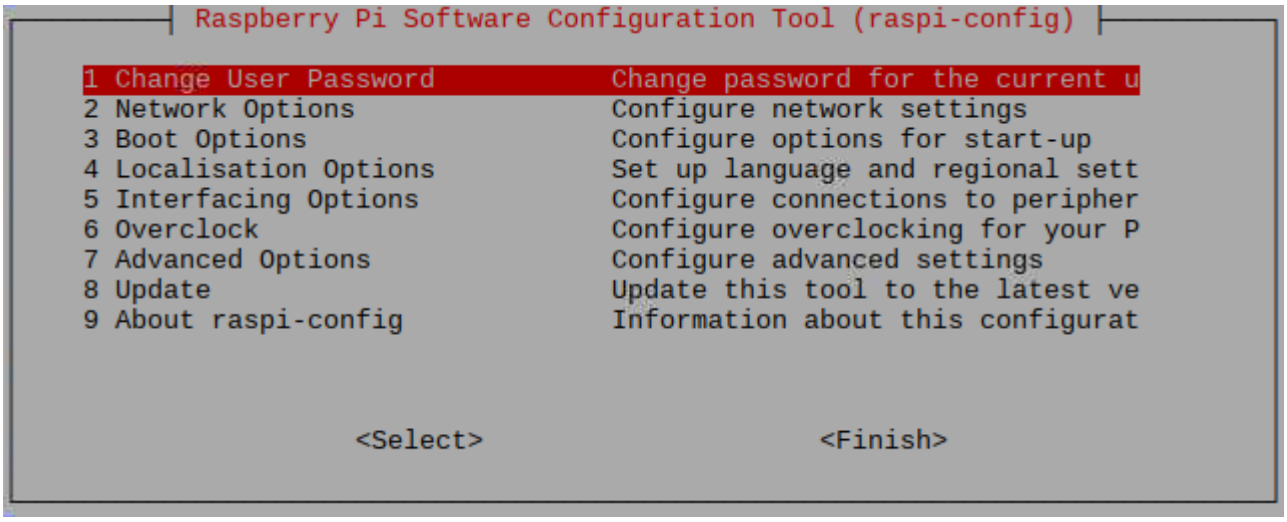
Enable I2C

The I2C interface in Raspberry Pi is disabled by default. You will need to open it manually and enable the I2C interface as follows:

Type command in the Terminal:

```
sudo raspi-config
```

Then open the following dialog box:



Choose “5 Interfacing Options” then “P5 I2C” then “Yes” and then “Finish” in this order and restart your RPi. The I2C module will then be started.

Type a command to check whether the I2C module is started:

```
lsmod | grep i2c
```

If the I2C module has been started, the following content will be shown. “bcm2708" refers to the CPU model. Different models of Raspberry Pi display different contents depending on the CPU installed:

Install I2C-Tools

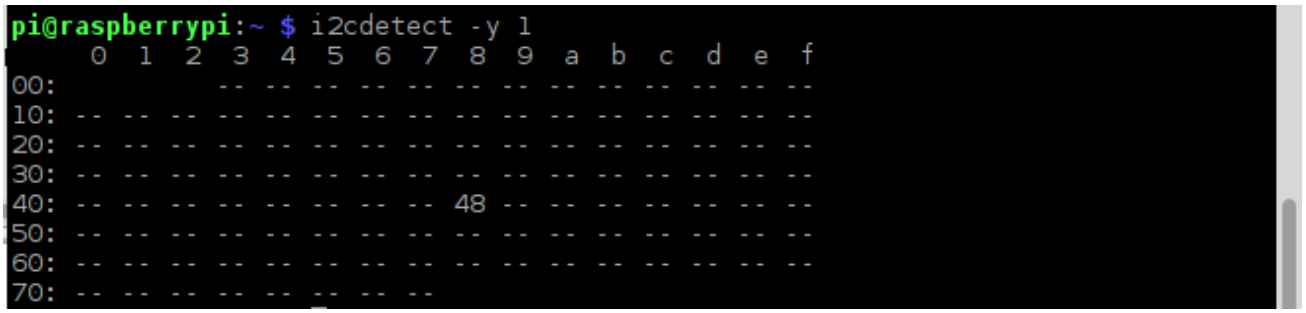
Next, type the command to install I2C-Tools. It is available with the Raspberry Pi OS by default.

```
sudo apt-get install i2c-tools
```

I2C device address detection:

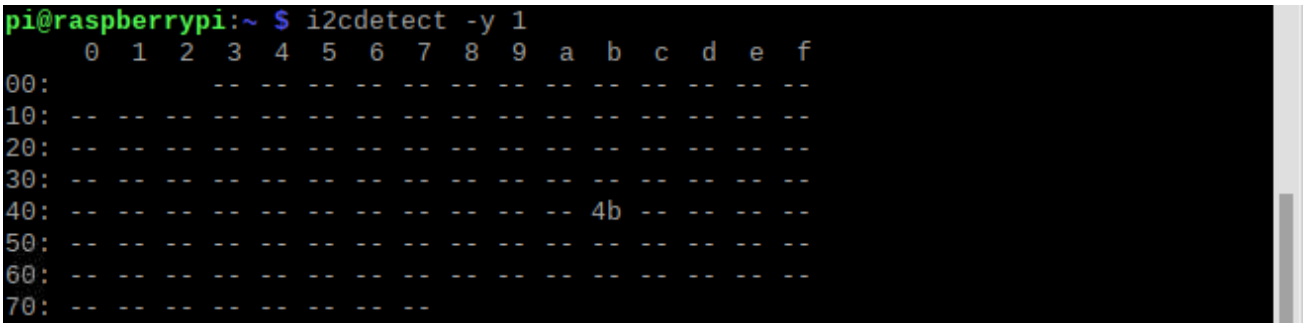
```
i2cdetect -y 1
```

When you are using the PCF8591 Module, the result should look like this:



Here, 48 (HEX) is the I2C address of ADC Module (PCF8591).

When you are using ADS, the result should look like this:



Here, 4b (HEX) is the I2C address of ADC Module (ADS7830).

Install Smbus Module

```
sudo apt-get install python-smbus
sudo apt-get install python3-smbus
```

Code

Python Code 7.1.1 ADC

For Python code, ADCDevice requires a custom module which needs to be installed.

1. Use cd command to enter folder of ADCDevice.

```
cd ~/Freenove_Kit/Libs/Python-Libs/
```

2. Unzip the file.

```
tar -zxvf ADCDevice-1.0.3.tar.gz
```

3. Open the unzipped folder.

```
cd ADCDevice-1.0.3
```

4. Install library for python3 and python2.

```
sudo python3 setup.py install
sudo python2 setup.py install
```

A successful installation, without error prompts, is shown below:

```
Installed /usr/local/lib/python3.7/dist-packages/ADCDevice-1.0.2-py3.7.egg
Processing dependencies for ADCDevice==1.0.2
Finished processing dependencies for ADCDevice==1.0.2
```

Execute the following command. Observe the project result and then learn about the code in detail.

If you have any concerns, please contact us via: support@freenove.com

1. Use cd command to enter 07.1.1_ADC directory of Python code.

```
cd ~/Freenove_Kit/Code/Python_GPIOZero_Code/07.1.1_ADC
```

2. Use the Python command to execute the Python code "ADC.py".

```
python ADC.py
```

After the program is executed, adjusting the potentiometer will produce a readout display of the potentiometer voltage values in the Terminal and the converted digital content.

```
ADC Value : 168, Voltage : 2.17
ADC Value : 168, Voltage : 2.17
ADC Value : 168, Voltage : 2.17
ADC Value : 168, Voltage : 2.17
ADC Value : 168, Voltage : 2.17
ADC Value : 168, Voltage : 2.17
ADC Value : 168, Voltage : 2.17
ADC Value : 168, Voltage : 2.17
ADC Value : 169, Voltage : 2.19
ADC Value : 168, Voltage : 2.17
ADC Value : 168, Voltage : 2.17
```

The following is the code:

```
1  import time
2  from ADCDevice import *
3
4  adc = ADCDevice() # Define an ADCDevice class object
5
6  def setup():
7      global adc
8      if(adc.detectI2C(0x48)): # Detect the pcf8591.
9          adc = PCF8591()
10     elif(adc.detectI2C(0x4b)): # Detect the ads7830
11         adc = ADS7830()
12     else:
13         print("No correct I2C address found, \n"
14         "Please use command 'i2cdetect -y 1' to check the I2C address! \n"
15         "Program Exit. \n");
```

```
        exit(-1)

def loop():
    while True:
        value = adc.analogRead(0)    # read the ADC value of channel 0
        voltage = value / 255.0 * 3.3  # calculate the voltage value
        print ('ADC Value : %d, Voltage : %.2f'%(value,voltage))
        time.sleep(0.1)

def destroy():
    adc.close()

if __name__ == '__main__':   # Program entrance
    print ('Program is starting ... ')
    try:
        setup()
        loop()
    except KeyboardInterrupt: # Press ctrl-c to end the program.
        destroy()
```

In this code, a custom Python module "ADCDevice" is used. It contains the method of utilizing the ADC Module in this project, through which the ADC Module can easily and quickly be used. In the code, you need to first create an ADCDevice object adc.

```
adc = ADCDevice() # Define an ADCDevice class object
```

Then in setup(), use detecticIC(addr), the member function of ADCDevice, to detect the I2C module in the circuit. Different modules have different I2C addresses. Therefore, according to the address, we can determine which ADC Module is in the circuit. When the correct module is detected, a device specific class object is created and assigned to adc. The default address of PCF8591 is 0x48, and that of ADS7830 is 0x4b.

```
def setup():
    global adc
    if(adc.detectI2C(0x48)): # Detect the pcf8591.
        adc = PCF8591()
    elif(adc.detectI2C(0x4b)): # Detect the ads7830
        adc = ADS7830()
    else:
        print("No correct I2C address found, \n"
        "Please use command 'i2cdetect -y 1' to check the I2C address! \n"
        "Program Exit. \n");
        exit(-1)
```

When you have a class object of a specific device, you can get the ADC value of the specified channel by calling the member function of this class, analogRead(chn). In loop(), get the ADC value of potentiometer.

```
        value = adc.analogRead(0)    # read the ADC value of channel 0
```

Then according to the formula, the voltage value is calculated and displayed on the terminal monitor.

```
        voltage = value / 255.0 * 3.3  # calculate the voltage value
        print ('ADC Value : %d, Voltage : %.2f'%(value,voltage))
        time.sleep(0.1)
```

Reference

About smbus Module:

smbus Module

The System Management Bus Module defines an object type that allows SMBus transactions on hosts running the Linux kernel. The host kernel must support I2C, I2C device interface support, and a bus adapter driver. All of these can be either built-in to the kernel, or loaded from modules.

In Python, you can use help(smbus) to view the relevant functions and their descriptions.

bus=smbus.SMBus(1): Create an SMBus class object.

bus.read_byte_data(address,cmd+chn): Read a byte of data from an address and return it.

bus.write_byte_data(address,cmd,value): Write a byte of data to an address.

class ADCDevice(object)

This is a base class.

```
int detectI2C(int addr);
```

This is a member function, which is used to detect whether the device with the given I2C address exists. If it exists, it returns true. Otherwise, it returns false.

```
class PCF8591(ADCDevice)
class ADS7830(ADCDevice)
```

These two classes are derived from the ADCDevice and the main function is analogRead(chn).

```
int analogRead(int chn);
```

This returns the value read on the supplied analog input pin.

Parameter chn: For PCF8591, the range of chn is 0, 1, 2, 3. For ADS7830, the range is 0, 1, 2, 3, 4, 5, 6, 7.

You can find the source file of this library in the folder below:

```
~/Freenove_Kit/Libs/Python-Libs/ADCDevice-1.0.3/src/ADCDevice/ADCdevice.py
```

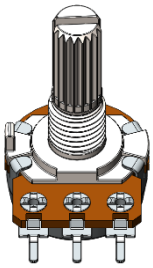
Chapter 8 Potentiometer & LED

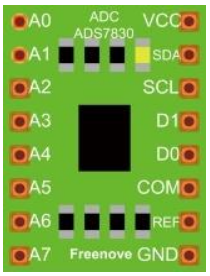
Earlier we learned how to use ADC and PWM. In this chapter, we learn to control the brightness of an LED by using a potentiometer.

Project 8.1 Soft Light

In this project, we will make a soft light. We will use an ADC Module to read ADC values of a potentiometer and map it to duty cycle ratio of the PWM used to control the brightness of an LED. Then you can change the brightness of an LED by adjusting the potentiometer.

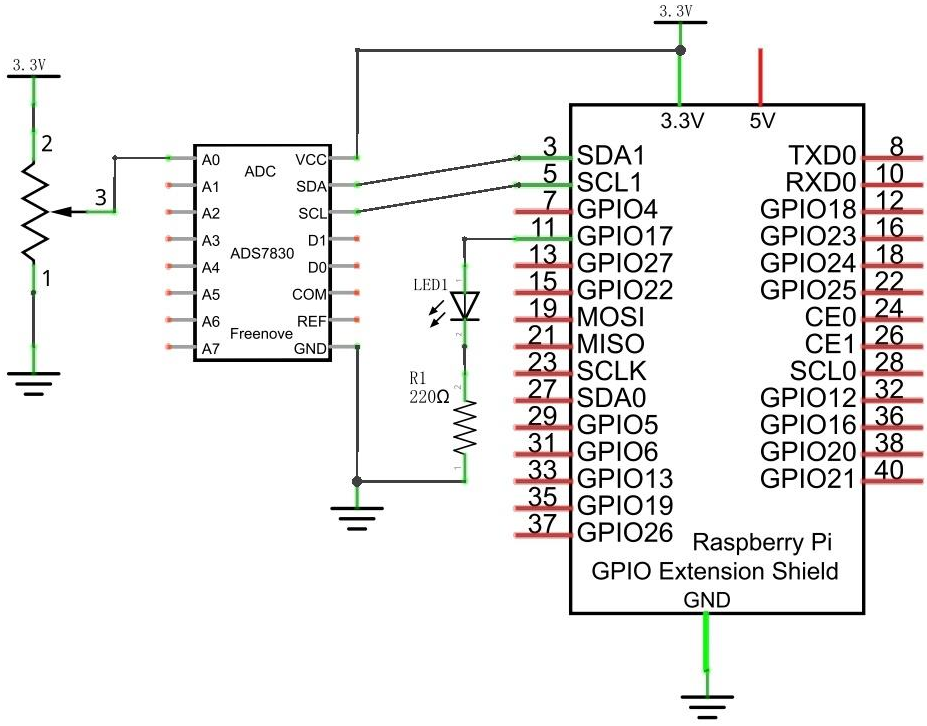
Component List

<table>
<tr><td colspan="2">Raspberry Pi x1
GPIO Extension Board & Ribbon Cable x1
Breadboard x1</td><td colspan="3">Jumper Wire M/M x17
  </td></tr>
<tr><td>Rotary Potentiometer x1
  </td><td>ADC Module x1 （Only one）
 

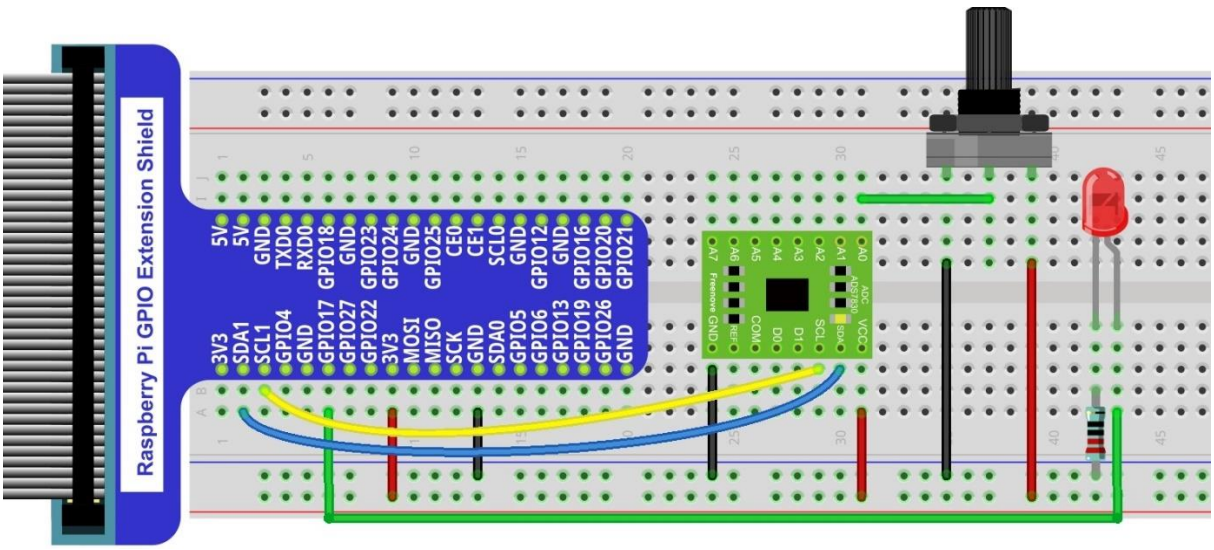
or

</td><td>10kΩ x2
  </td><td>220Ω x1
  </td><td>LED x1
  </td></tr>
</table>

Circuit with ADS7830

Schematic diagram



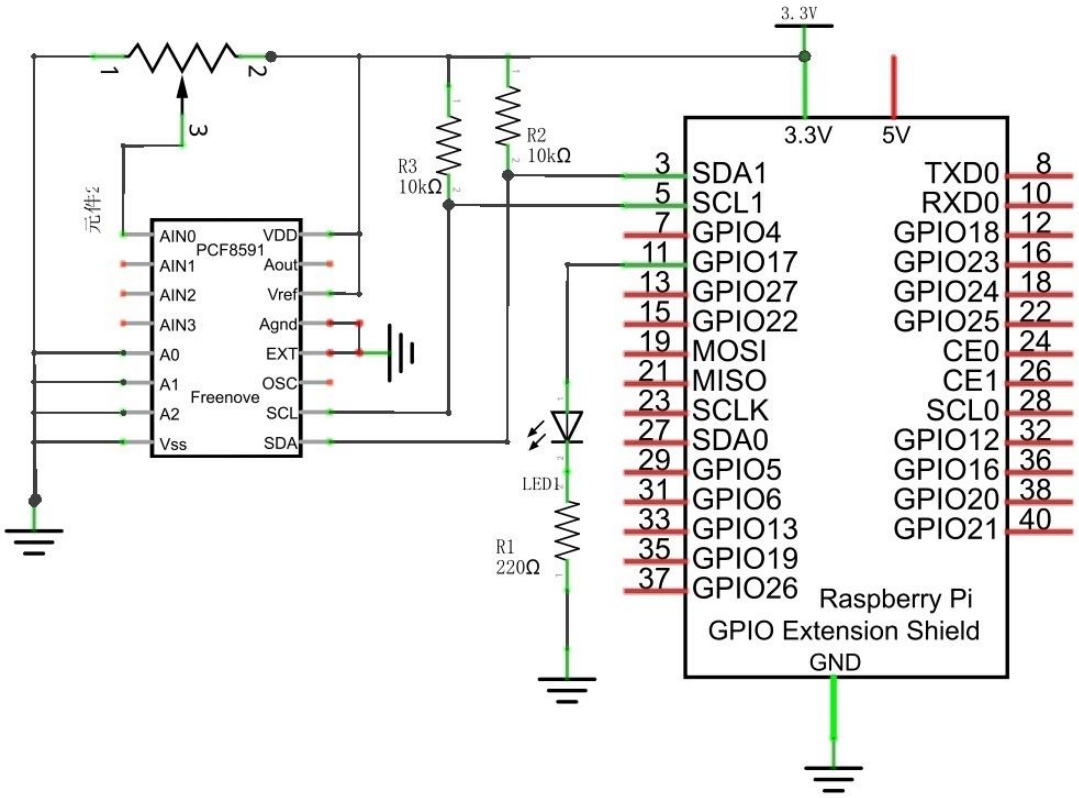
Hardware connection. If you need any support, please feel free to contact us via: support@freenove.com



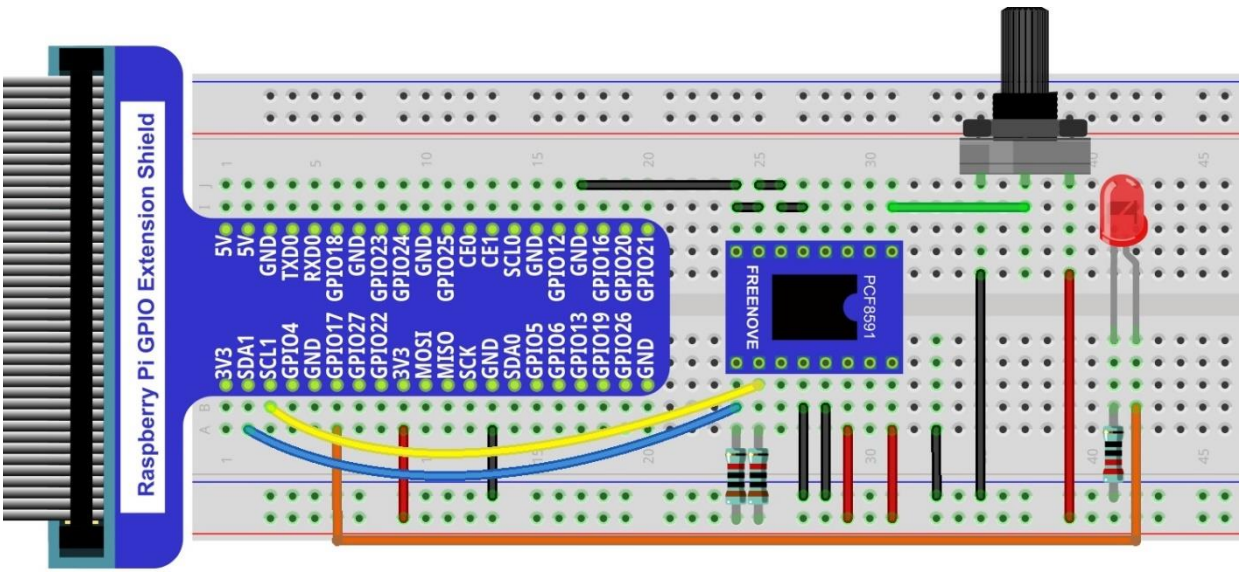
Video: https://youtu.be/YMEfe9IWU6I

Circuit with PCF8591

Schematic diagram



Hardware connection



Code

Python Code 8.1.1 Softlight

If you did not configure I2C, please refer to Chapter 7. If you did, please continue.
First, observe the project result, and then learn about the code in detail.

If you have any concerns, please contact us via: support@freenove.com

1. Use cd command to enter 08.1.1_Softlight directory of Python code

```
cd ~/Freenove_Kit/Code/Python_GPIOZero_Code/08.1.1_Softlight
```

2. Use the python command to execute the Python code "Softlight.py".

```
python Softlight.py
```

After the program is executed, adjusting the potentiometer will display the voltage values of the potentiometer in the Terminal window and the converted digital quantity. As a consequence, the brightness of LED will be changed.

The following is the code:

```
1  from gpiozero import PWMLED
2  import time
3  from ADCDevice import *
4
5  led = PWMLED(17,frequency=1000)       # define LED pin according to BCM Numbering
6  adc = ADCDevice() # Define an ADCDevice class object
7
8  def setup():
9      global adc
10     if(adc.detectI2C(0x48)): # Detect the pcf8591.
11         adc = PCF8591()
12     elif(adc.detectI2C(0x4b)): # Detect the ads7830
13         adc = ADS7830()
14     else:
15         print("No correct I2C address found, \n"
16         "Please use command 'i2cdetect -y 1' to check the I2C address! \n"
17         "Program Exit. \n");
18         exit(-1)
19
20 def loop():
21     while True:
22         value = adc.analogRead(0)       # read the ADC value of channel 0
23         led.value = value / 255.0       # Mapping to PWM duty cycle
24         voltage = value / 255.0 * 3.3   # calculate the voltage value
25         print ('ADC Value : %d, Voltage : %.2f'%(value,voltage))
26         time.sleep(0.03)
```

```
27
28  def destroy():
29      led.close()
30      adc.close()
31
32  if __name__ == '__main__':   # Program entrance
33      print ('Program is starting ... ')
34      try:
35          setup()
36          loop()
37      except KeyboardInterrupt: # Press ctrl-c to end the program.
38          destroy()
39          print("Ending program")
```

In the code, read ADC value of potentiometers and map it to the duty cycle of the PWM to control LED brightness.

```
            value = adc.analogRead(0)       # read the ADC value of channel 0
            led.value = value / 255.0       # Mapping to PWM duty cycle
```

Chapter 9 Photoresistor & LED

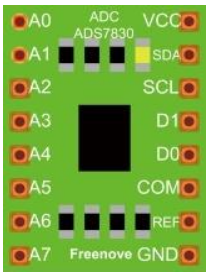
In this chapter, we will learn how to use a photoresistor to make an automatic dimming nightlight.

Project 9.1 NightLamp

A Photoresistor is very sensitive to the amount of light present. We can take advantage of the characteristic to make a nightlight with the following function. When the ambient light is less (darker environment), the LED will automatically become brighter to compensate and when the ambient light is greater (brighter environment) the LED will automatically dim to compensate.

Component List

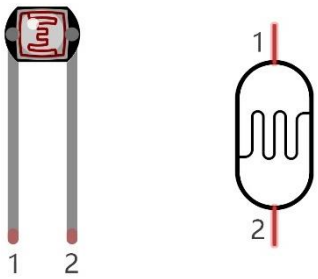
<table>
<tr><td colspan="3">Raspberry Pi x1
GPIO Extension Board & Ribbon Cable x1
Breadboard x1</td><td colspan="3">Jumper Wires M/M x15
  </td></tr>
<tr><td>Photoresistor x1
  </td><td colspan="2">ADC module x1
 

or

</td><td>10kΩ x3
  </td><td>220Ω x1
  </td><td>LED x1
  </td></tr>
</table>

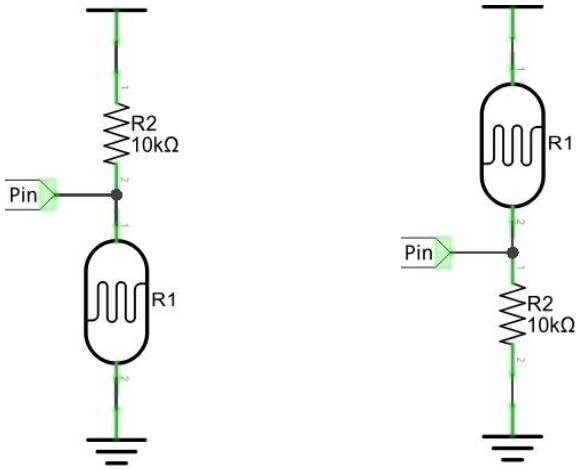
Component knowledge

Photoresistor

A Photoresistor is simply a light sensitive resistor. It is an active component that decreases resistance with respect to receiving luminosity (light) on the component's light sensitive surface. A Photoresistor's resistance value will change in proportion to the ambient light detected. With this characteristic, we can use a Photoresistor to detect light intensity. The Photoresistor and its electronic symbol are as follows.



The circuit below is used to detect the change of a Photoresistor's resistance value:

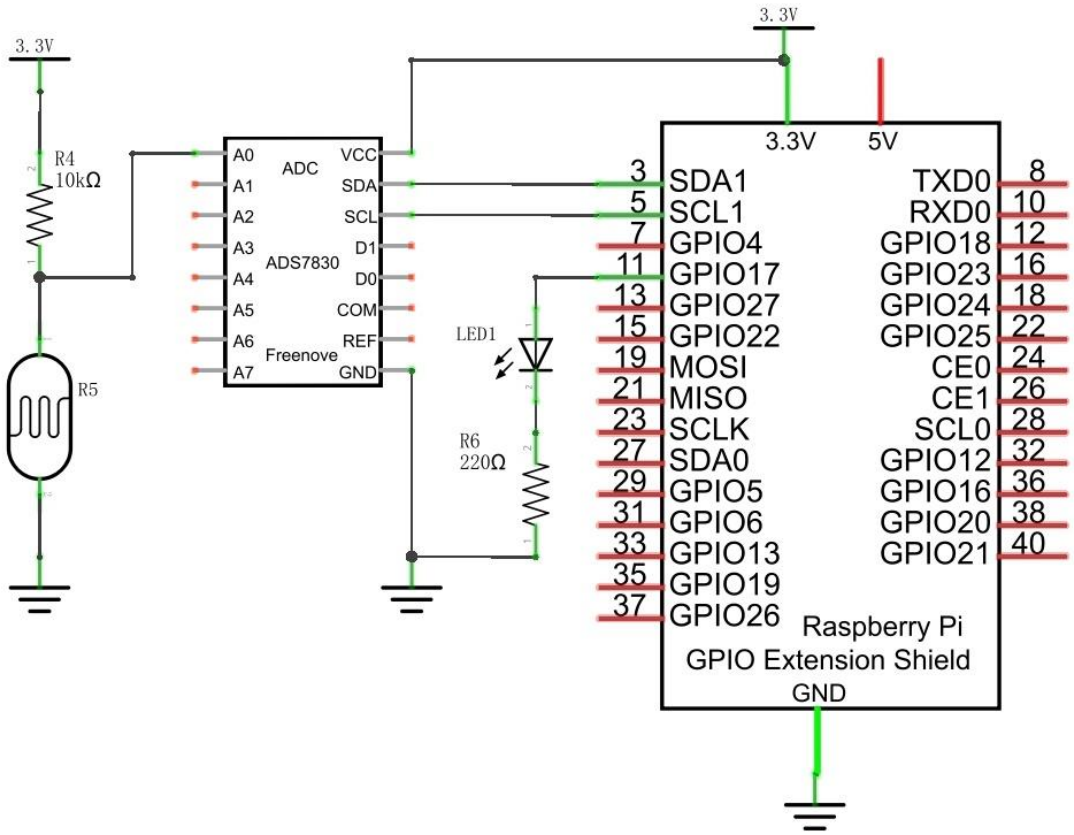


In the above circuit, when a Photoresistor's resistance vale changes due to a change in light intensity, the voltage between the Photoresistor and Resistor R1 will also change. Therefore, the intensity of the light can be obtained by measuring this voltage.

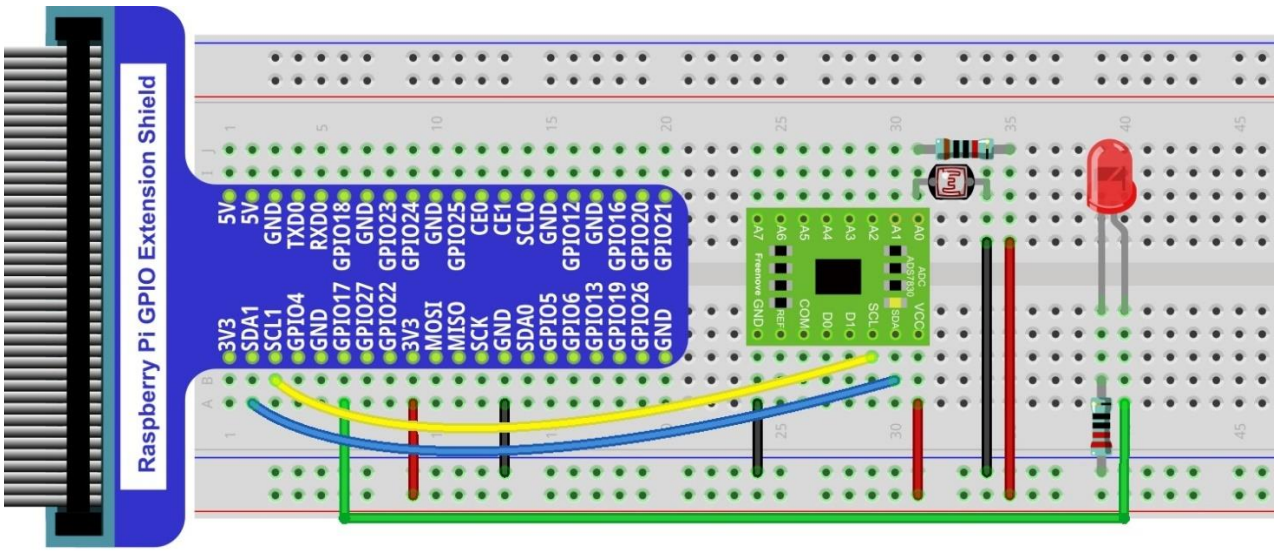
Circuit with ADS7830

The circuit used is similar to the Soft light project. The only difference is that the input signal of the AIN0 pin of ADC changes from a Potentiometer to a combination of a Photoresistor and a Resistor.

Schematic diagram



Hardware connection. If you need any support, please feel free to contact us via: support@freenove.com

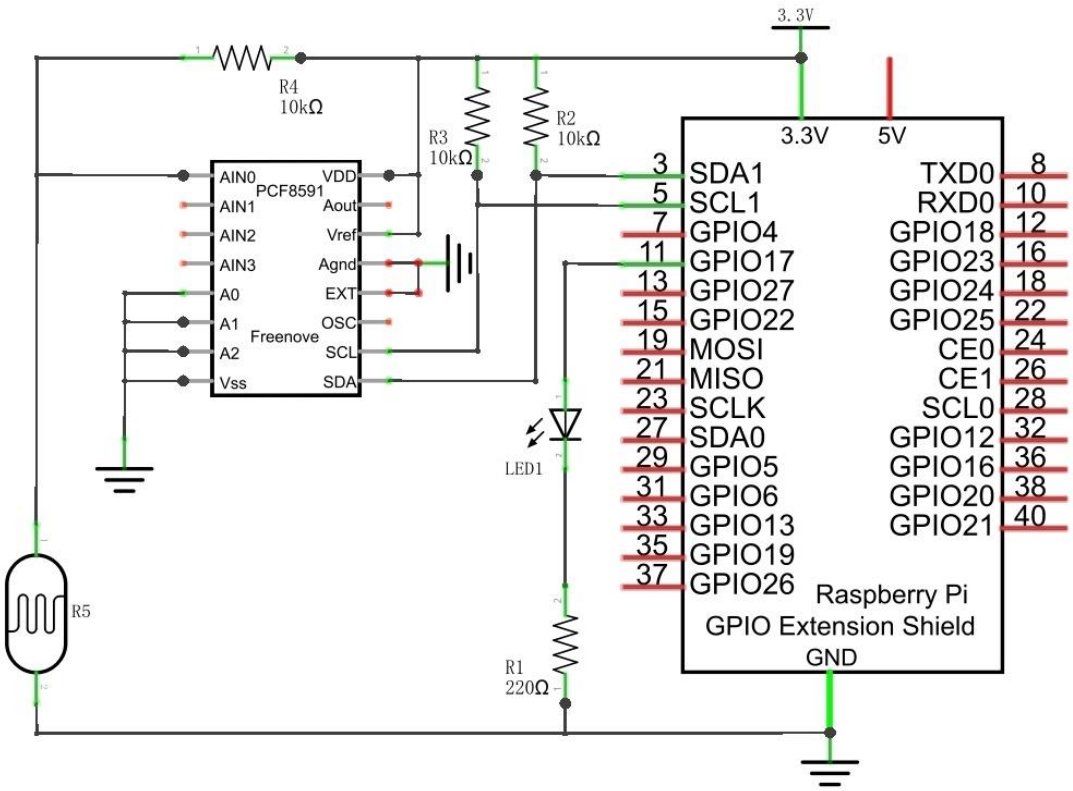


Video: https://youtu.be/r6p3zhXsyko

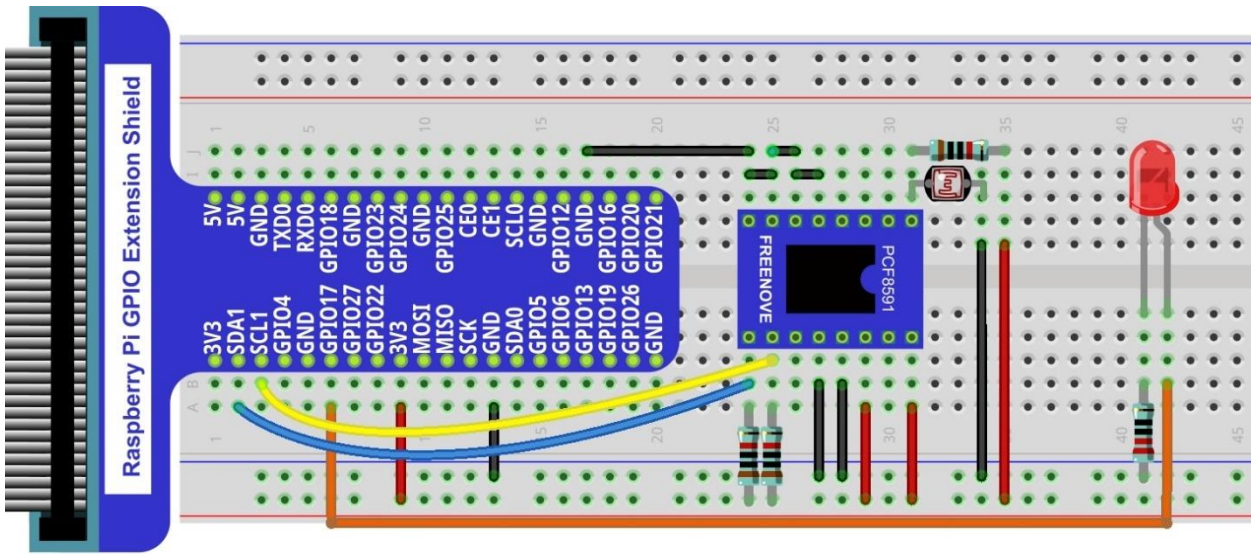
Circuit with PCF8591

The circuit used is similar to the Soft light project. The only difference is that the input signal of the AIN0 pin of ADC changes from a Potentiometer to a combination of a Photoresistor and a Resistor.

Schematic diagram



Hardware connection



Code

The code used in this project is identical with what was used in the last chapter.

Python Code 9.1.1 Nightlamp

If you did not configure I2C, please refer to Chapter 7. If you did, please continue.

First, observe the project result, and then learn about the code in detail.

If you have any concerns, please contact us via: support@freenove.com

1. Use cd command to enter 10.1_Nightlamp directory of Python code.

```
cd ~/Freenove_Kit/Code/Python_GPIOZero_Code/9.1.1_Nightlamp
```

2. Use the python command to execute the Python code "Nightlamp.py".

```
python Nightlamp.py
```

After the program is executed, if you cover the Photoresistor or increase the light shining on it, the brightness of the LED changes accordingly. As in previous projects the Terminal window will display the current input voltage value of ADC module A0 pin and the converted digital quantity.

The following is the program code:

```
from gpiozero import PWMLED
import time
from ADCDevice import *

ledPin = 17 # define ledPin
led = PWMLED(ledPin)
adc = ADCDevice() # Define an ADCDevice class object

def setup():
    global adc
    if(adc.detectI2C(0x48)): # Detect the pcf8591.
        adc = PCF8591()
    elif(adc.detectI2C(0x4b)): # Detect the ads7830
        adc = ADS7830()
    else:
        print("No correct I2C address found, \n"
        "Please use command 'i2cdetect -y 1' to check the I2C address! \n"
        "Program Exit. \n");
        exit(-1)

def loop():
    while True:
        value = adc.analogRead(0)     # read the ADC value of channel 0
        led.value = value / 255.0     # Mapping to PWM duty cycle
        voltage = value / 255.0 * 3.3
        print ('ADC Value : %d, Voltage : %.2f'%(value,voltage))
        time.sleep(0.01)
```

```
28
29  def destroy():
30      led.close()
31      adc.close()
32
33  if __name__ == '__main__':   # Program entrance
34      print ('Program is starting ... ')
35      setup()
36      try:
37          loop()
38      except KeyboardInterrupt:  # Press ctrl-c to end the program.
39          destroy()
40          print("Ending program")
```

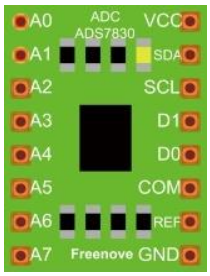
Chapter 10 Thermistor

In this chapter, we will learn about Thermistors which are another kind of Resistor.

Project 10.1 Thermometer

A Thermistoris a type of Resistor whose resistance value is dependent on temperature and changes in temperature. Therefore, we can take advantage of this characteristic to make a Thermometer.

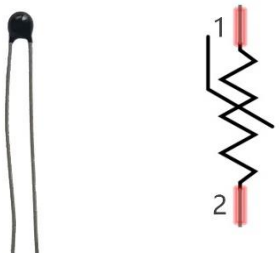
Component List

Raspberry Pi x1 GPIO Extension Board & Ribbon Cable x1 Breadboard x1	Jumper Wire M/M x14 	
Thermistor x1 	ADC module x1  or 	Resistor 10kΩ x3 

Component knowledge

Thermistor

Thermistor is a temperature sensitive resistor. When it senses a change in temperature, the resistance of the Thermistor will change. We can take advantage of this characteristic by using a Thermistor to detect temperature intensity. A Thermistor and its electronic symbol are shown below.



The relationship between resistance value and temperature of a thermistor is:

Rt=R*EXP [B*(1/T2-1/T1)]

Where:

Rt is the thermistor resistance under T2 temperature;

R is in the nominal resistance of thermistor under T1 temperature;

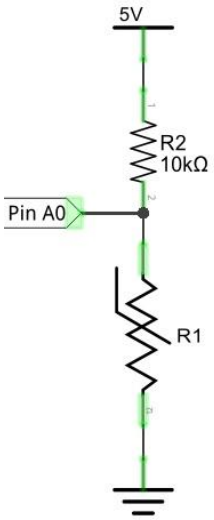
EXP[n] is nth power of e;

B is for thermal index;

T1, T2 is Kelvin temperature (absolute temperature). Kelvin temperature=273.15 + Celsius temperature.

For the parameters of the Thermistor, we use: B=3950, R=10k, T1=25.

The circuit connection method of the Thermistor is similar to photoresistor, as the following:



We can use the value measured by the ADC converter to obtain the resistance value of Thermistor, and then we can use the formula to obtain the temperature value.

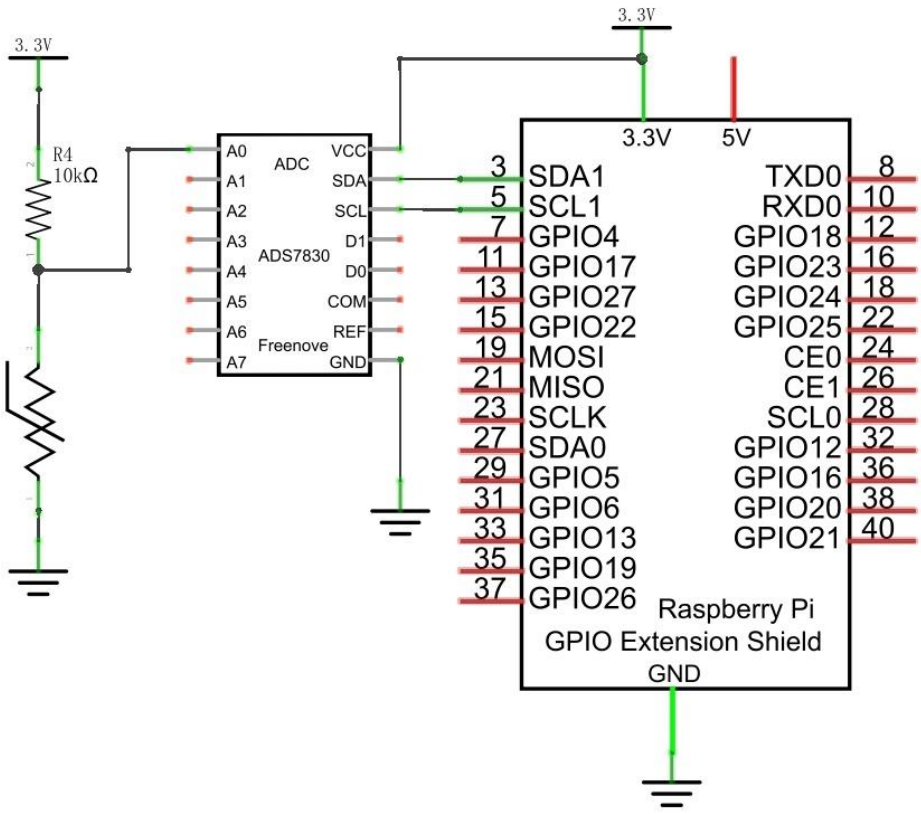
Therefore, the temperature formula can be derived as:

T2 = 1/(1/T1 + ln(Rt/R)/B)

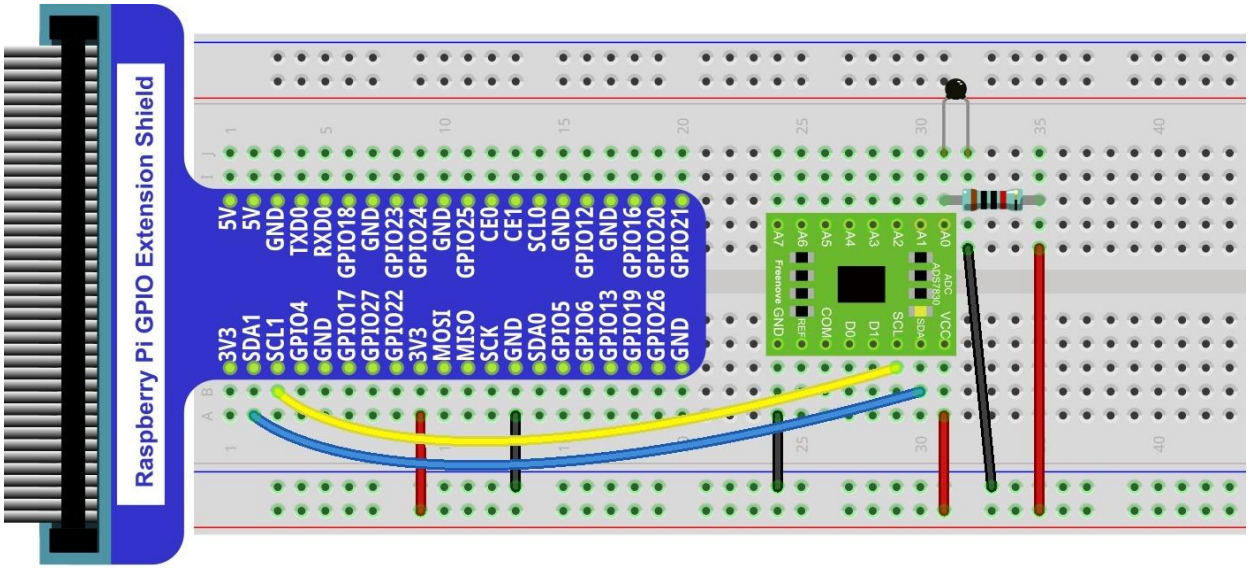
Circuit with ADS7830

The circuit of this project is similar to the one in last chapter. The only difference is that the Photoresistor is replaced by the Thermistor.

Schematic diagram



Hardware connection. If you need any support, please feel free to contact us via: support@freenove.com



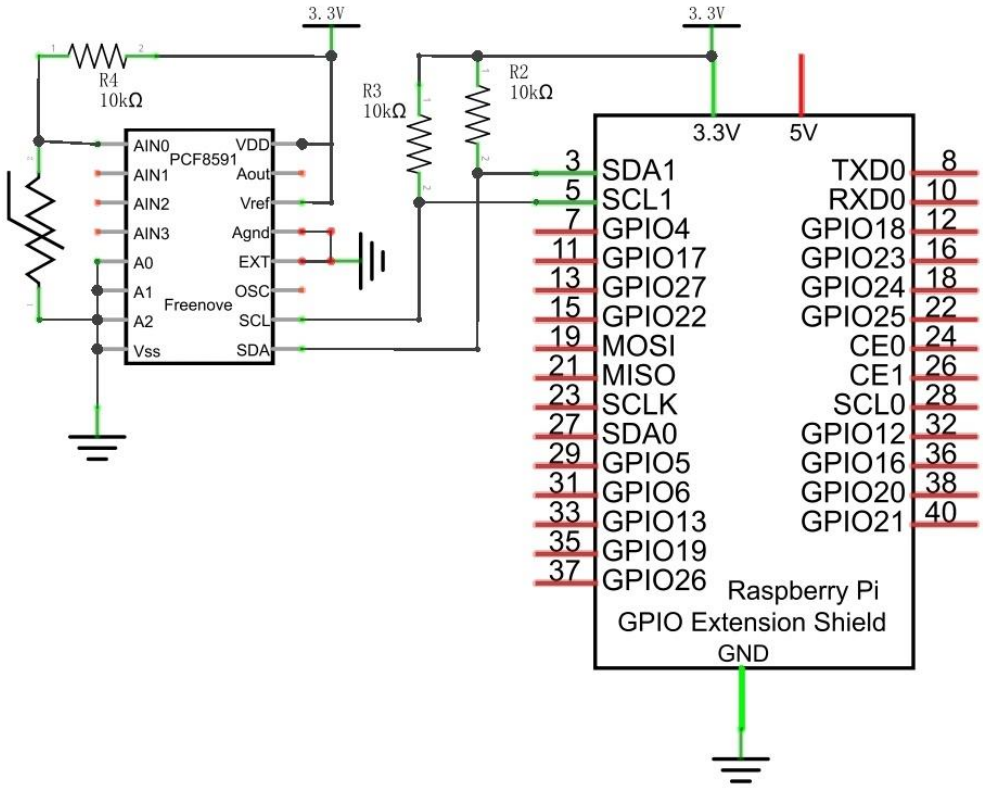
Thermistor has longer pins than the one shown in circuit.

Video: https://youtu.be/spOaIxanNMc

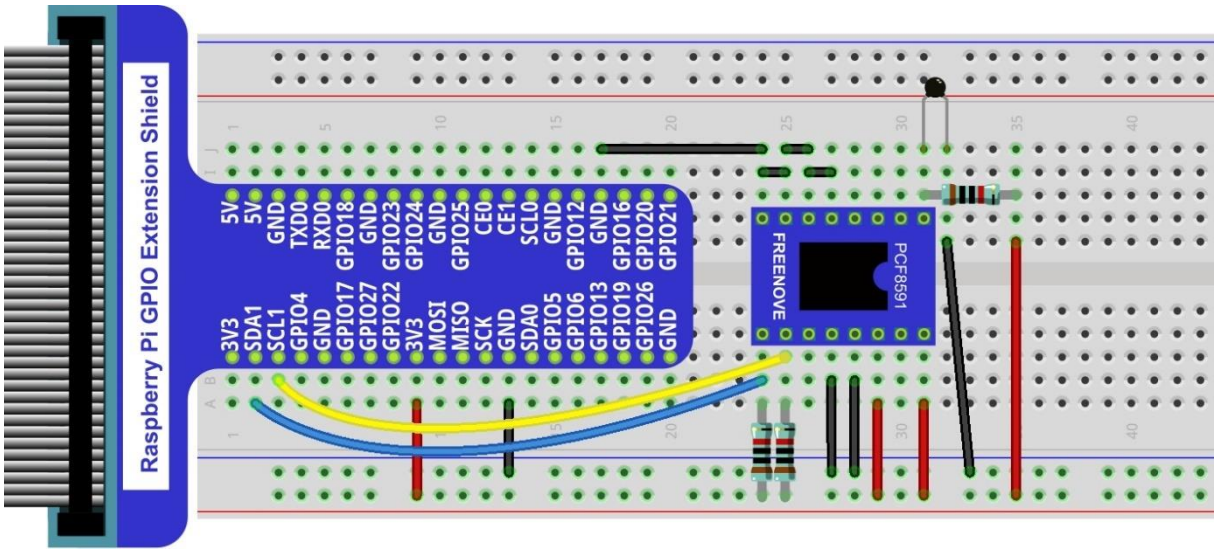
Circuit with PCF8591

The circuit of this project is similar to the one in the last chapter. The only difference is that the Photoresistor is replaced by the Thermistor.

Schematic diagram



Hardware connection. If you need any support, please feel free to contact us via: support@freenove.com



Thermistor has longer pins than the one shown in circuit.

Code

In this project code, the ADC value still needs to be read, but the difference here is that a specific formula is used to calculate the temperature value.

Python Code 10.1.1 Thermometer

If you did not configure I2C, please refer to Chapter 7. If you did, please continue.
First, observe the project result, and then learn about the code in detail.

If you have any concerns, please contact us via: support@freenove.com

1. Use cd command to enter 10.1.1_Thermometer directory of Python code.

```
cd ~/Freenove_Kit/Code/Python_GPIOZero_Code/10.1.1_Thermometer
```

2. Use python command to execute Python code "Thermometer.py".

```
python Thermometer.py
```

After the program is executed, the Terminal window will display the current ADC value, voltage value and temperature value. Try to "pinch" the thermistor (without touching the leads) with your index finger and thumb for a brief time, you should see that the temperature value increases.

```
ADC Value : 107, Voltage : 1.38, Temperature : 32.48
ADC Value : 107, Voltage : 1.38, Temperature : 32.48
ADC Value : 107, Voltage : 1.38, Temperature : 32.48
ADC Value : 107, Voltage : 1.38, Temperature : 32.48
ADC Value : 107, Voltage : 1.38, Temperature : 32.48
ADC Value : 107, Voltage : 1.38, Temperature : 32.48
ADC Value : 107, Voltage : 1.38, Temperature : 32.48
ADC Value : 107, Voltage : 1.38, Temperature : 32.48
ADC Value : 107, Voltage : 1.38, Temperature : 32.48
ADC Value : 107, Voltage : 1.38, Temperature : 32.48
ADC Value : 107, Voltage : 1.38, Temperature : 32.48
ADC Value : 107, Voltage : 1.38, Temperature : 32.48
ADC Value : 107, Voltage : 1.38, Temperature : 32.48
ADC Value : 107, Voltage : 1.38, Temperature : 32.48
```

The following is the code:

```
1  import time
2  import math
3  from ADCDevice import *
4
5  adc = ADCDevice() # Define an ADCDevice class object
6
7  def setup():
8      global adc
9      if(adc.detectI2C(0x48)): # Detect the pcf8591.
10         adc = PCF8591()
11     elif(adc.detectI2C(0x4b)): # Detect the ads7830
12         adc = ADS7830()
13     else:
```

```
14          print("No correct I2C address found, \n"
15          "Please use command 'i2cdetect -y 1' to check the I2C address! \n"
16          "Program Exit. \n");
17          exit(-1)
18
19  def loop():
20      while True:
21          value = adc.analogRead(0)         # read ADC value A0 pin
22          voltage = value / 255.0 * 3.3         # calculate voltage
23          Rt = 10 * voltage / (3.3 - voltage)    # calculate resistance value of thermistor
24          tempK = 1/(1/(273.15 + 25) + math.log(Rt/10)/3950.0) # calculate temperature (Kelvin)
25          tempC = tempK -273.15         # calculate temperature (Celsius)
26          print ('ADC Value : %d, Voltage : %.2f, Temperature : %.2f'%(value,voltage,tempC))
27          time.sleep(0.01)
28
29  def destroy():
30      adc.close()
31
32  if __name__ == '__main__':  # Program entrance
33      print ('Program is starting ... ')
34      setup()
35      try:
36          loop()
37      except KeyboardInterrupt: # Press ctrl-c to end the program.
38          destroy()
39          print("Ending program")
```

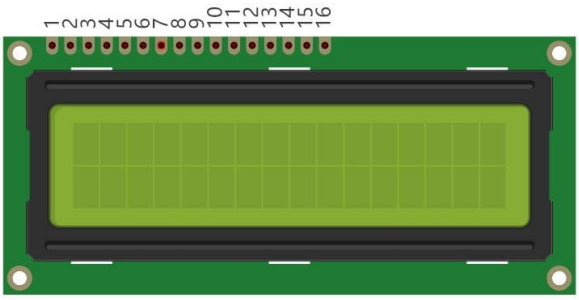
In the code, the ADC value of ADC module A0 port is read, and then calculates the voltage and the resistance of Thermistor according to Ohms Law. Finally, it calculates the temperature sensed by the Thermistor, according to the formula.

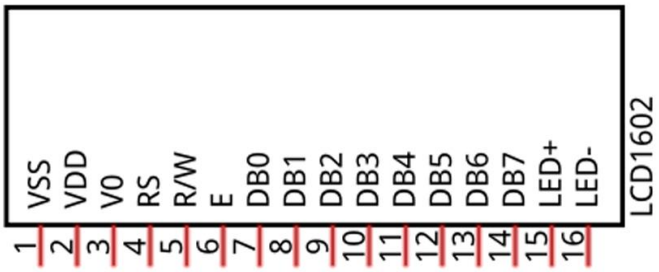
Chapter 11 LCD1602

In this chapter, we will learn about the LCD1602 Display Screen,

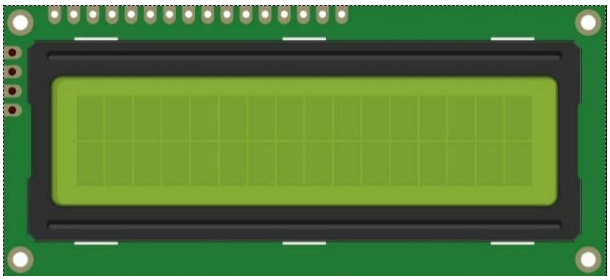
Project 11.1 I2C LCD1602

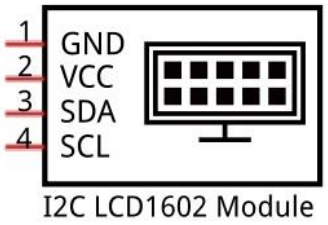
There are LCD1602 display screen and the I2C LCD. We will introduce both of them in this chapter. But what we use in this project is an I2C LCD1602 display screen. The LCD1602 Display Screen can display 2 lines of characters in 16 columns. It is capable of displaying numbers, letters, symbols, ASCII code and so on. As shown below is a monochrome LCD1602 Display Screen along with its circuit pin diagram





I2C LCD1602 Display Screen integrates a I2C interface, which connects the serial-input & parallel-output module to the LCD1602 Display Screen. This allows us to only use 4 lines to operate the LCD1602.

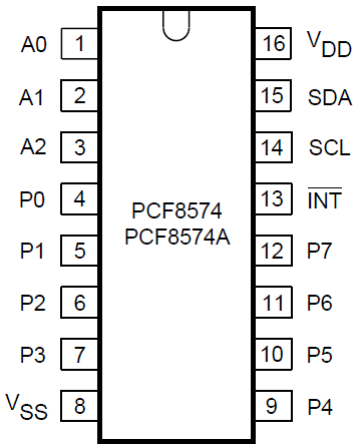




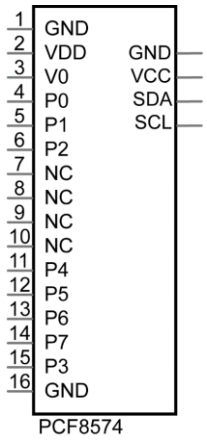
The serial-to-parallel IC chip used in this module is PCF8574T (PCF8574AT), and its default I2C address is 0x27(0x3F). You can also view the RPI bus on your I2C device address through command "i2cdetect -y 1" (refer to the "configuration I2C" section below).

Below is the PCF8574 chip pin diagram and its module pin diagram:

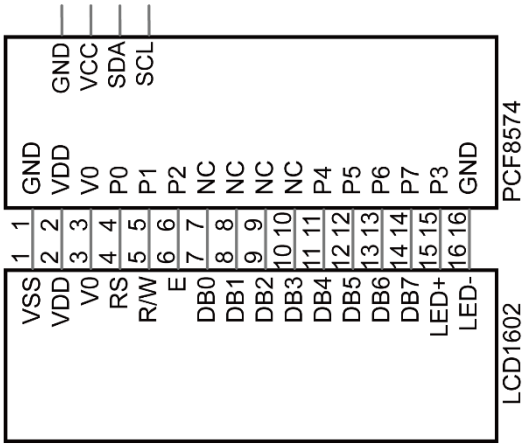
PCF8574 chip pin diagram:



PCF8574 module pin diagram



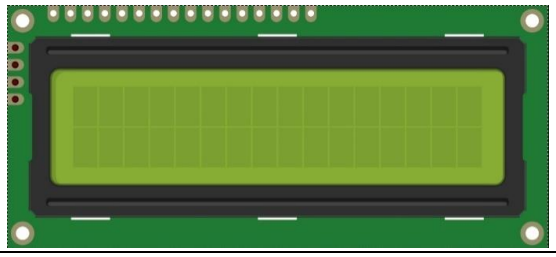
PCF8574 module pins and LCD1602 pins correspond to each other and connected to each other:



Because of this, as stated earlier, we only need 4 pins to control the16 pins of the LCD1602 Display Screen through the I2C interface.

In this project, we will use the I2C LCD1602 to display some static characters and dynamic variables.

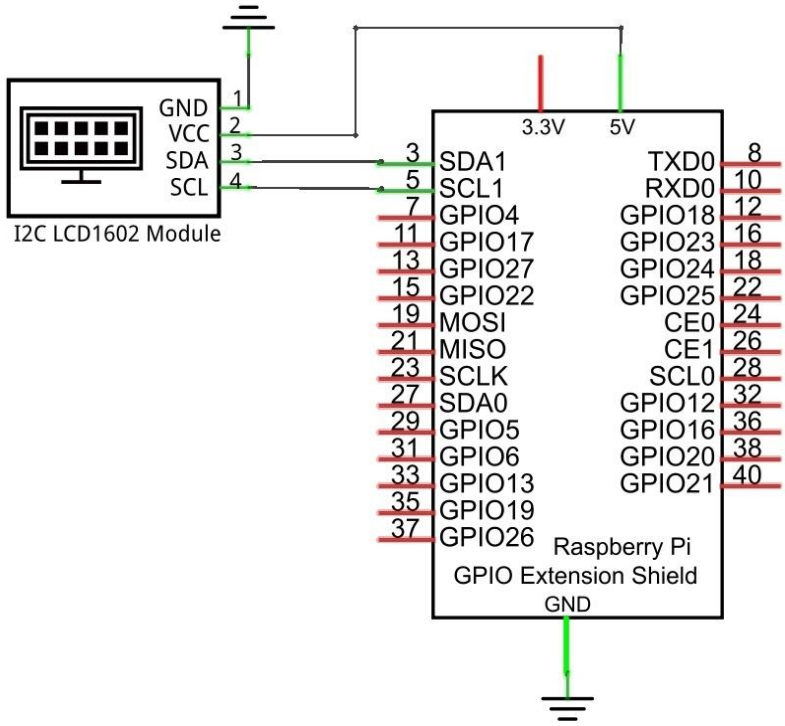
Component List

Raspberry Pi (with 40 GPIO) x1 GPIO Extension Board & Ribbon Cable x1 Breadboard x1	Jumper Wire x4 
I2C LCD1602 Module x1 	

Circuit

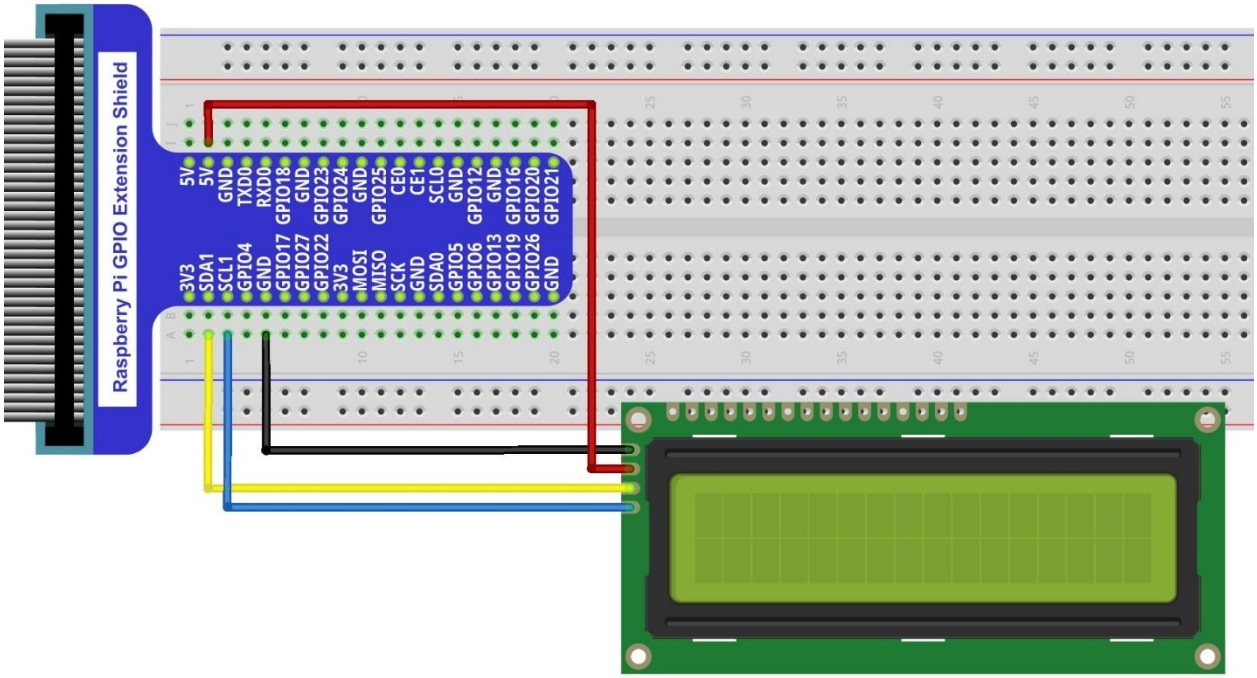
Note that the power supply for I2C LCD1602 in this circuit is 5V.

Schematic diagram



Hardware connection. If you need any support, please feel free to contact us via: support@freenove.com

NOTE: It is necessary to configure 12C and install Smbus first (see chapter 7 for details)



Video: https://www.youtube.com/watch?v=L1bgX2f4Yio

Code

This code will have your RPi's CPU temperature and System Time Displayed on the LCD1602.

Python Code 11.1.1 I2CLCD1602

If you did not configure I2C and install Smbus, please refer to Chapter 7. If you did, continue.

First, observe the project result, and then learn about the code in detail.

If you have any concerns, please contact us via: support@freenove.com

1. Use cd command to enter 11.1.1_ I2CLCD1602 directory of Python code.

```
cd ~/Freenove_Kit/Code/Python_GPIOZero_Code/11.1.1_I2CLCD1602
```

2. Use Python command to execute Python code "I2CLCD1602.py".

```
python I2CLCD1602.py
```

After the program is executed, the LCD1602 Screen will display your RPi's CPU Temperature and System Time.

So far, at this writing, we have two types of LCD1602 on sale. One needs to adjust the backlight, and the other does not.

The LCD1602 that does not need to adjust the backlight is shown in the figure below.

The following is the program code:

```
1  import smbus
2  from time import sleep, strftime
3  from datetime import datetime
4  from LCD1602 import CharLCD1602
5
6  lcd1602 = CharLCD1602()
7  def get_cpu_temp():     # get CPU temperature from file
8  "/sys/class/thermal/thermal_zone0/temp"
9      tmp = open('/sys/class/thermal/thermal_zone0/temp')
10     cpu = tmp.read()
11     tmp.close()
12     return '{:.2f}'.format( float(cpu)/1000 ) + ' C '
13
14 def get_time_now():     # get system time
15     return datetime.now().strftime('    %H:%M:%S')
16
17 def loop():
18     lcd1602.init_lcd()
19     count = 0
```

```
20      while(True):
21          lcd1602.clear()
22          lcd1602.write(0, 0, 'CPU: ' + get_cpu_temp() )# display CPU temperature
23          lcd1602.write(0, 1, get_time_now() )   # display the time
24          sleep(1)
25  def destroy():
26      lcd1602.clear()
27  if __name__ == '__main__':
28      print ('Program is starting ... ')
29      try:
30          loop()
31      except KeyboardInterrupt:
32          destroy()
```

In a while loop, set the cursor position, and display the CPU temperature and time.

```
        while(True):
            lcd1602.clear()
            lcd1602.write(0, 0, 'CPU: ' + get_cpu_temp() )# display CPU temperature
            lcd1602.write(0, 1, get_time_now() )   # display the time
            sleep(1)
```

CPU temperature is stored in file "/sys/class/thermal/thermal_zone0/temp". Open the file and read content of the file, and then convert it to Celsius degrees and return. Subfunction used to get CPU temperature is shown below:

```
def get_cpu_temp():     # get CPU temperature and store it into file
  "/sys/class/thermal/thermal_zone0/temp"
    tmp = open('/sys/class/thermal/thermal_zone0/temp')
    cpu = tmp.read()
    tmp.close()
    return '{:.2f}'.format( float(cpu)/1000 ) + ' C'
```

Subfunction used to get time:

```
def get_time_now():     # get the time
    return datetime.now().strftime('     %H:%M:%S')
```

Details about LCD1602.py:

Module LCD1602

This module provides the basic operation method of LCD1602, including class CharLCD1602.
Some member functions are described as follows:

def init_lcd(self,addr=None, bl=1) : LDC1602 initializes the setting. When the addr is None, the I2C address of the device will be automatically scanned. You can also specify the I2C address, bl=1 to enable the backlight setting.

def clear(self): clear the screen

def send_command(self,comm): set the cursor position

def i2c_scan(self): scan the device I2C address

def write(self,x, y, str): display contents

More information can be viewed through opening LCD1602.py.

Chapter 12 Web IoT

In this chapter, we will learn how to use GPIO to control the RPi remotely via a network and how to build a WebIO service on the RPi.
This concept is known as “IoT” or Internet of Things. The development of IoT will greatly change our habits and make our lives more convenient and efficient

Project 12.1 Remote LED

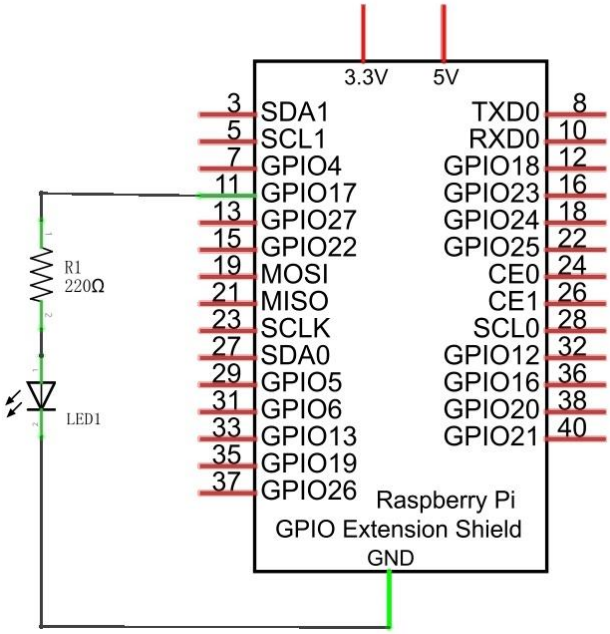
In this project, we need to build a WebIOPi service, and then use the RPi GPIO to control an LED through the web browser of phone or PC.

Component List

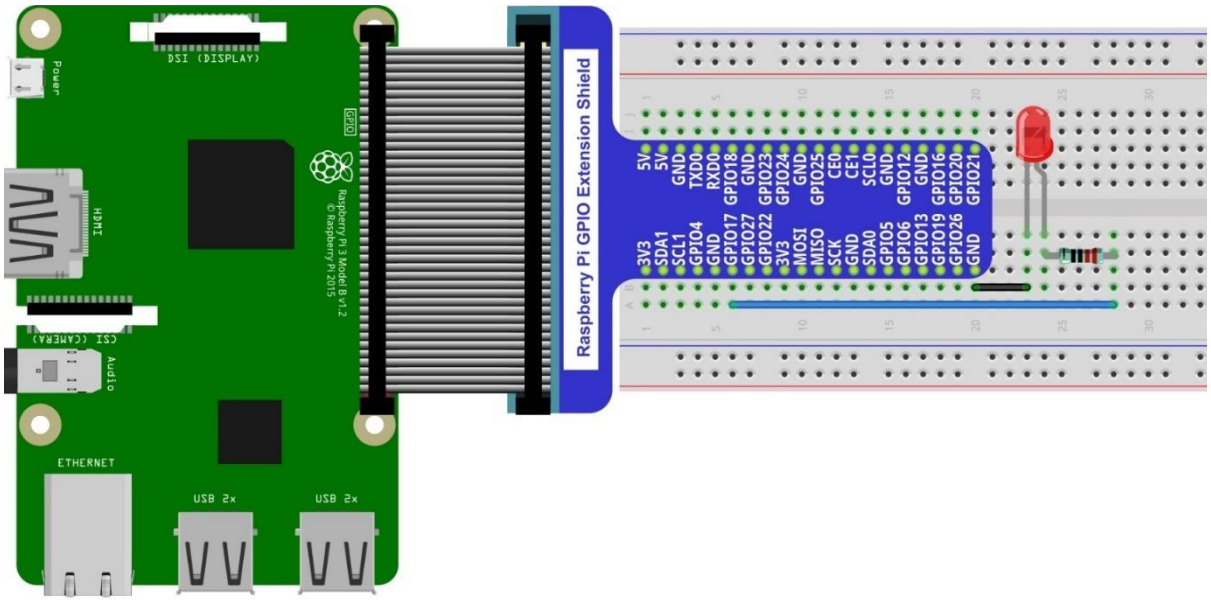
Raspberry Pi (with 40 GPIO) x1 GPIO Extension Board & Ribbon Cable x1 Breadboard x1	LED x1 	Resistor 220Ω x1 
Jumper M/M x2 		

Circuit

Schematic diagram



Hardware connection. If you need any support, please feel free to contact us via: support@freenove.com



Solution from E-Tinkers

Here is a solution fromblog E-Tinkers, author Henry Cheung. For more details, please refer to link below:
https://www.e-tinkers.com/2018/04/how-to-control-raspberry-pi-gpio-via-http-web-server/

1, Make sure you have set python3 as default python. Then run following command in terminal to install http.server in your Raspberry Pi.

```
sudo apt-get install http.server
```

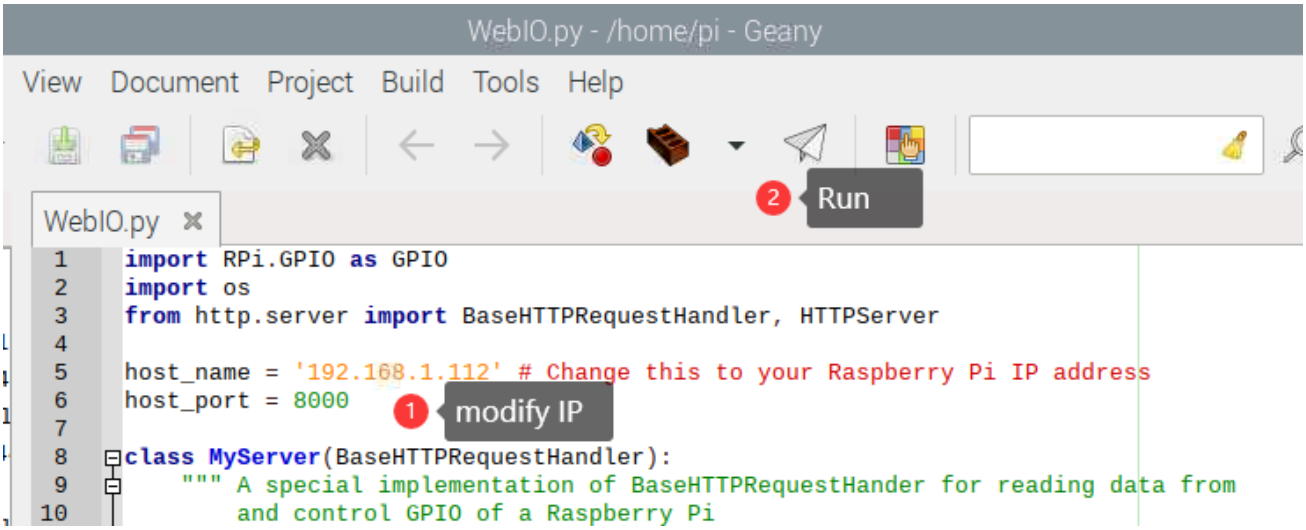
2, Open WebIO.py

```
cd ~/Freenove_Kit/Code/Python_GPIOZero_Code/12.1.1_WebIO
geany WebIO.py
```

3, Change the host_name into your Raspberry Pi IP address.

```
host_name = '192.168.1.112'     # Change this to your Raspberry Pi IP address
```

Then run the code WebIO.py



3, Visit http://192.168.1.112:8000/ in web brower on compter under local area networks. **Change IP to your Raspberry Pi IP address.**

WebIOPi Service Framework

Note: **If you have a Raspberry Pi 4B, you may have some trouble. The reason for changing the file in the configuration process is that the newer generation models of the RPi CPUs are different form the older ones and you may not be able to access the GPIO Header at the end of this tutorial.** A solution to this is given in an online tutorial by from E-Tinkers blogger Henry Cheung. For more details, please refer to previouse section.

The following is the key part of this chapter. The installation steps refer to WebIOPi official. And you also can directly refer to the official installation steps. The latest version (in 2016-6-27) of WebIOPi is 0.7.1. So, you may encounter some issues in using it. We will explain these issues and provide the solution in the following installation steps.

Here are the steps to build a WebIOPi:

Installation

1. Get the installation package. You can use the following command to obtain.

```
wget https://github.com/Freenove/WebIOPi/archive/master.zip -O WebIOPi.zip
```

2. Extract the package and generate a folder named "WebIOPi-master". Then enter the folder.

```
unzip WebIOPi.zip
cd WebIOPi-master/WebIOPi-0.7.1
```

3. Patch for Raspberry Pi B+, 2B, 3B, 3B+.

```
patch -p1 -i webiopi-pi2bplus.patch
```

4. Run setup.sh to start the installation, the process takes a while and you will need to be patient.

```
sudo ./setup.sh
```

5. If setup.sh does not have permission to execute, execute the following command

```
sudo sh ./setup.sh
```

Run

After the installation is completed, you can use the webiopi command to start running.

```
$ sudo webiopi [-h] [-c config] [-l log] [-s script] [-d] [port]

    Options:
      -h, --help            Display this help
      -c, --config  file    Load config from file
      -l, --log     file    Log to file
      -s, --script  file    Load script from file
      -d, --debug           Enable DEBUG

    Arguments:
      port                  Port to bind the HTTP Server
```

Run webiopi with verbose output and the default config file:

```
sudo webiopi -d -c /etc/webiopi/config
```

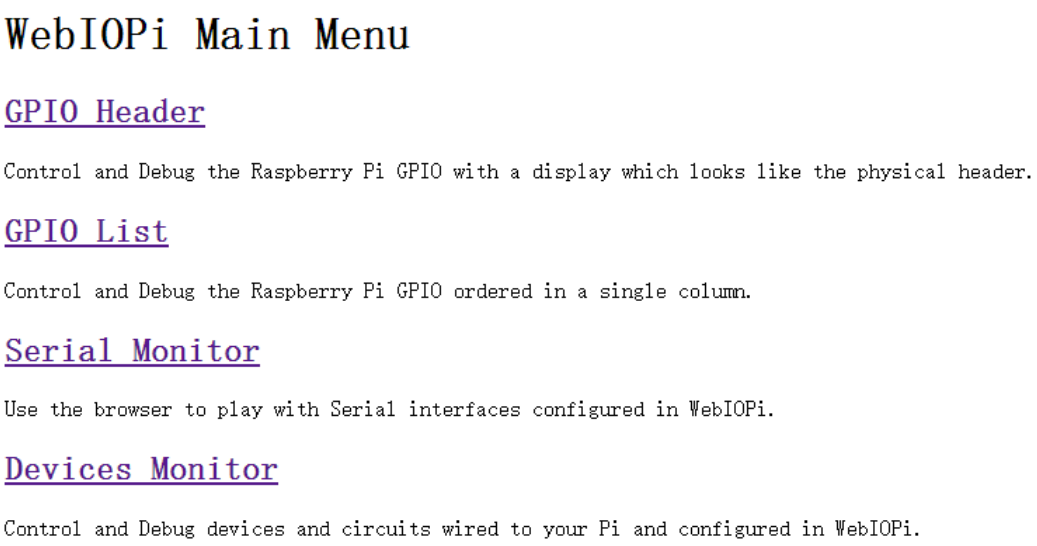
The Port is 8000 in default. Now WebIOPi has been launched. Keep it running.

Access WebIOPi over local network

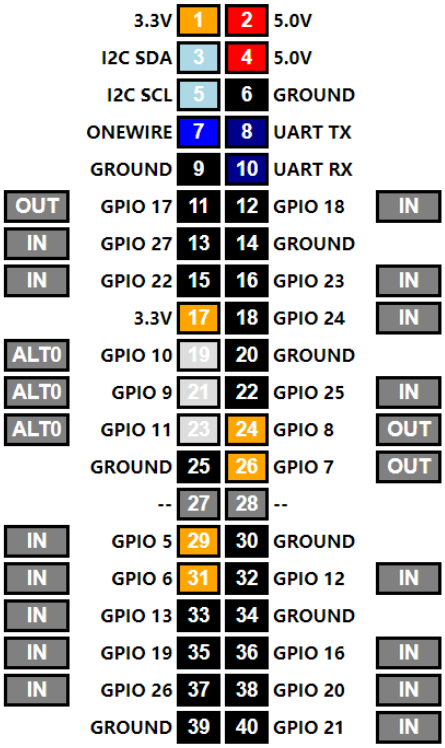
Under the same network, use a mobile phone or PC browser to open your RPi IP address, and add a port number like 8000. For example, my personal Raspberry Pi IP address is 192.168.1.109. Then, in the browser, I then should input: http://192.168.1.109:8000/

Default user is "webiopi" and password is "raspberry"。

Then, enter the main control interface:



Click on GPIO Header to enter the GPIO control interface.



Control methods:

- Click/Tap the OUT/IN button to change GPIO direction.
- Click/Tap pins to change the GPIO output state.

Completed

According to the circuit we build, set GPIO17 to OUT, then click Header11 to control the LED.
You can end the webioPi in the terminal by "Ctr+C".

What's Next?

THANK YOU for participating in this learning experience! If you have completed all of the projects successfully you can consider yourself a Raspberry Pi Master.

We have reached the end of this Tutorial. If you find errors, omissions or you have suggestions and/or questions about the Tutorial or component contents of this Kit, please feel free to contact us: **support@freenove.com**
We will make every effort to make changes and correct errors as soon as feasibly possible and publish a revised version.

If you are interesting in processing, you can study the Processing.pdf in the unzipped folder.

If you want to learn more about Arduino, Raspberry Pi, Smart Cars, Robotics and other interesting products in science and technology, please continue to visit our website. We will continue to launch fun, cost-effective, innovative and exciting products.
http://www.freenove.com/

Thank you again for choosing Freenove products.